From Vibes to Variables: A Field Guide to Open Media Science

Building Reproducible Research with R, Quarto, and the Science of Storytelling

SIM Lab, SIUE

2026-01-29

Table of contents

From Vibes to Variables 14

Welcome to From Vibes to Variables 15
- What You'll Learn . . . 15
- Who This Book Is For . . . 15
- How to Use This Book . . . 16
- What Makes This Different . . . 16

The Science of Storytelling 17
- Learning Objectives . . . 17
- Overview . . . 17
  - The Neuroscience of Narrative: Why Humans Tell Stories . . . 17
  - The Sacred Flaw: Hypotheses as Narrative Tension . . . 18
  - Mapping Narrative Structure onto Research Design . . . 18
  - Case Study: "Twitch in the Time of Quarantine" . . . 20
  - Anecdote vs. Data: The Difference Between Journalism and Science . . . 20
  - The Journalist-Scientist Identity . . . 21
  - The Journey Ahead . . . 21
- Practice: Applying the Narrative Framework . . . 22
  - Exercise 1.1: Identifying Narrative Elements in Research . . . 22
  - Exercise 1.2: From Anecdote to Research Question . . . 22
  - Exercise 1.3: Distinguishing Journalism from Science . . . 23
- Reflection Questions . . . 23
- Chapter Summary . . . 24
- Key Terms . . . 24
- Looking Ahead . . . 24

The Publishing Engine 25
- Learning Objectives . . . 25
- Overview . . . 25
  - The Replication Crisis: When Science Broke Its Promise . . . 25
  - The Fragility of Point-and-Click Workflows . . . 25
  - Code as Recipe: The Literate Programming Solution . . . 26
  - The Five Layers of Reproducible Research . . . 26
  - Why Reproducibility Matters Beyond Academia . . . 27

The Tools of the Publishing Engine . . . 28
The One-Click Report Philosophy . . . 29
Computational Literacy as Media Literacy . . . 30
Overcoming Code Anxiety . . . 30
The Ecosystem Beyond This Course . . . 30
Setting Up Your Publishing Engine . . . 31
Phase 1: Markdown and Note-Taking (Obsidian) . . . 31
Your First Markdown Document in Obsidian . . . 32
Markdown Essentials for Research Writing . . . 34
Debugging Common Errors . . . 40
Project Organization: Building a Research Vault . . . 41
Hands-On Practice: Markdown and Obsidian Skills . . . 42
Reflection Questions . . . 46
Chapter Summary . . . 46
Key Terms . . . 47
Looking Ahead . . . 47

The Scientific Approach **48**
Learning Objectives . . . 48
Overview . . . 48
Epistemology: The Problem of Knowledge . . . 48
Falsifiability: The Demarcation Problem . . . 50
Induction vs. Deduction: Two Modes of Reasoning . . . 50
Ethics in the Digital Age . . . 51
The IRB: Gatekeepers of Ethical Research . . . 51
Informed Consent in the Age of Big Data . . . 52
Inclusion and Representation: Ethics Beyond Harm Prevention . . . 52
The Sacred Trust of Science . . . 53
Navigating the IRB Process . . . 53
Understanding IRB Categories . . . 53
Writing an IRB Protocol . . . 54
Sample Consent Language . . . 54
Ethical Dilemmas in Digital Research . . . 55
Case Study 1: The Facebook Emotional Contagion Study (2014) . . . 55
Case Study 2: The OkCupid Dataset (2016) . . . 55
Case Study 3: The Grindr Location Data Leak (2018) . . . 56
Practice: Applying Ethical Reasoning . . . 56
Exercise 3.1: Categorizing Research Ethics . . . 56
Exercise 3.2: Designing Informed Consent . . . 56
Exercise 3.3: Epistemology in Action . . . 57
Reflection Questions . . . 57
Chapter Summary . . . 58
Key Terms . . . 58

Looking Ahead . . . 59

Intelligence Gathering **60**
Learning Objectives . . . 60
Overview . . . 60
The Archivist Mindset . . . 60
Mining the Archives . . . 61
Zotero: Your External Brain . . . 62
Real Data: The Complete Shakespeare Works . . . 63
The Search is Part of the Argument . . . 65
When to Stop Searching . . . 65
Case Study: Mapping Twitch Toxicity Research . . . 66
Practice: Developing Search Skills . . . 66
Exercise 4.1: Boolean Search Construction . . . 66
Exercise 4.2: Building Your Search Log . . . 67
Exercise 4.3: Zotero Practice . . . 67
Exercise 4.4: Saturation Assessment . . . 67
Reflection Questions . . . 68
Chapter Summary . . . 68
Key Terms . . . 69
Looking Ahead . . . 69

Theory as a Lens **70**
Learning Objectives . . . 70
Overview . . . 70
The Lens Metaphor . . . 70
Levels of Theory . . . 71
Theories Relevant to Mass Communication & SIM Lab Research . . . 71
Real Data: Stack Overflow Developer Survey 2024 . . . 74
From Theory to Hypothesis . . . 75
Variables: The Building Blocks of Hypotheses . . . 76
Thinking Like a Theorist . . . 76
Case Study: Applying Theory to a Research Question . . . 77
Practice: Applying Theoretical Frameworks . . . 77
Exercise 5.1: Identifying Variables . . . 77
Exercise 5.2: Theory Selection . . . 78
Exercise 5.3: From Theory to Hypothesis . . . 79
Exercise 5.4: Mediator vs. Moderator . . . 79
Reflection Questions . . . 79
Chapter Summary . . . 80
Key Terms . . . 80
Looking Ahead . . . 81

The Project Brief **82**
Learning Objectives . . . 82
Overview . . . 82
The Scope Creep Villain . . . 82
The Holy Trinity of Research Design . . . 83
The One-Page Prospectus . . . 84
Common Pitfalls and How to Avoid Them . . . 85
Defending Your Choices . . . 86
The Feasibility Reality Check . . . 87
From Prospectus to Execution . . . 87
Practice: Crafting Your Prospectus . . . 87
Exercise 6.1: Diagnosing Scope Creep . . . 87
Exercise 6.2: The Holy Trinity Alignment Check . . . 88
Exercise 6.3: Write Your Own Prospectus (Draft 1) . . . 88
Exercise 6.4: Peer Review Practice . . . 88
Reflection Questions . . . 89
Chapter Summary . . . 89
Key Terms . . . 89
Looking Ahead . . . 90

Structured Listening **91**
Learning Objectives . . . 91
Overview . . . 91
The Methodology of Listening . . . 91
The Translator Role . . . 92
Ethnographic Observation in Digital Spaces . . . 93
Formulating Research Questions (RQs) . . . 94
From Observations to Hypotheses . . . 95
The Danger of Premature Coding . . . 95
Case Study: Observing Twitch Raid Culture . . . 95
Real Data: Bob's Burgers Episodes . . . 96
Practice: Developing Structured Listening Skills . . . 98
Exercise 7.1: Field Notes Practice . . . 98
Exercise 7.2: Translating Observations to RQs . . . 99
Exercise 7.3: Distinguishing Context-Dependent Meaning . . . 99
Exercise 7.4: From Listening to Theory . . . 100
Reflection Questions . . . 100
Chapter Summary . . . 100
Key Terms . . . 101
Looking Ahead . . . 101

From Vibes to Variables **102**
Learning Objectives . . . 102

Overview . . . 102
The Operationalization Gap . . . 102
Conceptual vs. Operational Definitions . . . 103
The Reliability-Validity Trade-off . . . 104
Levels of Measurement (NOIR) . . . 104
Operationalizing Complex Concepts . . . 106
Common Operationalization Mistakes . . . 107
Case Study: Operationalizing "Streamer Authenticity" . . . 108
Practice: Mastering Operationalization . . . 108
Exercise 8.1: Writing Operational Definitions . . . 108
Exercise 8.2: Identifying Levels of Measurement (NOIR) . . . 109
Exercise 8.3: The Reliability-Validity Trade-off . . . 110
Exercise 8.4: From Vibes to Variables (Your Turn) . . . 110
Real Data: American Idol Judge Feedback . . . 111
Step 1: Conceptual Definitions . . . 111
Step 2: Observable Indicators . . . 111
Step 3: Operational Definitions . . . 111
Step 4: Data Structure . . . 112
Step 5: Reliability Considerations . . . 112
Accessing the Data . . . 112
Reflection Questions . . . 113
Chapter Summary . . . 113
Key Terms . . . 114
Looking Ahead . . . 114

The Rulebook **115**
Learning Objectives . . . 115
Overview . . . 115
Content Analysis Methodology . . . 115
Anatomy of a Codebook . . . 116
Developing Categories . . . 116
Writing Operational Definitions (Reprise) . . . 118
Example Codebook: Twitch Chat Toxicity . . . 118
Handling Ambiguity . . . 120
Codebook Format . . . 120
Reliability and the Codebook . . . 121
Case Study: Coding Gender Representation in Esports News . . . 121
Practice: Building Reliable Codebooks . . . 122
Exercise 9.1: Ensuring Exhaustive and Mutually Exclusive Categories . . . 122
Exercise 9.2: Writing Decision Rules . . . 123
Real Data: American Idol Codebook Example . . . 123
Full Codebook: Judge Feedback Tone in American Idol . . . 123
1. Unit of Analysis . . . 123

2. Variables . . . 124
3. Coding Example Walkthrough . . . 126
4. Pilot Test Results . . . 127
Real Data Download . . . 128
Exercise 9.3: Codebook Construction . . . 129
Exercise 9.4: Interpreting Reliability Metrics . . . 129
Reflection Questions . . . 129
Chapter Summary . . . 130
Key Terms . . . 130
Looking Ahead . . . 131

The Sample **132**
Learning Objectives . . . 132
Overview . . . 132
Populations and Sampling Frames . . . 132
Sampling Techniques . . . 133
Sample Size Considerations . . . 135
Pilot Testing the Codebook . . . 135
Inter-Coder Reliability (ICR) . . . 136
When Reliability is Low: Troubleshooting . . . 137
Lone Coder Problem . . . 138
Sampling Bias and Generalizability . . . 138
Case Study: Pilot Testing a Twitch Toxicity Codebook . . . 138
Real Data: American Idol Sampling and Reliability . . . 139
Research Setup . . . 139
Step 1: Define Sampling Frame . . . 140
Step 2: Stratify by Round . . . 140
Step 3: Calculate Sample Size . . . 140
Step 4: Random Selection Within Strata . . . 140
Step 5: Pilot Reliability Study . . . 141
Step 6: Re-Pilot and Calculate Reliability . . . 141
Step 7: Calculate Demographics Reliability . . . 142
Step 8: Document Sampling and Reliability . . . 142
Practice: Sampling and Reliability . . . 143
Exercise 10.1: Sampling Technique Selection . . . 143
Exercise 10.2: Calculating Inter-Coder Reliability . . . 143
Exercise 10.3: Sample Size Decisions . . . 144
Exercise 10.4: Diagnosing Low Reliability . . . 144
Exercise 10.5: Stratified Sampling in Practice . . . 145
Working Example: Complete Sampling and Reliability Workflow . . . 145
Study: Gender Representation in Gaming News . . . 145
Exercise 10.3: Identifying Sampling Bias . . . 149
Exercise 10.4: Planning Your Pilot Study . . . 149

Reflection Questions . . . 149
Chapter Summary . . . 150
Key Terms . . . 150
Looking Ahead . . . 151

Wrangling the Data **152**
Learning Objectives . . . 152
Overview . . . 152
The Tidy Data Philosophy . . . 152
Importing Data into R . . . 153
Common Data Problems and Solutions . . . 154
Essential `dplyr` Verbs . . . 155
Reshaping Data . . . 159
Factors: Taming Categorical Data . . . 160
Documenting Your Cleaning Steps . . . 160
Case Study: Cleaning a Twitch Chat Dataset . . . 161
Real Data: American Idol Data Wrangling . . . 162
Scenario: Raw Coding Sheet to Analysis-Ready Data . . . 162
Step 1: Set Up the Cleaning Script . . . 163
Step 2: Fix Episode IDs . . . 163
Step 3: Standardize Judge Tone Coding . . . 163
Step 4: Convert Advancement to Binary . . . 164
Step 5: Standardize Demographics . . . 164
Step 6: Handle Missing Data . . . 165
Step 7: Add Judge as Factor . . . 165
Step 8: Create Analysis Variables . . . 166
Step 9: Remove Duplicates . . . 166
Step 10: Save Cleaned Data . . . 166
Step 11: Create Data Dictionary . . . 167
Practice: Data Wrangling in R . . . 167
Exercise 11.1: Tidy Data Diagnosis . . . 167
Exercise 11.2: dplyr Practice . . . 168
Exercise 11.3: Handling Missing Data . . . 169
Exercise 11.4: Data Cleaning Log . . . 169
Reflection Questions . . . 170
Chapter Summary . . . 170
Key Terms . . . 170
Looking Ahead . . . 171

Visualizing the Narrative **172**
Learning Objectives . . . 172
Overview . . . 172
The Grammar of Graphics . . . 172

Visualization as Storytelling . . . 174
Common Visualizations . . . 175
Frequency Tables . . . 177
Descriptive Statistics . . . 178
Combining Tables and Visualizations . . . 179
Principles of Effective Visualization . . . 179
Answering Descriptive Research Questions . . . 180
Real Data: American Idol Judge Feedback Visualization . . . 181
Question 1: Judge Tone by Contestant Demographics . . . 181
Question 2: Does Positive Feedback Predict Advancement? . . . 183
Descriptive Statistics Summary . . . 184
Case Study: Visualizing Twitch Toxicity Data . . . 185
Practice: Creating Effective Visualizations . . . 187
Exercise 12.1: Chart Type Selection . . . 187
Exercise 12.2: Interpreting Descriptive Statistics . . . 187
Exercise 12.3: Building a Frequency Table . . . 188
Exercise 12.4: Improving Visualizations . . . 188
Exercise 12.5: ggplot2 Practice (Pseudocode) . . . 188
Reflection Questions . . . 189
Chapter Summary . . . 189
Key Terms . . . 190
Looking Ahead . . . 190

Making the Call **191**
Learning Objectives . . . 191
Overview . . . 191
Statistical Foundations Primer . . . 191
The Logic of Inference . . . 193
Exposing the Sacred Flaw . . . 194
The Chi-Square Test of Independence . . . 194
Running Chi-Square in R . . . 195
Effect Size: Beyond Significance . . . 196
Reporting Results . . . 197
Statistical Significance vs. Practical Significance . . . 198
Common Misinterpretations . . . 198
When Chi-Square Isn't Appropriate . . . 198
Real Data: American Idol Judge Feedback Analysis . . . 199
Research Question 1: Does Judge Tone Differ by Contestant Gender? . . . 199
Research Question 2: Does Advancement Differ by Judge Feedback Tone? . . . 201
Research Question 3: Do Multiple Judges Show Consistent Patterns? . . . 203
Case Study: Testing Toxicity by Game . . . 205
Choosing the Right Test: A Decision Guide . . . 206
Test Selection Matrix . . . 206

The Independent Samples t-Test . . . 207
When to Use t-Test . . . 207
The Null Hypothesis for t-Test . . . 207
How t-Test Works (Conceptually) . . . 208
Assumptions . . . 208
Running t-Test in R . . . 208
Effect Size for t-Test: Cohen's d . . . 209
Analysis of Variance (ANOVA) . . . 210
When to Use ANOVA . . . 210
Why Not Just Run Multiple t-Tests? . . . 210
The Null Hypothesis for ANOVA . . . 211
How ANOVA Works (Conceptually) . . . 211
Running One-Way ANOVA in R . . . 211
Effect Size for ANOVA: Eta-Squared (^{2}) . . . 213
Correlation: Measuring Linear Relationships . . . 213
When to Use Correlation . . . 213
The Null Hypothesis for Correlation . . . 213
How Correlation Works (Conceptually) . . . 214
Running Correlation in R . . . 214
When Correlation Breaks Down . . . 216
Exercise 13.1: Interpreting P-Values . . . 216
Exercise 13.2: Chi-Square Assumptions and Calculations . . . 216
Exercise 13.3: Statistical vs. Practical Significance . . . 217
Exercise 13.4: Null Hypothesis Formulation . . . 217
Exercise 13.5: Type I and Type II Errors . . . 218
Reflection Questions . . . 218
Chapter Summary . . . 218
Key Terms . . . 219
Looking Ahead . . . 220

The One-Click Report **221**
Learning Objectives . . . 221
Overview . . . 221
The One-Click Philosophy . . . 221
The IMRaD Structure . . . 221
14.2.1 Introduction (The Story) . . . 222
14.2.2 Methods (The Recipe) . . . 223
14.2.3 Results (The Data Speak) . . . 224
14.2.4 Discussion (The Interpretation) . . . 225
Writing in Quarto: Integrating Code and Narrative . . . 226
Real Data: American Idol Research Report Example . . . 227
American Idol Study: Full Report Outline . . . 227
Deploying to GitHub Pages . . . 230

The Narrative Arc Completed . . . 231
Reflection: What You've Built . . . 232
Beyond the Semester . . . 232
Practice: Assembling Your Research Portfolio . . . 232
Exercise 14.1: Writing IMRaD Sections . . . 232
Exercise 14.2: Integrating Code and Narrative . . . 233
Exercise 14.3: GitHub Pages Deployment Checklist . . . 234
Exercise 14.4: Peer Review of Complete Project . . . 234
Exercise 14.5: Final Reflection Essay . . . 234
Reflection Questions . . . 235
Chapter Summary . . . 235
Key Terms . . . 236
The End (and the Beginning) . . . 236

Appendix A: R Quick Reference Guide **237**
Data Import and Export . . . 237
Basic Data Inspection . . . 237
Data Manipulation (dplyr) . . . 238
Data Visualization (ggplot2) . . . 239
Inferential Statistics . . . 240
String Manipulation . . . 241
Handling Missing Data . . . 241
Frequency Tables . . . 242
Useful Functions . . . 242
Getting Help . . . 243
Common Operators . . . 243
Keyboard Shortcuts (RStudio) . . . 244

Appendix B: Troubleshooting Common Errors **245**
R and RStudio Errors . . . 245
Error: "could not find function" . . . 245
Error: "object not found" . . . 246
Error: "unexpected symbol" or "unexpected ','" . . . 246
Error: "cannot open file" . . . 246
Error: "non-numeric argument to binary operator" . . . 247
Warning: "NAs introduced by coercion" . . . 247
Error: "subscript out of bounds" . . . 248
Quarto Rendering Errors . . . 248
Error: "failed to compile" . . . 248
Error: "object not found" (but code works in Console) . . . 249
Error: "LaTeX Error" or "PDF rendering failed" . . . 250
ggplot2 Errors . . . 250
Error: "Aesthetics must be either length 1 or the same as the data" . . . 250

Error: “Don’t know how to automatically pick scale for object of type” 251
Error: “Discrete value supplied to continuous scale” 251
Statistical Test Errors 252
Warning: “Chi-squared approximation may be incorrect” 252
Error: “data are essentially constant” 252
Git/GitHub Errors 252
Error: “failed to push some refs” 252
Error: “merge conflict” 253
General Troubleshooting Strategy 253
Getting Help Resources 254
Prevention is Better Than Cure 254

Appendix C: Sample Codebook Template **255**
Project Title 255
Study Overview 255
Coding Procedure 256
Step 1: Data Preparation 256
Step 2: Coder Training 256
Step 3: Coding Rules 256
Variable 1: [Variable Name] 256
Category 1: [Name] 257
Category 2: [Name] 257
Category 3: Unclear/Other 257
Variable 2: [Variable Name] 257
Decision Rules for Edge Cases 258
Edge Case 1: [Describe situation] 258
Edge Case 2: [Describe situation] 258
Edge Case 3: Emojis, Emotes, and Non-Text 258
Sample Coding Sheet 258
Inter-Coder Reliability 259
Codebook Revision History 259
References 260
Appendices 260
Appendix D: Twitch Emote Reference 260
Notes for Future Researchers 260
Quick Coding Checklist 261

Appendix D: Data Management Best Practices **262**
The Three Rules of Data Management 262
Project Organization Structure 262
File Naming Conventions 263
Raw Data Protection 264
Rule: NEVER Edit Raw Data Directly 264

Backup Strategy . . . 264
3-2-1 Rule . . . 264
Automated Backups . . . 265
Documentation Requirements . . . 265
README.md (Project Root) . . . 265
Data Documentation (data/README.md) . . . 266
Version Control with Git . . . 267
Basic Git Workflow . . . 267
What to Commit . . . 267
.gitignore File . . . 268
Data Security and Ethics . . . 268
Sensitive Data . . . 268
Data Sharing . . . 269
Common Data Disasters (and How to Avoid Them) . . . 269
Disaster 1: "I accidentally deleted my data" . . . 269
Disaster 2: "My computer crashed and I lost 3 weeks of work" . . . 269
Disaster 3: "I can't remember what this variable means" . . . 270
Disaster 4: "My analysis results changed and I don't know why" . . . 270
Quality Checks Before Finalizing Data . . . 270
Checklist: Before Submitting Final Project . . . 271

From Vibes to Variables

A Field Guide to Open Media Science

Welcome to From Vibes to Variables

Building Reproducible Research with R, Quarto, and the Science of Storytelling

This textbook teaches you how to transform qualitative observations ("vibes") into quantifiable research variables through systematic content analysis. Whether you're studying Twitch chat toxicity, media bias, social media algorithms, or online community norms, you'll learn a practical, reproducible workflow for turning your curiosity into publication-ready findings.

What You'll Learn

- **Content Analysis Fundamentals**: Design rigorous coding schemes that capture the "vibe" you're investigating
- **Data Wrangling in R**: Transform messy, hand-coded data into analysis-ready datasets
- **Statistical Analysis**: From descriptive statistics to inferential tests that answer your research questions
- **Reproducibility**: Document your methods so anyone can verify (and build on) your work
- **The Science of Storytelling**: Present your findings in ways that stick with your audience

Who This Book Is For

- **Media and Communication Students**: Learning to conduct original research
- **Journalism and Digital Media Students**: Understanding the data behind stories
- **Social Science Researchers**: Building quantitative foundations from qualitative observations
- **Anyone with Curiosity**: If you've ever wondered *why* a phenomenon happens online, this teaches you how to find out

How to Use This Book

Each chapter combines **theory** (why we do this) with **practice** (step-by-step tutorials in R). Code examples are included throughout. You can follow along in RStudio or just read and learn the concepts.

What Makes This Different

Unlike traditional statistics textbooks that assume you already know *what* you want to measure, this book starts with your observations and teaches you to operationalize them. We use real examples from media and online communities—not abstract datasets about iris flowers.

Let's get started. Choose a chapter below.

The Science of Storytelling

Learning Objectives

- Reframe empirical research as a structured form of storytelling.
- Understand how narrative elements map onto research design components.
- Recognize the neuroscience foundations of story-making and hypothesis testing.

Overview

Research methods courses often fail mass communication students because they present statistical inquiry as alien to storytelling. This chapter challenges that false dichotomy. The human brain doesn't toggle between "creative mode" and "analytical mode"—it uses the same narrative architecture for both. Understanding research as storytelling transforms statistics from abstract formulas into plot devices that help us uncover truth about media phenomena.

The Neuroscience of Narrative: Why Humans Tell Stories

Neuroscientist Lisa Feldman Barrett's theory of constructed emotion suggests the brain is fundamentally a prediction machine. It constantly generates models of the world, tests those models against incoming sensory data, and updates beliefs when predictions fail. This prediction-error cycle is identical to the scientific method.

Will Storr, in *The Science of Storytelling*, argues that narratives emerged as cognitive tools for managing social complexity. Stories model cause-and-effect relationships, simulate outcomes, and allow groups to coordinate behavior around shared beliefs. When our predictions about the world fail—when the story breaks—we experience cognitive dissonance. Resolution requires either changing the story (updating our hypothesis) or rejecting the evidence (confirmation bias).

The Control-Seeking Brain: Humans crave control and predictability. Uncertainty triggers anxiety. Stories reduce uncertainty by organizing chaotic experience into patterns: beginning, middle, end. Scientific research performs the same function at scale. Where personal anecdotes offer localized control ("I understand my experience"), empirical research offers generalized control ("We understand a phenomenon that affects many people").

The Sacred Flaw: Hypotheses as Narrative Tension

In Storr's framework, every compelling character has a "sacred flaw"—a deeply held but erroneous belief about the world. The protagonist clings to this belief even as evidence mounts against it. The story's dramatic tension arises from the inevitable collision between the sacred flaw and reality.

In research, the **null hypothesis** is the sacred flaw. It's the default story the universe tells: "Nothing interesting is happening here. Any pattern you see is random noise." The researcher's job is to accumulate evidence so overwhelming that the null hypothesis becomes untenable. When we "reject the null," we force the universe to admit its story was wrong and adopt a new one—our alternative hypothesis.

This framing transforms p-values from abstract probabilities into dramatic reveals. A p-value of .001 doesn't just mean "statistically significant"—it means the old story (the null) is so incompatible with the evidence that maintaining it would be absurd.

Mapping Narrative Structure onto Research Design

The Inciting Incident: The Research Problem

Every story begins with disruption. The protagonist's world is stable until something unexpected happens. In research, the inciting incident is an **anomaly**—an observation that doesn't fit existing explanations.

Example from Gaming Research: Anecdotally, streamers report that "toxic" games (competitive FPS titles) generate more viewer engagement than "wholesome" games (cozy simulators). But is this true? If so, why? This anomaly—the counterintuitive relationship between negativity and engagement—is the inciting incident that launches inquiry.

The Protagonist: The Researcher as Detective

The protagonist is the seeker of truth. In detective stories, the detective gathers clues (data), formulates theories (hypotheses), and tests them against evidence. The researcher performs the same role. Like Sherlock Holmes, the researcher must remain skeptical of easy answers and willing to revise theories when evidence contradicts them.

The Antagonist: Confounds, Bias, and Noise

In research narratives, the antagonist isn't a person—it's the chaos that obscures truth. Confounding variables muddy causal relationships. Sampling bias makes findings ungeneralizable. Measurement error introduces noise. The researcher must systematically eliminate these antagonists through rigorous design.

Rising Action: Literature Review and Theory

Before confronting the antagonist, the protagonist needs training and allies. In research, the **literature review** provides both. Previous studies reveal what is already known, where gaps exist, and which methods have succeeded or failed. Theory provides the conceptual "weapons"—frameworks that sharpen the research questions and guide variable selection.

The Climax: Data Analysis

The climax is the moment of confrontation. In research, this is the **statistical test**. All the setup—the research question, the sample, the codebook—has led to this: running the Chi-Square test, reading the p-value, and discovering whether the data support or refute the hypothesis. This is the narrative payoff.

Falling Action: Interpretation and Limitations

After the climax, the detective explains what happened and acknowledges what remains unknown. The **Discussion section** performs this function. What do the findings mean? How do they fit into the broader literature? What alternative explanations exist? What questions remain?

Resolution: Implications and Future Research

The story concludes by showing how the protagonist's world has changed. In research, the **Implications section** argues why the findings matter to practitioners, policymakers, or future scholars. The story may be over, but it opens new narrative threads for others to pursue.

Case Study: "Twitch in the Time of Quarantine"

Let's trace how this narrative structure applies to a real SIM Lab project.

Inciting Incident: During the COVID-19 pandemic, specifically following the March 2020 lockdowns, Twitch viewership spiked by 87%. Anecdotal evidence suggested people were turning to livestreams to cope with isolation, but it was unclear if this usage was actually fulfilling psychological needs or simply passing time.

Protagonist's Quest: The research team set out to determine if the increased engagement on Twitch during the pandemic was successfully meeting specific user needs.

Theory as Training: The team turned to **Uses and Gratifications Theory**, specifically focusing on two fundamental motivations: **social integrative needs** (the need to socialize with friends and family) and **tension release needs** (the need to escape or relieve anxiety). If Twitch was meeting these needs, the data should show changes in how users interacted with the platform's synchronous chat functions.

The Antagonist: The challenge was methodological: How do you measure human emotion and social connection across millions of lines of text? The data was non-parametric—meaning it didn't follow a normal distribution—and heavily skewed by outliers, making standard mean-based statistical tests unreliable.

Climax: Instead of surveys, the team collected and analyzed chat logs from 50 randomly selected Twitch channels, comparing data from "pre-pandemic" (January 2020) to "early pandemic" (April 2020). They used R and SQL to count unique chatters and total messages, and applied the NRC Emotion Association Lexicon to analyze the sentiment of the text.

Resolution: The study revealed a nuanced reality. Contrary to expectations, the number of unique people chatting did not significantly increase. However, the *volume* of messages skyrocketed, and the emotional content of general chat messages became significantly more intense, with increases in both positive and negative sentiments.

Sacred Flaw Exposed: The hypothesis that users would seek comfort through targeted, interpersonal interaction (tagging others) was largely unsupported. Instead, the data showed that users were "screaming into the void"—releasing tension by broadcasting emotions to the general chat rather than directing them at specific individuals.

Anecdote vs. Data: The Difference Between Journalism and Science

Mass communication students are skilled storytellers. They know how to find a compelling anecdote, interview a subject, and craft a narrative that moves audiences. This is **journalism**—and it's valuable. But journalism and science serve different epistemic functions.

An anecdote is a single data point used for emotional resonance. "Meet Sarah, a Twitch viewer who says chatting in a stream helped her vent her anxiety during lockdown." This story

is powerful because it's concrete and relatable. But it can't answer the question, "Is Sarah's experience typical, or is she an outlier?"

Data is the aggregation of many anecdotes, systematically collected and analyzed. In the Twitch study, the researchers didn't just ask users how they felt; they computationally analyzed the sentiment of thousands of messages to detect patterns of "Anger," "Sadness," "Joy," and "Trust" across the entire population. They found that while specific interpersonal targeting didn't change much, the general expression of emotion increased significantly.

The most powerful research combines both. The anecdote provides narrative entry—it makes readers care. The data provides evidentiary weight—it makes readers believe. A well-written research article might open with the user experience of isolation, then use Wilcoxon signed-rank tests to show that the increase in message volume was a statistically significant phenomenon, not just a feeling.

The Journalist-Scientist Identity

This textbook asks students to develop a dual identity. You remain a **journalist**—curious, skeptical, attuned to human stories. But you add the **scientist** toolkit: systematic methods, transparent processes, and quantifiable evidence. This fusion is not a compromise. It's the frontier of modern mass communication research.

Data journalism, computational social science, and digital ethnography all live at this intersection. The journalist's instinct asks, "Are people using Twitch to replace their lost social lives?" The scientist answers by defining "social integrative needs" and measuring them through unique chatter counts and message frequency. The journalist's narrative and the scientist's evidence work together to transform a hunch into a finding.

The Journey Ahead

The remaining chapters of this textbook follow the narrative arc outlined above. You will:

- Build your **publishing engine** (Chapter 2) so your work is reproducible.
- Learn the **rules of the game** (Chapter 3): ethics, epistemology, and the scientific method.
- Gather **intelligence** (Chapter 4) through archival research and literature review.
- Choose your **lens** (Chapter 5) by selecting a theoretical framework.
- Draft your **roadmap** (Chapter 6) with a project prospectus.
- Conduct **structured listening** (Chapter 7) to ground your variables in community norms.
- Operationalize **vibes into variables** (Chapter 8).
- Write the **rulebook** (Chapter 9) by constructing a codebook.
- Define your **sample** (Chapter 10) and test reliability.

- **Wrangle data** (Chapter 11) into tidy, analysis-ready formats.
- **Visualize the narrative** (Chapter 12) with descriptive statistics.
- **Make the call** (Chapter 13) using inferential tests.
- Assemble the **one-click report** (Chapter 14) and publish your portfolio.

By the end, you won't just understand how research works—you'll have produced a complete, publication-ready study. Your research report will be a story with a beginning (the research problem), a middle (the methods and analysis), and an end (the findings and implications). And unlike a traditional story, this one will be backed by data that others can verify and build upon.

Practice: Applying the Narrative Framework

Exercise 1.1: Identifying Narrative Elements in Research

Read the abstract below from a hypothetical study and identify the narrative elements:

> **Abstract**: Social media influencers increasingly promote cryptocurrency investments to young audiences. This study examined whether parasocial relationships with influencers predict investment behavior among 18-25 year olds. Survey data (n=450) revealed that followers with strong parasocial bonds were 3.2 times more likely to invest in cryptocurrencies mentioned by influencers, even when controlling for financial literacy. These findings suggest regulatory attention to influencer finance content may be warranted.

Your Task: Map this abstract onto the narrative structure:

1. **Inciting Incident**: _____
2. **Protagonist**: _____
3. **Antagonist**: _____
4. **Theory as Training**: _____
5. **Climax (Analysis)**: _____
6. **Resolution (Implications)**: _____

Exercise 1.2: From Anecdote to Research Question

You notice that Twitch streamers who play "cozy" games (Stardew Valley, Animal Crossing) seem to have more supportive, less toxic chat environments than competitive FPS streamers.

Your Task:

1. Write this as an **anecdote** (2-3 sentences, narrative style)

2. Translate it into a **research question** (1 sentence, formal)
3. Identify what **data** you would need to test this systematically
4. What **theory** might explain this pattern? (Guess based on common sense, we'll learn theories formally in Chapter 5)

Exercise 1.3: Distinguishing Journalism from Science

For each statement, identify whether it's **journalistic evidence** (anecdote, case study) or **scientific evidence** (systematic, generalizable):

1. "Sarah, a 19-year-old Twitch viewer, says she feels less lonely when watching her favorite streamer."
2. "A survey of 500 Twitch viewers found that 68% report using streams to cope with social isolation."
3. "Analysis of 10,000 chat messages revealed that viewers use significantly more 'we' language (indicating community identity) during collaborative gameplay than competitive gameplay."
4. "An investigative report profiled three streamers who quit due to harassment, highlighting the platform's moderation failures."

Discussion: Which types of evidence are more **emotionally compelling**? Which are more **generalizable**? How might you combine both in a research report?

Reflection Questions

1. **Reframing Resistance**: Many students fear research methods courses. After reading this chapter, how has your perception of "research" changed? Do you see connections between research and skills you already have (storytelling, critical thinking, detective work)?

2. **Prediction Machines**: Barrett's theory suggests our brains are constantly making predictions and updating them when wrong. Think of a time when your prediction about media was proven wrong by evidence. What was your "sacred flaw" (false belief), and what evidence challenged it?

3. **Anecdotal Thinking**: Reflect on your own media consumption. When you make claims like "TikTok is ruining attention spans" or "podcasts are the future of news," are you relying on anecdotes or data? How would you test these claims systematically?

Chapter Summary

This chapter established the foundational philosophy of the course: **research is storytelling with evidence**. Key takeaways:

- The human brain uses the same narrative architecture for both creative storytelling and scientific hypothesis testing
- The **null hypothesis** functions as a "sacred flaw"—a belief that must be challenged with overwhelming evidence
- Research follows a narrative arc: inciting incident (research problem) → rising action (theory and methods) → climax (data analysis) → resolution (implications)
- **Anecdotes** provide emotional resonance but can't establish generalizability; **data** provides systematic evidence across many cases
- The journalist-scientist identity fuses narrative skill with empirical rigor
- This course trains you to conduct research by *doing* research, not just reading about it

Key Terms

- **Anecdote**: A single illustrative example or case study
- **Confirmation bias**: The tendency to seek information that confirms existing beliefs
- **Control**: The ability to predict and influence outcomes
- **Hypothesis**: A testable prediction derived from theory
- **Narrative arc**: The structure of a story (exposition, rising action, climax, falling action, resolution)
- **Null hypothesis**: The assumption that no relationship or effect exists; the "default story"
- **P-value**: The probability of observing data as extreme as yours if the null hypothesis were true
- **Prediction error**: The difference between what the brain expects and what actually occurs
- **Sacred flaw**: A deeply held but erroneous belief (Storr's term)
- **Systematic inquiry**: Research conducted using consistent, replicable methods

Looking Ahead

Chapter 2 introduces the **Publishing Engine**—the technical infrastructure (R, RStudio, Quarto, Git) that makes research reproducible. You'll learn why code-based workflows prevent replication crises and how computational tools transform research from static documents into dynamic, verifiable reports.

The Publishing Engine

Learning Objectives

- Understand the philosophy and necessity of reproducible research in the digital age.
- Recognize how computational workflows prevent replication crises.
- Grasp the distinction between static documents and dynamic, code-driven reports.

Overview

The replication crisis in social science emerged from a simple fact: most published research cannot be reproduced. Not because the authors lied, but because the methods were described vaguely, data were lost, or analytical choices were made through point-and-click interfaces that left no record. This chapter introduces the **Publishing Engine**—a workflow built on code, version control, and dynamic documents that makes research inherently reproducible.

The Replication Crisis: When Science Broke Its Promise

In 2015, the Open Science Collaboration attempted to replicate 100 psychology studies published in top-tier journals. Only 36% successfully replicated. This wasn't fraud—it was fragility. Small, undisclosed analytical decisions ("Should I exclude this outlier?" "Which covariates should I include?") dramatically affected results. When these decisions were buried in researchers' heads or lost in proprietary software, replication became impossible.

Mass communication research faces the same problem. Content analysis codebooks get lost. Data cleaning steps go undocumented. Statistical software like SPSS saves output but not the sequence of menu clicks that produced it. If you can't reproduce your own analysis six months later, how can anyone else?

The Fragility of Point-and-Click Workflows

Consider a typical undergraduate workflow:

1. Code media content in an Excel spreadsheet.

2. Import into SPSS.
3. Click through menus to generate frequency tables.
4. Copy-paste output into a Word document.
5. Manually format tables.
6. Submit the Word doc as the "final" product.

What's wrong with this?

- **No record of decisions**: Did you recode variables? Exclude cases? Transform data? Those choices exist only in your memory.
- **Manual errors**: Copy-paste mistakes are invisible but consequential.
- **Non-reproducibility**: If your data changes (new coders add rows, you find errors), you must repeat every manual step.
- **Collaboration friction**: Collaborators can't see or verify your process.

Code as Recipe: The Literate Programming Solution

In 1984, computer scientist Donald Knuth proposed **literate programming**: code and explanatory text woven together in the same document. The reader sees not just results but the logic that produced them.

Modern tools like **Quarto** (and its predecessor R Markdown) realize Knuth's vision for research. A Quarto document (.qmd) contains:

- **Narrative text**: Your explanations, interpretations, and context (written in Markdown).
- **Executable code**: R chunks that load data, clean it, analyze it, and generate visualizations.
- **Output**: Tables, figures, and statistical results generated automatically when the document is rendered.

The recipe metaphor: Think of your .qmd file as a recipe.

- **Ingredients**: Raw data files.
- **Instructions**: R code ("load this file," "filter rows where X > 5," "run a Chi-Square test").
- **Final dish**: The rendered report (HTML, PDF, or Word).

If the ingredients change (new data arrives), you don't rewrite the recipe—you just re-run it. The output updates automatically.

The Five Layers of Reproducible Research

Reproducibility exists on a spectrum. Here are five levels, from weakest to strongest:

Level 0: Publication Only

You publish a paper describing your methods. No data, no code. Replication requires someone to re-collect data and guess at your analytical choices. **Reproducibility: ~0%.**

Level 1: Data Sharing

You publish the paper and upload a CSV of your raw data to a repository (OSF, Dataverse). Others can verify your descriptive statistics but not your analytical pipeline. **Reproducibility: ~30%.**

Level 2: Code Sharing

You share data *and* scripts (.R or .py files). Others can run your code and reproduce your tables. But scripts often assume a specific directory structure, package versions, or operating system. **Reproducibility: ~60%.**

Level 3: Computational Environment

You use tools like `renv` (R) or Docker (containers) to document package versions and system dependencies. Others can replicate your exact computational environment. **Reproducibility: ~85%.**

Level 4: Literate, Executable Documents

You publish a Quarto document (or Jupyter Notebook) that contains narrative text, code, data links, and output in a single file. Anyone can click "Render" and regenerate your entire paper. This is the gold standard. **Reproducibility: ~95%.**

This textbook trains you to work at **Level 4**.

Why Reproducibility Matters Beyond Academia

Reproducibility isn't just an academic virtue—it's a professional necessity.

For employers: Data scientists and media analysts work in teams. If your boss asks, "Why did engagement drop last quarter?" and you respond with a static PDF, they can't interrogate your assumptions. If you respond with a Quarto report linked to a live dashboard, they can adjust parameters and explore alternative explanations.

For your future self: You are your own most important collaborator. Six months after completing a project, you will forget why you made specific decisions. A well-documented Quarto file is a time capsule that reminds "future you" what "past you" was thinking.

For democracy: Misinformation thrives when analysis is opaque. Journalists who publish data-driven investigations with open code and data invite scrutiny, which *strengthens* credibility. Reproducibility is a defense against accusations of bias.

The Tools of the Publishing Engine

Four tools form the core of the reproducible workflow:

R: The Statistical Engine

R is a programming language designed for statistical analysis. Unlike SPSS or Excel, R is:

- **Free and open-source**: No licensing barriers.
- **Extensible**: Thousands of packages extend its capabilities (text analysis, network analysis, machine learning).
- **Scripted**: Every operation is recorded as code, making analysis reproducible.

Why not Python? Python is excellent for machine learning and web scraping, but R remains dominant in academic statistics and social science. Both are valuable; this course prioritizes R because of its tight integration with Quarto and its rich ecosystem for content analysis.

RStudio: The Integrated Development Environment (IDE)

RStudio makes R easier to use. It provides:

- A script editor with syntax highlighting.
- A console for running code interactively.
- A pane for viewing plots, data frames, and help documentation.
- Built-in support for Quarto and Git.

Think of R as the engine and RStudio as the dashboard.

Quarto: The Publishing Engine

Quarto converts .qmd files (plain text with code) into polished reports (HTML, PDF, Word, PowerPoint, websites). Key features:

- **Multi-format output**: Write once, render to any format.
- **Cross-references**: Automatically number figures and tables.
- **Citations**: Integrate bibliographies using BibTeX or CSL.
- **Reproducibility**: Code execution is logged; if a code chunk fails, the render fails (preventing silent errors).

Quarto is the successor to R Markdown, with better performance and support for multiple languages (R, Python, Julia, Observable).

Git and GitHub: Version Control and Collaboration

Git is a version control system that tracks every change to your files. It's like "Track Changes" in Word, but for entire projects.

GitHub is a hosting platform for Git repositories. It enables:

- **Backup**: Your work is saved in the cloud.
- **Collaboration**: Multiple people can work on the same project without overwriting each other.
- **Transparency**: Public repositories demonstrate your skills to employers or admissions committees.
- **Deployment**: GitHub Pages can host your final portfolio as a live website.

Why version control matters: Research is iterative. You'll try multiple analytical strategies, revise your codebook, and refine your writing. Git lets you experiment fearlessly because you can always revert to an earlier version.

The One-Click Report Philosophy

The ultimate goal of this workflow is the **one-click report**: a Quarto document that, when rendered, regenerates your entire analysis from raw data to final figures to formatted text.

Imagine this scenario: You've completed your study on Twitch toxicity. You submit your report. A week later, your instructor says, "Great work, but can you add data from one more game?"

In a traditional workflow, this is a nightmare: re-code new data, re-run analyses, re-generate tables, re-copy-paste, re-format.

In a Quarto workflow, it's trivial: append new rows to your CSV, click "Render," and the updated report appears in 30 seconds.

This isn't just convenient—it's transformative. It makes iteration cheap, which means you'll iterate more, which means your final work will be better.

Computational Literacy as Media Literacy

The 21st-century media landscape is shaped by algorithms: recommendation systems curate your news feed, sentiment analysis automates content moderation, and predictive models decide which ads you see. Understanding these systems requires computational literacy.

Learning to code isn't about becoming a software engineer. It's about understanding the logics that govern digital communication. When you write an R script to analyze Twitter sentiment, you're not just producing research—you're demystifying the same techniques that platforms use to manipulate your attention.

Computational literacy is media literacy. And reproducible workflows are a form of media transparency.

Overcoming Code Anxiety

Many students fear coding because they assume it requires mathematical genius or innate technical ability. This is false.

Coding is a language, and like any language, it's learned through practice, not talent. You don't need to memorize syntax—you look things up (Google, Stack Overflow, ChatGPT). You don't need to write perfect code—you debug iteratively.

The hardest part of coding is the first 10 hours, when nothing works and errors are cryptic. After that, patterns emerge. You'll recognize error types. You'll develop debugging strategies. You'll copy-paste code from old projects and adapt it to new ones.

This textbook assumes zero prior coding experience. Every code example will be explained line-by-line. By Chapter 14, you'll be comfortable reading, modifying, and writing R code for media research.

The Ecosystem Beyond This Course

The tools you learn here—R, Quarto, Git—are not course-specific. They're industry-standard tools used by:

- **Academic researchers**: Publishing computational social science.

- **Data journalists**: Producing investigative reports at *The New York Times*, *FiveThirtyEight*, and *The Pudding*.
- **UX researchers**: Analyzing user behavior at tech companies.
- **Marketing analysts**: Measuring campaign effectiveness.
- **Policy researchers**: Evaluating social programs.

Learning these tools once opens career pathways across domains. This course is your entry point into a global community of computational social scientists.

Setting Up Your Publishing Engine

Phase 1: Markdown and Note-Taking (Obsidian)

You will **not** install RStudio today. Instead, you'll start with **Obsidian**, a note-taking application that teaches Markdown—the lightweight formatting language that underlies Quarto. This is intentional: learning Markdown first, separate from code, builds confidence and clarity before adding R's complexity.

Why Obsidian First?

Problem with jumping straight to RStudio: Students arrive to find a complex IDE with four panes, a console, package managers, and mysterious error messages. The cognitive load is high, and frustration sets in before learning begins.

Better approach: Learn Markdown in a clean, forgiving environment (Obsidian), master the formatting language that 100% of Quarto documents use, *then* transition to RStudio when you need code.

What you'll learn in Obsidian: - Markdown formatting (headings, lists, bold, italics, links) - Organizing research notes with folders and tags - Creating internal links between notes ("backlinks") - Exporting notes for sharing - File structure and project organization

Skill transfer: Everything you learn about Markdown in Obsidian carries directly into Quarto. When you see `# Heading` in Quarto, you'll already know it creates a big title.

Step 1: Install Obsidian

1. Visit https://obsidian.md/
2. Download the free version for your operating system
3. Run the installer with default settings
4. On first launch, create a new vault (folder) called `research-notes`

What you're installing: Obsidian is a note-taking application that stores files as plain-text Markdown. It's designed for researchers, students, and knowledge workers.

Note: If you prefer alternative Markdown editors, you can use VS Code, Typora, or even Notion. The key is learning Markdown in a low-stakes environment before adding code.

Step 2: Create a GitHub Account

1. Visit https://github.com/
2. Sign up for a free account
3. Verify your email address

What you're creating: A cloud-based repository hosting service for version control and collaboration. You'll use GitHub in Chapters 3-6 to document research decisions, and in Chapter 14 to publish your final portfolio.

Step 3: (Deferred) RStudio & R Installation

You will **not** install RStudio or R yet. We'll introduce these tools in **Chapter 9**, after you've grounded your research questions and variables. At that point, you'll have context for why R is useful, and the syntax won't feel arbitrary.

Your First Markdown Document in Obsidian

Let's create a simple research note to practice Markdown.

Step 1: Open Obsidian

Launch Obsidian and open your `research-notes` vault.

Step 2: Create a New Note

1. Click the **New Note** button (or press Ctrl+N / Cmd+N)
2. Name it "Research Topic: Gaming Communities"
3. Start typing

Step 3: Practice Markdown Syntax

Obsidian renders Markdown in real-time. Type the following, observing how it formats:

```
# Gaming Communities Research

## Why This Topic Matters

I'm curious about how players in competitive gaming form social bonds.

## Initial Observations

- Toxicity seems common but varies by game and community
- Some streamers have highly engaged chat communities
- In-game communication (voice chat vs. text) affects teamwork

## Key Questions

1. What communication norms do communities develop?
2. How do newcomers learn these norms?
3. Does platform (Discord, Twitch, in-game) affect behavior?

## Related Concepts

- **Parasocial interaction**: One-sided emotional connection to content creators
- **In-group/out-group dynamics**: Favoring your team/community over rivals
- **Social norms**: Unwritten rules that govern behavior

## Sources to Read

- Check out the [*Gaming Communities* Wikipedia page](https://en.wikipedia.org/wiki/Online_c
- Look for articles on esports toxicity
```

Step 4: Linking Notes Together

Obsidian's power lies in **linking** between notes. Create a new note called "Parasocial Interaction" and link to it:

In your "Gaming Communities" note, change the line to:

```
- **Parasocial interaction**: [[Parasocial Interaction|One-sided emotional connection to con
```

Now click that link. Obsidian shows a preview, and you can jump between related ideas.

Step 5: Export and Share

1. Right-click your note
2. Select **Export as PDF** or **Copy as Markdown**
3. You now have a shareable document

This same workflow—write in Markdown, organize with links, export to multiple formats—is exactly what Quarto does, but with added R code.

Markdown Essentials for Research Writing

Markdown is a lightweight **formatting language**. Instead of clicking buttons in Word, you use simple symbols to indicate formatting. Obsidian renders these symbols in real-time, showing you the formatted result.

Headings

```
# Heading 1 (main title, use once per document)
## Heading 2 (major sections)
### Heading 3 (subsections)
#### Heading 4 (sub-subsections)
```

When to use: - `#` for your main document title - `##` for major sections (Introduction, Methods, Results) - `###` for subsections (Literature Review, Conceptual Framework)

Text Formatting

```
**Bold text** (important concepts, variable names)
*Italic text* (emphasis, citations)
***Bold and italic***
~~Strikethrough~~ (for corrections)
```

When to use: - **Bold**: Key terms, research variables, important findings - *Italic*: Study names, book titles, emphasis - Strikethrough: Showing a revision or change of mind

Lists

Unordered (bullets):

```
- Bullet point 1
  - Nested bullet (indent with 2 spaces)
    - Deeper nest (4 spaces)
- Bullet point 2
```

Ordered (numbered):

```
1. First step
2. Second step
3. Third step
```

Nested lists:

```
1. Research question
   - Sub-question A
   - Sub-question B
2. Methodology
   - Data collection
   - Analysis
```

When to use: - Unordered lists for related ideas of equal importance - Ordered lists for sequential steps or ranked priorities - Nested lists to show hierarchy

Links and References

External links:

```
[Link text](https://example.com)
[Read the paper](https://scholar.google.com/)
```

Internal links (Obsidian feature):

```
[[Note Name|Display Text]]
[[Parasocial Interaction Theory]]
```

Footnotes (Obsidian supports these):

```
This concept is important[^1].

[^1]: Parasocial interaction refers to the illusion of personal relationship with media figu
```

Code and Technical Terms

For inline code (technical terms, variable names, function names):

```
`variable_name`, `read.csv()`, `ggplot2`
```

For code blocks (multiple lines):

```
```
# This is a multi-line code block
data <- read.csv("file.csv")
mean(data$age)
```
```

Tables

In Obsidian, create tables using pipes and dashes:

```
| Variable | Meaning | Example |
|----------|---------|---------|
| Game | The title played | Valorant, Fortnite |
| Viewers | Concurrent audience | 1200, 500 |
| Toxicity | Rate of toxic messages | Low, High |
```

Block Quotes

```
> "Research is a conversation with data."
> - Kurt Lewin

> This is a key insight from the literature.
```

Emphasis Through Lists and Headings

A common mistake is using paragraph text for complex ideas. Use structure instead:

Less effective:

```
This study examines whether streamers with webcams get more donations.
We collected data from 50 streams. We measured average donations.
We compared streamers with and without webcams. We used a t-test to determine significance.
```

More effective:

```
## Research Question

Do streamers with webcams receive higher average donations?

## Data Collection

- **Sample**: 50 Twitch streams
- **Metrics**: Average donation per stream, streamer webcam use
- **Time period**: January-March 2024

## Analysis

- Compared donation amounts between groups
- Calculated means and standard deviations
- Used independent samples t-test
```

The second version uses headings and lists to guide the reader through the structure.

```
---
title: "Webcam Use and Viewer Engagement"
author: "Your Name"
date: today
format: html
---

## Research Question

Do streamers who use webcams attract more viewers than streamers without webcams?

## Data Collection
```

```
I observed 20 Twitch streams and recorded:
- Streamer name
- Game played
- Webcam use (yes/no)
- Average concurrent viewers


::: {.cell}

```{.r .cell-code}
library(tidyverse)
library(ggplot2)
```
:::

::: {.cell}

```{.r .cell-code}
# Load data
streamers <- read.csv("data/streamer_observations.csv")

# Preview
head(streamers)
```
:::

Analysis

Descriptive Statistics

::: {.cell}

```{.r .cell-code}
# Calculate means by webcam use
streamers %>%
  group_by(webcam) %>%
  summarize(
    n = n(),
```
```

```
    mean_viewers = mean(avg_viewers),
    sd_viewers = sd(avg_viewers)
  )
```
:::

Visualization

::: {.cell}

```{.r .cell-code}
ggplot(streamers, aes(x = webcam, y = avg_viewers, fill = webcam)) +
  geom_boxplot() +
  labs(
    title = "Average Viewers by Webcam Use",
    x = "Uses Webcam",
    y = "Average Concurrent Viewers"
  ) +
  theme_minimal() +
  scale_fill_manual(values = c("No" = "gray", "Yes" = "steelblue"))
```
:::

Statistical Test

::: {.cell}

```{.r .cell-code}
# Independent samples t-test
t_result <- t.test(avg_viewers ~ webcam, data = streamers)
t_result
```
:::

Results

In this example, streamers who used webcams had higher average viewers than those without. Th

```
## Conclusion

This demonstrates how Quarto integrates code, analysis, and narrative into one reproducible
```

This example demonstrates: - Loading packages - Reading external data - Data summarization - Visualization - Statistical testing - How code and narrative work together - The complete research workflow in one document

Debugging Common Errors

When learning R, you'll encounter errors. Here's how to troubleshoot:

Error: "Object not found"

```
Error: object 'data' not found
```

Problem: You're trying to use a variable that doesn't exist.
Solution: Check spelling. Did you assign it? Is it called something else?

Error: "Could not find function"

```
Error: could not find function "ggplot"
```

Problem: The package isn't loaded.
Solution: Run `library(ggplot2)` first.

Error: "File not found"

```
Error: cannot open file 'data.csv': No such file or directory
```

Problem: The file path is wrong.
Solution: Check your working directory with `getwd()`. Use correct relative path.

Pro tip: Use `here::here("data", "file.csv")` for robust file paths.

Project Organization: Building a Research Vault

Before jumping into analysis, researchers need a structure for organizing notes, sources, and ideas. Obsidian's **vault** (folder) system mirrors the structure you'll use for actual research projects.

Organizing Your Obsidian Vault

Create folders within your `research-notes` vault:

```
research-notes/
    Research Topics/         # One note per interest
        Gaming Communities
        Social Media Toxicity
    Literature/              # Papers, articles, summaries
        Parasocial Interaction
        Social Identity Theory
    Methods/                 # Research design decisions
        Sampling Strategies
        Codebook Planning
    Templates/               # Reusable note formats
        Literature Review Template
        Research Question Template
    Archive/                 # Older, completed projects
```

Why this structure? - **Research Topics**: Start with curiosity; each idea gets its own note - **Literature**: Summarize what others have found (you'll systematize this in Chapter 4) - **Methods**: Record decisions as you make them (you'll formalize these in Chapters 6-9) - **Templates**: Build consistent formats for literature reviews, RQs, codebooks - **Archive**: Keep finished projects but don't clutter active work

This mirrors how you'll later organize your research project files:

```
my-research-project/
    data/          (raw, coded, analysis-ready)
    code/          (cleaning, analysis scripts)
    output/        (figures, tables, reports)
    docs/          (notes on decisions)
    README.md      (project overview)
```

Hands-On Practice: Markdown and Obsidian Skills

Exercise 2.1: Create a Research Topic Note

In Obsidian, create a new note called "Research Topic: [Your Topic]" (e.g., "Research Topic: Twitch Streamer Communities").

Use all the Markdown elements we covered:

```
# Gaming Communities on Twitch

## My Initial Observations

- Streamers with consistent chat engagement seem to build loyal communities
- Different games have different toxicity norms
- Voice chat vs. text chat affects communication dynamics

## Questions I Want to Explore

1. Do streamers who acknowledge chat (respond to messages) get more donations?
2. How do new viewers learn community norms?
3. Is voice communication associated with toxicity?

## Related Concepts

- **Parasocial interaction**: [[Parasocial Interaction Theory]]
- **Community norms**: [[Social Norms]]
- **Engagement metrics**: Donations, followers, chat frequency

## Initial Sources

I haven't read these yet, but they seem relevant:
- [Twitch Creator Camp - Building Community](https://www.twitch.tv/creatorcamp/)
- Academic articles on livestream communities
```

Learning goal: Practice Markdown formatting, organization, and linking ideas.

Exercise 2.2: Create Linked Notes

Create three new notes: 1. `Parasocial Interaction Theory` — Summarize the concept in 3-4 sentences 2. `Social Norms` — Explain what norms mean in research contexts 3. `Engagement Metrics` — Define: donations, followers, chat frequency

In your original "Gaming Communities" note, update the links:

```
- **Parasocial interaction**: [[Parasocial Interaction Theory|The illusion of personal connec
```

Now click those links. This is how you build a **knowledge graph**—connected ideas that form the foundation of your research.

Learning goal: Understand how ideas relate to each other; practice internal linking.

Exercise 2.3: Build a Literature Review Template

Create a new note called "Templates/Literature Review Template":

```
# Literature Review Template

## Source Information
- **Title**: [Article title]
- **Author(s)**: [Authors]
- **Year**: [Publication year]
- **URL**: [Link if available]
- **Type**: [Journal article, book chapter, blog post, etc.]

## Main Argument
In 2-3 sentences, what is this source's main claim?

## Key Findings
- Finding 1
- Finding 2
- Finding 3

## Relevance to My Research
How does this source relate to my topic?

## Methodology
- **Design**: (experimental, survey, content analysis, etc.)
- **Sample**: (how many people/documents? who?)
- **Data**: (what was measured?)

## Strengths
- What does this study do well?

## Limitations
```

```
- What are the weaknesses?

## Quotes Worth Remembering
> "Quote 1"
> "Quote 2"

## Tags
#literature #toxicity #communities
```

You'll use this template in Chapter 4 when reading papers. Having a structure now means you won't waste time deciding what to write down.

Learning goal: Build systems for organizing research information before you need them.

Exercise 2.4: Practice Table Creation

Create a note called "Data Planning" and build a table comparing different research data types:

```
# Types of Data for Media Research

| Data Type | Example | Pros | Cons |
|-----------|---------|------|------|
| **Text** | Chat logs, tweets, comments | Rich detail, natural language | Hard to quantify
| **Numerical** | Viewership, engagement counts | Easy to analyze | Lacks context |
| **Audio** | Podcast episodes, streams | Captures tone, emotion | Requires transcription |
| **Video** | Stream recordings, TikToks | Visual information | Large files, slow to analyze
| **Surveys** | Questionnaire responses | Directly asks questions | Low response rates |
```

Update the table with your own examples. You'll do this regularly when planning research (Chapter 8-10).

Learning goal: Organize information in structured formats (tables); practice thinking about different data types.

Exercise 2.5: Export and Share

Export your "Gaming Communities" note in three formats:

1. **Export as PDF**: Right-click > Export as PDF. Share this with a classmate for feedback.
2. **Copy as Markdown**: Right-click > Copy as Markdown. Paste into an email or Google Doc.

3. **Share via link**: Create a link and send the note to someone else in the class. Can they see your linked notes?

Learning goal: Understand how to share and present your work; practice collaborative research.

Exercise 2.6: Building a Codebook Planning Note

This exercise previews Chapter 9, but practicing now builds familiarity:

Create a note called "Codebook Planning: [Your Topic]" and start thinking about variables:

```
# Codebook Planning: Twitch Chat Toxicity

## Research Question
Is toxic chat more prevalent in competitive games (ranked matches) than casual games?

## Variables to Measure

### Independent Variable: Game Type
- **Definition**: Whether the stream shows ranked (competitive) or unranked (casual) gameplay
- **Possible Values**: Ranked, Unranked, Mixed
- **Decision Rules**:
  - If title says "ranked" = Ranked
  - If streamer is in ranked queue = Ranked
  - Otherwise = Unranked

### Dependent Variable: Message Toxicity
- **Definition**: Whether a chat message contains insulting, hateful, or derogatory language
- **Possible Values**: Toxic, Neutral
- **Examples**:
  - Toxic: "You're trash," "Get better," "Kill yourself"
  - Neutral: "nice play," "good job," "what rank are you?"
- **Decision Rules**:
  - If message contains insult = Toxic
  - If message is sarcastic insult = Toxic (because context matters)
  - Otherwise = Neutral
```

You'll refine this extensively in Chapter 9, but starting now means you're thinking systematically about measurement.

Learning goal: Bridge Obsidian practice to research planning; preview codebook construction.

Exercise 2.7: Reflection on Structure

In a note called "Reflection: Research Organization," answer these questions:

```
# Reflection: Why Organization Matters

## Why do researchers need systems?
- [Your answer]

## How does Obsidian help you think more clearly?
- [Your answer]

## What's the connection between organizing notes and research quality?
- [Your answer]

## What part of Obsidian feels most useful? Most frustrating?
- Most useful: [Your answer]
- Most frustrating: [Your answer]
```

Learning goal: Reflect on why workflow and structure matter before adding code complexity.

Reflection Questions

1. **Markdown vs. Code**: How is learning Markdown in Obsidian different from jumping straight into RStudio? What's the advantage of separating "thinking and writing" from "coding and analyzing"?
2. **Organization Philosophy**: Why does structure matter in research? How could poor organization lead to research errors?
3. **Delayed RStudio**: You deferred RStudio until Chapter 9. How does this change your relationship to code? Does deferring it feel good or frustrating?

Chapter Summary

This chapter introduced the **first part of the Publishing Engine**—the research thinking and note-taking infrastructure:

- **Obsidian** teaches Markdown—the formatting language underlying all reproducible documents
- **Markdown** allows structured writing: headings, lists, emphasis, links, tables

- **Organization** (vault structure, templates, note-linking) supports clear thinking
- **Reproducibility** begins with organized notes, not code—clarity first, automation second
- **RStudio, R, and Quarto** come later, once you've grounded your research questions and variables
- The **one-click report philosophy** requires one-click organization first

Key Terms

- **Backlink**: A link from one note to another (Obsidian feature allowing bidirectional linking)
- **Code chunk**: A section of executable code (will appear in Quarto; not yet in Obsidian)
- **Computational literacy**: The ability to read, write, and think with data and code
- **Markdown**: A lightweight markup language for formatting text using simple symbols
- **Note-linking**: Creating connections between related ideas using `[[bracketed references]]`
- **Obsidian**: A note-taking application for researchers and knowledge workers
- **Reproducibility**: The ability for others to regenerate your findings using your data and documented process
- **Vault**: An Obsidian folder containing interconnected notes
- **YAML**: A configuration language (will be important in Chapters 9-14 for Quarto documents)

Looking Ahead

Chapter 3 introduces the **rules of the game**—the epistemological foundations and ethical principles that constrain scientific inquiry. You'll learn how we distinguish knowledge from belief, why falsifiability matters, and how research ethics protect human dignity in the digital age. All your Obsidian notes on ethics, decisions, and methods will become crucial as you formalize your research design.

The Scientific Approach

Learning Objectives

- Distinguish between ways of knowing: tenacity, authority, intuition, and empiricism.
- Understand the principle of falsifiability and its role in hypothesis testing.
- Recognize ethical obligations in digital research, including respect for human dignity and privacy.

Overview

Having established curiosity (Chapter 1) and tools (Chapter 2), we now introduce discipline. Science is not just systematic curiosity—it's curiosity constrained by rules designed to minimize bias and maximize truth-seeking. This chapter explores **epistemology** (how we know what we know), **falsifiability** (what makes a claim scientific), and **research ethics** (the moral obligations that govern inquiry).

Epistemology: The Problem of Knowledge

How do you know that vaccines work? That climate change is real? That certain media messages influence behavior? Different people answer these questions using different **ways of knowing**.

Tenacity: "It's Always Been This Way"

Tenacity is belief sustained through repetition and tradition. If a claim is repeated often enough, it feels true. This is the epistemology of folklore, cultural norms, and inherited wisdom.

Example: "Violent video games cause real-world aggression" became conventional wisdom in the 1990s despite weak evidence. Parents, politicians, and pundits repeated the claim so often that it acquired the status of common sense.

Strength: Tenacity provides social cohesion. Shared beliefs bind communities.
Weakness: Tenacity resists new evidence. It privileges tradition over truth.

Authority: "An Expert Said So"

Authority-based knowledge relies on testimony from credible sources: professors, journalists, scientists, religious leaders. This is efficient—you can't personally verify every claim, so you defer to those with expertise.

Example: Most people believe the Earth orbits the Sun not because they've done the math, but because astronomers say so.

Strength: Authority allows knowledge to scale. Civilization depends on trust in specialized expertise.
Weakness: Authority can be abused. Experts can be wrong, biased, or corrupt. Appeals to authority shut down critical thinking.

Intuition: "It Feels Right"

Intuition is gut-level knowing. It's fast, automatic, and often accurate in familiar domains (experienced doctors diagnosing symptoms, chess masters recognizing patterns). But intuition is also vulnerable to cognitive biases.

Example: The "illusory truth effect" demonstrates that repeated exposure to a claim makes it *feel* true, even when it's false. This is why propaganda works.

Strength: Intuition is fast and often adaptive.
Weakness: Intuition is error-prone in novel or complex domains. It mistakes familiarity for truth.

Empiricism: "I Observed It Systematically"

Empiricism grounds knowledge in observation and measurement. The scientific method is formalized empiricism: make observations, form hypotheses, collect data, test predictions, revise theories.

Example: Claims about video game effects are tested through experiments, surveys, and longitudinal studies. We don't rely on intuition or anecdotes—we measure outcomes and calculate probabilities.

Strength: Empiricism is self-correcting. When new evidence contradicts old theories, theories change.
Weakness: Empiricism is slow and resource-intensive. It can't answer ethical questions ("Should we ban violent games?") only empirical ones ("Do they increase aggression?").

The scientific attitude: Science doesn't reject authority, intuition, or tradition. It subjects them to empirical scrutiny. A claim isn't true because it's old, popular, or endorsed by experts—it's true because it survives attempts to disprove it.

Falsifiability: The Demarcation Problem

Philosopher Karl Popper argued that science is defined not by verification but by **falsifiability**: the possibility of being proven wrong. A claim is scientific if you can specify conditions under which it would be false.

Falsifiable Claims

Example 1: "Streamers with webcams receive more donations than streamers without webcams."
This is falsifiable. You could collect data from 100 streamers, compare donation rates, and find no difference (or the opposite pattern). The claim is *at risk* of being proven false.

Example 2: "Exposure to negative news increases anxiety."
Falsifiable. You could randomly assign participants to read negative vs. neutral news, measure anxiety, and find no difference.

Unfalsifiable Claims

Example 1: "Good streamers are inherently likable."
Not falsifiable. "Good" and "likable" are circular—if a streamer isn't likable, you can redefine them as "not good." There's no observation that could disprove this.

Example 2: "Everything happens for a reason."
Not falsifiable. No matter what happens, the claim holds. Unfalsifiable claims may be meaningful (philosophically, spiritually) but they're not scientific.

Why falsifiability matters: Science advances by eliminating bad ideas. If your theory is immune to disconfirmation, it can't guide research. Falsifiable hypotheses force researchers to take intellectual risks—and that risk-taking produces reliable knowledge.

Induction vs. Deduction: Two Modes of Reasoning

Inductive Reasoning: Data to Theory

Induction moves from specific observations to general principles. You gather data, notice patterns, and infer a theory.

Example: After coding 500 tweets from esports fans, you notice that tweets mentioning rival games ("Valorant is trash") predict higher engagement (retweets, replies). You induce a theory: "In-group/out-group dynamics drive social media engagement in gaming communities."

Strength: Induction is exploratory and generative. It discovers new patterns.
Weakness: Induction is vulnerable to overgeneralization. Patterns in one sample may not replicate.

Deductive Reasoning: Theory to Data

Deduction moves from general principles to specific predictions. You start with a theory, derive a hypothesis, and test it.

Example: Social Identity Theory predicts that people enhance self-esteem by favoring their in-group and denigrating out-groups. You deduce a hypothesis: "Gamers will rate their preferred platform (Xbox, PlayStation, PC) more positively than rival platforms." You collect data to test this prediction.

Strength: Deduction is confirmatory and rigorous. It tests whether theories hold under scrutiny.
Weakness: Deduction assumes the starting theory is correct. If the theory is wrong, deduction can't correct it.

The research cycle: Science oscillates between induction (exploring data to generate theories) and deduction (testing theories with new data). Good researchers do both.

Ethics in the Digital Age

Research ethics emerged from historical atrocities: Nazi experiments, Tuskegee syphilis study, Milgram's obedience experiments. These cases established principles codified in the Belmont Report (1979):

1. **Respect for Persons**: Treat participants as autonomous agents capable of informed consent.
2. **Beneficence**: Maximize benefits, minimize harms.
3. **Justice**: Distribute research risks and benefits fairly.

Digital media research complicates these principles. Is a public tweet "human subjects research"? What about scraping Reddit data? Who owns the data trail you leave on social media?

The IRB: Gatekeepers of Ethical Research

The **Institutional Review Board (IRB)** reviews research proposals to ensure ethical compliance. At most universities, any research involving human subjects requires IRB approval or exemption.

When is IRB approval required?

- **Surveys or interviews**: You're collecting data directly from people.
- **Experiments**: You're manipulating conditions that affect participants.
- **Observation of private behavior**: You're studying behavior in non-public settings (e.g., private Discord servers).

When is research typically exempt?

- **Analysis of public media**: Coding news articles, YouTube videos, or public tweets typically qualifies for exemption because the data is already public and involves no interaction with participants.
- **Secondary data analysis**: Using existing datasets (e.g., Pew Research surveys) where identifiers have been removed.

Informed Consent in the Age of Big Data

Traditional informed consent assumes people know they're being studied and agree to participate. But digital platforms generate data constantly. Did you consent to have your likes, shares, and viewing patterns analyzed by researchers?

The "Public vs. Private" Problem

A tweet is technically public—anyone can see it. But users don't necessarily expect tweets to be aggregated, analyzed, and quoted in academic papers. Context collapse occurs when content intended for one audience (followers) is repurposed for another (researchers).

Best practice: Even when studying public data, consider:

- **Platform Terms of Service**: Does scraping violate the platform's rules?
- **User Expectations**: Would users be surprised to learn their content is being studied?
- **Potential Harm**: Could publishing findings expose users to harassment or doxxing?

Inclusion and Representation: Ethics Beyond Harm Prevention

Early medical research disproportionately studied white men, producing findings that didn't generalize to women or people of color. Mass communication research faces analogous problems: gaming research oversamples young, white, male players; social media studies focus on English-language content.

Ethical sampling requires actively seeking diverse perspectives. If your study claims to describe "gamers" but only includes competitive FPS players, you're making exclusionary claims.

The Sacred Trust of Science

Science enjoys public trust because it adheres to norms: transparency, peer review, replication, self-correction. As a novice researcher, you inherit this trust. Upholding it requires honesty, transparency, rigor, and respect.

Navigating the IRB Process

Understanding IRB Categories

Institutional Review Boards categorize research into three levels based on risk:

Exempt Research

Minimal risk, typically observational or using publicly available data:

- Analysis of public social media posts
- Content analysis of published news articles
- Secondary analysis of de-identified datasets

Example: Coding 200 public YouTube comments for sentiment = likely exempt

Process: Submit abbreviated application. Review typically takes 1-2 weeks.

Expedited Review

Minimal risk but involves interaction with participants:

- Anonymous surveys with non-sensitive questions
- Interviews about non-sensitive topics
- Observational studies in public settings

Example: Surveying college students about their favorite streaming platforms = expedited

Process: Single reviewer examines protocol. Review takes 2-4 weeks.

Full Board Review

Higher risk or vulnerable populations:

- Studies involving deception
- Research with minors, prisoners, or cognitively impaired individuals
- Studies collecting sensitive information (mental health, illegal behavior)

Example: Interviewing teenagers about cyberbullying experiences = full board

Process: Committee meets monthly to review. Can take 4-8 weeks.

Writing an IRB Protocol

Key sections of an IRB application:

1. **Purpose and Objectives**: What question are you answering?
2. **Background**: Why is this study necessary?
3. **Participant Population**: Who will you recruit? How many?
4. **Recruitment**: How will you find participants? (Include recruitment scripts)
5. **Procedures**: What will participants do? How long will it take?
6. **Risks**: What physical, psychological, social, or economic risks exist?
7. **Benefits**: How might participants or society benefit?
8. **Confidentiality**: How will you protect participant identity?
9. **Informed Consent**: What will participants be told before agreeing?

Sample Consent Language

For a Survey Study:

> You are invited to participate in a research study examining social media use among college students. Participation involves completing a 10-minute online survey. Your responses will be anonymous—no identifying information will be collected. There are no known risks beyond those of everyday computer use. Your participation is voluntary, and you may skip questions or withdraw at any time without penalty.

Ethical Dilemmas in Digital Research

Case Study 1: The Facebook Emotional Contagion Study (2014)

What happened: Facebook and academic researchers manipulated 689,000 users' News Feeds to show more positive or negative content, then measured whether users' own posts became more positive or negative.

Ethical issues:

- No informed consent (users didn't know they were in a study)
- Potential psychological harm (exposing people to negative content)
- Power imbalance (Facebook controlled users' information environment)

The defense: Facebook's Terms of Service allowed "data analysis" for research, and the manipulation was minimal compared to normal algorithmic curation.

The verdict: The study sparked widespread criticism and led to new ethical guidelines for platform-based research.

Discussion question: If analyzing public social media posts is exempt, why was this study ethically problematic?

Case Study 2: The OkCupid Dataset (2016)

What happened: Researchers scraped 70,000 OkCupid dating profiles (usernames, locations, answers to personal questions about sexuality and politics) and posted the dataset publicly "for research."

Ethical issues:

- Violated OkCupid's Terms of Service
- Exposed users' sexual orientations, political views, and identities
- Data could be used to doxx or harass individuals
- No consent, no IRB approval

The defense: Profiles were "publicly accessible" (though behind login walls).

The verdict: The dataset was removed from most repositories, but had already been downloaded thousands of times.

Discussion question: If a profile is technically "public," does that make it ethical to scrape and republish?

Case Study 3: The Grindr Location Data Leak (2018)

What happened: Researchers demonstrated that Grindr (a gay dating app) leaked precise location data, allowing anyone to track users' movements. LGBTQ+ users in countries where homosexuality is criminalized were at risk.

Ethical issues:

- Researchers had to collect sensitive data to prove the vulnerability
- Publishing findings could help fix the problem OR teach bad actors how to exploit it
- Balancing public benefit (security awareness) with individual harm (outing users)

Resolution: Researchers coordinated with Grindr before publication. The company fixed the vulnerability.

Discussion question: Is it ethical to collect sensitive data to demonstrate a security flaw?

Practice: Applying Ethical Reasoning

Exercise 3.1: Categorizing Research Ethics

For each scenario, determine:

1. IRB category (Exempt / Expedited / Full Board)
2. Key ethical concerns
3. How you would address those concerns

Scenario A: You want to analyze 500 tweets containing #GamerGate to study online harassment. Tweets are public. You will not contact users.

Scenario B: You want to survey 100 Twitch viewers about why they donate to streamers. Survey is anonymous, 15 minutes, asks about income and motivations.

Scenario C: You want to interview 10 female streamers about their experiences with sexual harassment. Interviews will be audio-recorded and ask about traumatic experiences.

Exercise 3.2: Designing Informed Consent

You're conducting a study where participants play a violent video game (Call of Duty) for 30 minutes, then complete a survey measuring aggression and mood.

Write an **informed consent statement** (150-200 words) that includes:

- Purpose of the study

- What participants will do
- Time commitment
- Potential risks (psychological discomfort, exposure to violent content)
- Benefits (contribution to science, $10 gift card)
- Voluntary nature and right to withdraw

Exercise 3.3: Epistemology in Action

For each claim, identify the **way of knowing** (Tenacity / Authority / Intuition / Empiricism) and evaluate its reliability:

1. "Video games cause violence because everyone knows that."
   - **Way of knowing**: _____
   - **Reliability**: _____
2. "The American Psychological Association released a report concluding that violent video games increase aggressive thoughts."
   - **Way of knowing**: _____
   - **Reliability**: _____
3. "I've played violent games for 10 years and never hurt anyone, so games don't cause violence."
   - **Way of knowing**: _____
   - **Reliability**: _____
4. "A meta-analysis of 150 studies found a small but statistically significant correlation (r = .15) between violent game exposure and aggressive behavior."
   - **Way of knowing**: _____
   - **Reliability**: _____

Reflection Questions

1. **Ethical Tensions**: Can you think of a research question you're interested in that might pose ethical dilemmas? (e.g., studying hate speech, analyzing data from marginalized communities) How would you balance scientific value against potential harms?
2. **Falsifiability Exercise**: Write a claim about media that is **unfalsifiable** (cannot be proven wrong). Then revise it to be **falsifiable** (can be tested empirically).
3. **Your Epistemological Mix**: Reflect on a belief you hold about media (e.g., "social media makes people lonelier"). What **way of knowing** originally convinced you? How would you test it empirically?

Chapter Summary

This chapter established the **epistemological and ethical foundations** of scientific inquiry:

- **Epistemology** examines how we know what we know: tenacity (tradition), authority (experts), intuition (gut feeling), and empiricism (systematic observation)
- **Falsifiability** (Popper) distinguishes science from non-science: scientific claims must be testable and potentially disprovable
- **Inductive reasoning** moves from observations to theory; **deductive reasoning** moves from theory to testable predictions
- Research ethics protect human dignity through three principles: **respect for persons** (autonomy), **beneficence** (maximize good, minimize harm), and **justice** (fair distribution of risks/benefits)
- The **IRB** reviews research protocols to ensure ethical compliance
- Digital research complicates traditional ethics: public data isn't necessarily "fair game," context collapse creates new risks, and platform Terms of Service matter
- **Informed consent** requires participants understand what they're agreeing to, the risks involved, and their right to withdraw

Key Terms

- **Authority**: Knowledge based on expert testimony
- **Belmont Report**: 1979 document establishing ethical principles for human subjects research
- **Beneficence**: Ethical principle of maximizing benefits and minimizing harms
- **Context collapse**: When content intended for one audience is repurposed for another
- **Deductive reasoning**: Deriving specific predictions from general theories
- **Empiricism**: Knowledge based on systematic observation and measurement
- **Epistemology**: The study of knowledge (how we know what we know)
- **Falsifiability**: The criterion that scientific claims must be potentially disprovable
- **Informed consent**: Participants' voluntary agreement to participate based on full disclosure of risks and procedures
- **Inductive reasoning**: Inferring general principles from specific observations
- **IRB (Institutional Review Board)**: Committee that reviews research protocols for ethical compliance
- **Intuition**: Knowledge based on gut feelings or rapid automatic processing
- **Justice**: Ethical principle that research risks and benefits should be distributed fairly
- **Respect for persons**: Ethical principle treating participants as autonomous agents
- **Tenacity**: Knowledge sustained through repetition and tradition
- **Type I Error**: False positive (rejecting a true null hypothesis)
- **Type II Error**: False negative (failing to reject a false null hypothesis)

Looking Ahead

Chapter 4 transitions from philosophy to practice with **Intelligence Gathering**—learning to conduct systematic literature reviews, use academic databases, manage sources in Zotero, and map the scholarly terrain before collecting new data.

Intelligence Gathering

Learning Objectives

- Conduct systematic Open Source Intelligence (OSINT) gathering for media research.
- Use academic databases (Nexis Uni) and social media archives strategically.
- Organize sources in Zotero as a research database.
- Distinguish between exploratory searching and systematic evidence gathering.

Overview

Before collecting new data, you must exhaustively map what already exists. This chapter trains you to think like an **archivist**: searching systematically, documenting every step, and building a database of evidence. In an era of information abundance, the challenge is not finding information—it's finding the *right* information and knowing when you've found enough.

The Archivist Mindset

Intelligence Gathering vs. "Googling"

"Googling" is exploratory and unsystematic. You type keywords, click the first three results, and call it research. **Intelligence Gathering** is deliberate, reproducible, and documented. An effective archivist:

- Uses **Boolean operators** (AND, OR, NOT) to craft precise queries.
- Records search strings, databases used, date ranges, and result counts.
- Tags and annotates findings for future retrieval.
- Iteratively refines search terms based on what is (and isn't) found.

Example: A casual search for "Twitch toxicity" returns 2.3 million results—useless. A systematic search uses:

```
"Twitch" AND ("toxicity" OR "harassment" OR "hate speech") AND ("streamer" OR "viewer") NOT "
```

This narrows results to relevant scholarly and journalistic sources. You document the exact string, the database (Google Scholar, Nexis Uni), the date you searched, and how many results were returned.

The Map: Literature Review as Territory

Your literature review is a map. It shows:

- **What is already known** (settled islands): "Multiple studies confirm that anonymity increases toxic behavior in online gaming."
- **What gaps remain** (unexplored waters): "Few studies examine toxicity in live-streaming contexts where streamers moderate chat in real time."
- **Where your study fits** (the new island you'll chart): "I will analyze chat logs from 50 Twitch streams to assess whether moderated channels show reduced toxicity compared to unmoderated channels."

You cannot claim to discover something "new" if you don't know the map. Literature reviews aren't summaries—they're arguments for why your study matters.

Mining the Archives

Nexis Uni (News & Legal Archives)

Nexis Uni provides access to newspapers, magazines, legal documents, and broadcast transcripts. It's especially valuable for tracking media coverage over time.

Search strategy:

1. **Define date range**: If studying 2024 esports coverage, search Jan 1, 2024 - Dec 31, 2024.
2. **Use Boolean operators**: `("esports" OR "e-sports" OR "competitive gaming") AND ("mainstream media" OR "legitimacy")`.
3. **Refine by source type**: Major newspapers (NYT, WSJ), trade publications (The Esports Observer), or all news.
4. **Export results**: Download to CSV or send to Zotero for coding.

Pro tip: Use proximity operators like `W/5` (within 5 words) to capture phrases: `"esports W/5 Olympics"` finds articles discussing esports' Olympic candidacy.

Social Media Archives

Platforms like Twitter, Reddit, and YouTube are both research objects (you study them) and research tools (you find sources there).

Note for beginners: Several advanced tools exist for collecting social media data programmatically (like Pushshift API for Reddit, YouTube Data API, etc.). **You will not use these independently in this course**—your instructor will provide pre-collected datasets or assist with data collection using these tools. For now, focus on:

- **Manual searches**: Using platform search features (e.g., Twitter Advanced Search, Reddit search)
- **Downloadable archives**: Pre-existing datasets your instructor provides
- **Published research**: Finding studies that already analyzed social media data

Why mention APIs now? You should know these tools exist for future research. After this course, if you pursue graduate studies or professional research roles, you may learn to use them. For now, think of them as "advanced tools your instructor uses behind the scenes."

Ethical note: Just because data is public doesn't mean it's ethically fair game. Always consider user expectations and potential harms.

Academic Databases

- **Google Scholar**: Broad search across disciplines. Good for exploratory searching.
- **Communication & Mass Media Complete**: Discipline-specific database. Best for communication theory and empirical studies.
- **PsycINFO**: Psychology database. Useful for media effects, parasocial relationships, and cognitive studies.
- **Web of Science / Scopus**: Citation indexing. Find who cited a seminal study to trace the evolution of an idea.

Zotero: Your External Brain

Zotero is a free, open-source reference manager. Think of it as a database where you store every source you encounter—articles, books, news stories, tweets, YouTube videos.

Why Zotero Beats Google Docs

Google Docs lists sources linearly. Zotero allows:

- **Tags**: Tag sources with themes (`toxicity`, `gender`, `streaming`).
- **Collections**: Organize by project or chapter.
- **Notes**: Attach annotations and summaries to each source.
- **Search**: Full-text search across PDFs and notes.
- **Citations**: Auto-generate bibliographies in APA, MLA, Chicago.

How to Use Zotero

1. **Install Zotero**: Desktop app + browser connector.
2. **Capture sources**: When you find an article in Google Scholar or a book on Amazon, click the Zotero connector icon. It auto-captures metadata (author, title, publication, DOI).
3. **Tag and annotate**: Add tags like `lit-review`, `methodology`, `key-study`. Write notes summarizing key findings.
4. **Build collections**: Create a collection for your project. Drag relevant sources into it.
5. **Generate citations**: In Word/Google Docs, use the Zotero plugin to insert citations and auto-format your bibliography.

Pro tip: Use the "Related" feature to link sources that build on each other. This visualizes how ideas evolve across studies.

Real Data: The Complete Shakespeare Works

To practice literature review with actual archival data, we'll use **The Complete Works of William Shakespeare**, available via TidyTuesday (dataset: September 17, 2024).

Why Shakespeare for a research methods course? - Structured, complete text archive (all 37 plays + sonnets) - Rich with research questions: gender representation, word frequency by character, scene structure - Public domain (ethical access, no permissions needed) - Demonstrates how literature scholars conduct text analysis

Accessing the Data

The Shakespeare dataset is freely available. You have two options:

Option 1: Direct download 1. Visit: https://github.com/rfordatascience/tidytuesday/blob/main/data/2024/2024-09-17/readme.md 2. Look for the "shakespeare" CSV files 3. Download to your computer

Option 2: Load directly in R (you'll do this starting Chapter 11)

```
# Once you have R/RStudio, you can load directly from GitHub
shakespeare <- readr::read_csv('https://raw.githubusercontent.com/rfordatascience/tidytuesday
```

Research Questions You Could Ask

With this data, systematic literature review would ask:

- **Descriptive**: "Which characters speak the most lines? How does this vary by gender?"
- **Relational**: "Do female-led plays have different word patterns than male-led plays?"
- **Historical**: "How does Shakespeare's language use evolve across his career (early vs. late plays)?"

Each research question requires a **search strategy**: - Search academic databases for papers analyzing Shakespeare's gender representation - Search digital humanities projects for text analysis of Shakespeare - Search literary journals for studies on character development, language patterns

Documenting Your Literature Review

When you review literature on Shakespeare (or your own topic), create a **search log** in your Obsidian vault:

Database	Search String	Date	Results	Notes
Google Scholar	"Shakespeare" AND "gender representation" AND "plays"	2024-12-10	4,200	Too broad
JSTOR	"Shakespeare gender" AND character	2024-12-10	89	Better focused

Database	Search String	Date	Results	Notes
Project Muse	"female characters" AND Shakespeare AND analysis	2024-12-10	34	Most relevant

This systematic approach mirrors how you'll review literature for your actual research project in Chapters 8-14.

The Search is Part of the Argument

Every search decision is a theoretical choice:

- Searching "gamers" vs. "players" vs. "users" assumes different identities and communities.
- Searching 2020-2024 vs. 2000-2024 privileges recent vs. historical contexts.
- Excluding industry sources (IGN, Polygon) vs. including them reflects assumptions about journalistic vs. academic authority.

Document these choices in a **search log**:

Database	Search String	Date	Results	Notes
Google Scholar	"Twitch" AND "parasocial"	2024-10-15	3,200	Too broad
Comm & Mass Media Complete	"Twitch" AND "parasocial" AND "streamer"	2024-10-15	87	Manageable

This log becomes an appendix in your final report, proving you conducted a systematic review.

When to Stop Searching

You've reached **saturation** when:

- New searches return sources you've already seen.
- Sources cite the same foundational studies repeatedly.
- You can predict what a new source will argue based on its title.

Saturation doesn't mean you've found *everything*—it means you've found *enough* to speak authoritatively about your topic.

Case Study: Mapping Twitch Toxicity Research

Research question: How has academic research on toxicity in live-streaming evolved from 2016-2024?

Step 1: Exploratory search on Google Scholar (`"Twitch" AND "toxicity"`) returns 4,100 results.

Step 2: Refine search (`"Twitch" AND "toxicity" AND ("chat" OR "moderation")`) returns 892 results.

Step 3: Filter by date (2016-2024) and relevance. Identify 45 peer-reviewed articles.

Step 4: Import to Zotero. Tag by theme (`moderation`, `gender`, `race`, `platform-design`).

Step 5: Read abstracts. Identify 3 foundational studies cited by most others:

- Massanari (2017): Toxic technocultures on Reddit
- Gray (2014): Racism, misogyny, and trolling in gaming
- Nakamura (2015): Cybertyping and representation

Step 6: Use "cited by" feature in Google Scholar to find recent studies building on these foundations.

Result: A map showing that early research focused on user behavior (2016-2019), while recent research examines platform design and moderation (2020-2024). Your study could fill the gap by analyzing AI-based moderation tools.

Practice: Developing Search Skills

Exercise 4.1: Boolean Search Construction

Construct effective Boolean search strings for the following research topics:

1. **Topic**: The relationship between esports viewing and traditional sports consumption
   - **Search string**: _____
   - **Rationale**: Why did you include/exclude specific terms?
2. **Topic**: How TikTok influencers disclose sponsored content
   - **Search string**: _____
   - **Databases to try**: _____

3. **Topic**: Representation of women in gaming journalism
    - **Search string**: _____
    - **Expected challenges**: _____

Exercise 4.2: Building Your Search Log

Choose a research topic you're interested in. Conduct three searches using different strategies:

Search 1 (Broad exploratory): - Database: Google Scholar - Search string: _____ - Results: _____ - Assessment: Too broad / Too narrow / Just right

Search 2 (Refined): - Database: Communication & Mass Media Complete OR PsycINFO - Search string: _____ - Results: _____ - Assessment: _____

Search 3 (Targeted): - Database: _____ - Search string: _____ - Results: _____ - Top 3 most relevant sources: _____

Exercise 4.3: Zotero Practice

If you have Zotero installed:

1. Create a collection called "Research Methods Practice"
2. Find 5 sources related to your research interest (mix of academic articles, news stories, websites)
3. Import them into Zotero using the browser connector
4. Add at least 2 tags to each source
5. Write a 2-sentence annotation for each source in the Notes field

If you don't have Zotero yet: Download and install it, then complete the exercise.

Exercise 4.4: Saturation Assessment

Read these five article abstracts (hypothetical titles):

1. "Parasocial Relationships with Twitch Streamers" (2019)
2. "Viewer Donations as Parasocial Reciprocity" (2020)
3. "The Role of Webcam Intimacy in Streamer-Viewer Bonds" (2021)
4. "Parasocial Interaction Theory and Live-streaming" (2022)
5. "Do Viewers Really Know Their Streamers? A PSI Study" (2023)

Questions: - What patterns do you notice in how the topic evolves? - What foundational concepts appear repeatedly? - If these were real sources, would you conclude you've reached saturation? Why or why not? - What gap might still exist?

Reflection Questions

1. **Your Archive Strategy**: Before this chapter, how did you typically search for information (academic or otherwise)? What habits do you need to change to conduct systematic, reproducible searches?
2. **The Citation Trail**: Why is it valuable to look at "cited by" references in Google Scholar? How does following citation networks help you understand the evolution of ideas?
3. **The Gray Literature Question**: Should your literature review include non-academic sources like blog posts, industry reports, or journalism? When are these sources valuable? When should you exclude them?

Chapter Summary

This chapter trained you to think like an **archivist**, conducting systematic intelligence gathering:

- **OSINT (Open Source Intelligence)** gathering requires systematic, documented, reproducible searching—not casual Googling
- **Boolean operators** (AND, OR, NOT) and proximity operators (W/5) create precise search strings
- Key databases include **Nexis Uni** (news), **Google Scholar** (broad), **Communication & Mass Media Complete** (discipline-specific), and platform APIs (social media)
- **Zotero** organizes sources, stores PDFs, enables tagging/annotation, and auto-generates citations
- Every search decision is a **theoretical choice**—document your rationale in a search log
- **Saturation** occurs when new searches return familiar sources and ideas; this signals you've mapped the terrain adequately
- Literature reviews are **arguments** for why your study fills a gap, not just summaries of existing research

Key Terms

- **Boolean operators**: Logical connectors (AND, OR, NOT) used to refine database searches
- **Citation trail**: Following references backward (works cited) and forward (works that cite it) to map scholarly conversations
- **Gray literature**: Non-peer-reviewed sources (reports, white papers, blog posts)
- **Literature review**: Systematic survey of existing research on a topic
- **Nexis Uni**: Database providing access to news archives, legal documents, and business information
- **OSINT (Open Source Intelligence)**: Gathering information from publicly available sources using systematic methods
- **Proximity operators**: Search tools (e.g., W/5) that find terms within a specified distance of each other
- **Saturation**: The point at which new searches yield diminishing returns (no new themes or sources)
- **Search log**: Documented record of search strategies, including databases, search strings, dates, and results
- **Zotero**: Free, open-source reference management software for organizing sources and generating citations

Looking Ahead

Chapter 5 explores **Theory as a Lens**—how theoretical frameworks focus your attention, generate hypotheses, and transform observation into explanation. You'll learn to identify independent and dependent variables, distinguish between grand theories and middle-range theories, and apply theoretical reasoning to your research question.

Theory as a Lens

Learning Objectives

- Understand theory as a lens that directs attention to specific aspects of media phenomena.
- Identify Independent Variables (IVs) and Dependent Variables (DVs) in published research.
- Apply a theoretical framework to your own research question.
- Distinguish between grand theories, middle-range theories, and empirical generalizations.

Overview

A theory is not a vague speculation—it's a precision tool. Like a camera lens, it brings certain elements into sharp focus while blurring others. This chapter demystifies theory by framing it as a practical instrument for shaping research design.

The Lens Metaphor

Imagine photographing a crowded esports arena:

- A **wide-angle lens** captures the entire venue (Uses & Gratifications: why do people attend?).
- A **telephoto lens** zooms in on one player's face (Emotional Contagion: how does their expression affect viewers?).
- A **fisheye lens** distorts for artistic effect (Cultivation Theory: does repeated exposure to esports normalize competitive aggression?).

Each lens (theory) reveals different patterns. You can't use all lenses simultaneously—you must choose one that fits your research question.

Levels of Theory

Grand Theories

Grand theories explain broad swaths of human behavior: Marxism, Psychoanalysis, Social Constructivism. They're philosophically rich but empirically vague.

Example: "Media industries reproduce capitalist power structures" (Critical Theory). True, perhaps, but hard to test with a content analysis of Twitch streams.

Middle-Range Theories

Middle-range theories operate at the "Goldilocks" level: specific enough to generate testable hypotheses, broad enough to apply across contexts. Most communication research uses middle-range theories.

Examples:

- **Social Identity Theory**: People derive self-esteem from group membership.
- **Agenda Setting**: Media influence what audiences think *about*, not what they think.
- **Parasocial Interaction**: Audiences form one-sided emotional bonds with media figures.

Empirical Generalizations

These are narrow, context-specific patterns: "Streamers who use webcams receive more donations than those who don't." Not a theory—just a regularity that invites theoretical explanation.

Theories Relevant to Mass Communication & SIM Lab Research

Parasocial Interaction (PSI)

Developed by Horton and Wohl (1956), PSI describes one-sided emotional relationships between audiences and media figures. Viewers treat TV hosts, streamers, or influencers as friends despite no reciprocal relationship.

Core concepts:

- **Illusion of intimacy**: Streamers address viewers directly, creating a sense of personal connection.
- **Pseudo-reciprocity**: Viewers respond as if the relationship is mutual (gifting subscriptions, defending streamers from criticism).

Empirical prediction: Viewers who watch the same streamer for >10 hours/week will report higher PSI scores (measured via validated scales) than casual viewers.

SIM Lab application: Does PSI with streamers predict donation behavior? Does it vary by platform (Twitch vs. YouTube Gaming)?

Social Identity Theory (Tajfel & Turner, 1979)

People categorize themselves into social groups (race, nationality, fandom). These memberships shape identity and self-esteem. When group identity is threatened, people defensively enhance in-group status and denigrate out-groups.

Core concepts:

- **In-group favoritism**: "My console is superior."
- **Out-group derogation**: "Their console is trash."
- **Self-esteem maintenance**: Defending your group defends your self-worth.

Empirical prediction: Players who identify strongly as "Valorant mains" will rate competing games (Overwatch, CS:GO) more negatively than players with weak Valorant identity.

SIM Lab application: Analyze console war discourse on Reddit (r/PlayStation vs. r/Xbox vs. r/PCMasterRace). Does in-group/out-group rhetoric predict community toxicity?

Agenda Setting (McCombs & Shaw, 1972)

Media doesn't tell audiences *what* to think, but *what to think about*. By covering certain topics frequently, media shape public priorities.

Core concepts:

- **First-level agenda setting**: Media coverage of Issue X increases public salience of Issue X.
- **Second-level agenda setting**: Media framing of Issue X (as crisis vs. opportunity) shapes how audiences think about Issue X.

Empirical prediction: Games featured on Twitch's front page experience a 20% increase in average concurrent viewers within 48 hours (media sets the gaming agenda).

SIM Lab application: Do esports news cycles (championship coverage) predict Reddit discussion volume? Does mainstream media coverage (NYT articles) legitimize esports for non-gamers?

Framing Theory (Entman, 1993)

Frames are organizing principles that shape how information is presented and interpreted. The *same* event can be framed differently, leading to divergent audience responses.

Example:

- **Frame 1**: "Esports athletes train 12 hours/day to compete at the highest level." (Legitimacy frame)
- **Frame 2**: "Video game addicts spend 12 hours/day staring at screens." (Pathology frame)

Empirical prediction: News articles framing esports as "competitive sport" generate more positive reader comments than articles framing esports as "addictive pastime."

SIM Lab application: Code 200 news articles about esports. Identify dominant frames. Test whether frames predict sentiment in reader comments.

Uses & Gratifications (Katz, Blumler, & Gurevitch, 1973)

Audiences actively select media to satisfy specific needs: information, entertainment, social connection, identity formation. This theory asks "What do people *do* with media?" rather than "What do media *do* to people?"

Core gratifications:

- **Surveillance**: Staying informed (news, updates).
- **Diversion**: Escape and entertainment (comedy, gaming).
- **Personal identity**: Reinforcing self-concept (fandom, lifestyle media).
- **Social utility**: Connecting with others (live chat, co-viewing).

Empirical prediction: Twitch viewers seek different gratifications than YouTube viewers. Twitch satisfies social utility (live chat), while YouTube satisfies diversion (on-demand binge-watching).

SIM Lab application: Survey 500 streaming viewers. Measure gratifications sought vs. gratifications obtained. Do discrepancies predict platform switching?

Real Data: Stack Overflow Developer Survey 2024

To practice theory application with real data, we'll examine the **Stack Overflow Annual Developer Survey 2024**, available via TidyTuesday (September 3, 2024).

Why this data for theory? - Large sample (~90,000 developers globally) - Rich variables: programming languages, job satisfaction, experience level, salary, demographics - Perfect for testing Social Identity Theory (programmer communities, language loyalty) - Demonstrates how technology companies use research to understand user behavior

Accessing the Data

Download from TidyTuesday: 1. Visit: https://github.com/rfordatascience/tidytuesday/blob/main/data/2024/2024-09-03/readme.md 2. Download the Stack Overflow survey CSV files 3. Load into R when you reach Chapter 11

Applying Theory to Stack Overflow Data

Research Question: Do developers show in-group favoritism for their preferred programming language (Social Identity Theory)?

Hypothesis: Developers who use Language A will rate Language A more positively than Language B, even when controlling for objective metrics like salary and job growth.

Variables: - **IV**: Programming language preference (Python, Java, JavaScript, C++, etc.) - **DV**: Job satisfaction rating (1-5 scale) - **Control**: Years of experience, salary, job industry

How to test it (you'll do this in Chapter 13):

```
# Load survey data
stack_overflow <- read.csv("stack_overflow_survey_2024.csv")

# Compare job satisfaction by language
by_language <- stack_overflow %>%
  group_by(primary_language) %>%
  summarize(
    mean_satisfaction = mean(job_satisfaction, na.rm = TRUE),
    n_developers = n()
  )

# Visualize: Do developers using Language X rate it higher?
# In-group bias would show as elevated satisfaction for own language
```

Why This Matters

This analysis reveals **computational tribalism**: the tendency for programmer communities to favor their own language/tools, similar to how sports fans favor their home team. Understanding this bias is crucial for technology companies designing communities and for researchers studying online subcultures.

From Theory to Hypothesis

A theory becomes useful when it generates **testable hypotheses**. Here's the process:

Step 1: Choose a Theory

Example: Parasocial Interaction (PSI)

Step 2: Identify Concepts

PSI theory includes concepts like:

- **Para social interaction intensity** (how strongly viewers feel connected)
- **Exposure frequency** (hours watched per week)
- **Behavioral outcomes** (donations, subscriptions, defending the streamer)

Step 3: Define Variables

- **Independent Variable (IV)**: Exposure frequency (hours watched per week)
- **Dependent Variable (DV)**: Donation behavior (dollars donated per month)
- **Mediator**: PSI intensity (measured via survey scale)

Step 4: State the Hypothesis

H1: Viewers who watch Streamer X for >10 hours/week will report higher PSI intensity than viewers who watch <5 hours/week.

H2: PSI intensity will predict monthly donation amounts, controlling for viewer income.

Variables: The Building Blocks of Hypotheses

Independent Variables (IV)

The **cause** or predictor. What you manipulate (in experiments) or measure (in correlational studies) to see if it affects something else.

Example: Streamer webcam use (present vs. absent)

Dependent Variables (DV)

The **effect** or outcome. What you measure to see if it changes in response to the IV.

Example: Viewer donations (dollars per stream)

Mediators

Variables that *explain* the relationship between IV and DV.

Example: Webcam use → Perceived intimacy → Donations

Webcam use doesn't directly cause donations. It increases perceived intimacy, which then increases donations. Perceived intimacy is the mediator.

Moderators

Variables that *change* the strength or direction of the IV-DV relationship.

Example: Webcam use increases donations for streamers with <1,000 followers, but not for streamers with >10,000 followers (follower count moderates the effect).

Thinking Like a Theorist

Theories aren't sacred texts. They're tools. If a theory doesn't fit your research question, don't force it. Ask:

1. **Does this theory explain my phenomenon?** If you're studying how Twitch algorithms recommend content, PSI theory won't help—you need platform studies or algorithmic bias theories.
2. **Does this theory generate testable predictions?** If a theory says "media influences society," it's too vague.
3. **Has this theory been tested in similar contexts?** If PSI has been validated for TV hosts but never for streamers, your study extends the theory—that's valuable.

Case Study: Applying Theory to a Research Question

Research Question: Why do some Twitch streamers receive more donations than others?

Theory Selection: Parasocial Interaction (PSI)

Theoretical reasoning: PSI predicts that viewers donate to streamers they feel emotionally connected to. Donations are a form of pseudo-reciprocity—viewers "give back" to streamers who make them feel valued.

Hypothesis:

- **H1**: Streamers who use webcams (increasing perceived intimacy) will receive higher average donations than streamers without webcams.
- **H2**: Streamers who respond to chat messages frequently (increasing interaction) will receive higher donations.
- **H3**: PSI intensity (measured via viewer surveys) will mediate the relationship between interaction frequency and donations.

Operationalization:

- **IV1**: Webcam use (yes/no)
- **IV2**: Chat response rate (% of chat messages acknowledged)
- **DV**: Average donation per stream (dollars)
- **Mediator**: PSI intensity (7-point Likert scale)

Data collection:

- Observe 50 Twitch streams. Code webcam use and chat interactions.
- Survey 200 viewers across those streams. Measure PSI using the PSI-Process Scale (Schramm & Hartmann, 2008).
- Correlate PSI scores with self-reported donation behavior.

Practice: Applying Theoretical Frameworks

Exercise 5.1: Identifying Variables

For each hypothesis, identify the **IV (independent variable)**, **DV (dependent variable)**, and any **mediators or moderators**:

1. "Streamers who play competitive games receive more toxic chat messages than streamers who play casual games."
    - **IV**: _____
    - **DV**: _____

2. "Exposure to influencer cryptocurrency promotions predicts investment behavior, but only among followers with low financial literacy."
   - **IV**: _____
   - **DV**: _____
   - **Moderator**: _____
3. "News framing of esports as 'legitimate sport' increases reader approval of esports, mediated by perceptions of skill and athleticism."
   - **IV**: _____
   - **DV**: _____
   - **Mediator**: _____

Exercise 5.2: Theory Selection

For each research question, select the MOST appropriate theory from this chapter and explain why:

Theories to choose from: Parasocial Interaction, Social Identity Theory, Agenda Setting, Framing Theory, Uses & Gratifications

1. **RQ**: "Why do Reddit users in r/PlayStation mock Xbox users more frequently than they discuss PlayStation games?"
   - **Theory**: _____
   - **Reasoning**: _____
2. **RQ**: "Do viewers who watch the same streamer daily feel more emotionally connected than viewers who watch weekly?"
   - **Theory**: _____
   - **Reasoning**: _____
3. **RQ**: "Which gratifications (social connection, entertainment, information-seeking) do TikTok users prioritize, and do unmet gratifications predict platform switching?"
   - **Theory**: _____
   - **Reasoning**: _____
4. **RQ**: "Does mainstream media coverage of esports tournaments increase public interest in competitive gaming?"
   - **Theory**: _____
   - **Reasoning**: _____

Exercise 5.3: From Theory to Hypothesis

Choose a media phenomenon you're curious about. Follow these steps:

Step 1: Write a research question (1 sentence) - **RQ**: _____

Step 2: Select a theory (from this chapter or your prior knowledge) - **Theory**: _____

Step 3: Identify 2-3 concepts from that theory - **Concept 1**: _____ - **Concept 2**: _____ - **Concept 3** (optional): _____

Step 4: Define variables - **IV**: _____ - **DV**: _____

Step 5: Write a testable hypothesis - **H1**: _____

Exercise 5.4: Mediator vs. Moderator

Distinguish between mediation and moderation:

Scenario A: Webcam use increases donations, but this effect is *explained by* viewers' sense of intimacy. Intimacy is the mechanism. - This is a **mediator / moderator** (circle one)

Scenario B: Webcam use increases donations for small streamers but not large streamers. Streamer size changes the strength of the effect. - This is a **mediator / moderator** (circle one)

Now create your own example: - **Mediator example**: _____ - **Moderator example**: _____

Reflection Questions

1. **Theory as Lens**: Think of a media phenomenon you've observed (e.g., "TikTok trends spread faster than YouTube trends"). What theory might explain this? How does choosing a theory focus your attention on specific variables while ignoring others?
2. **Theoretical Humility**: Have you ever tried to force a theory to fit your research question? Reflect on why theories sometimes don't fit. What's the difference between *applying* a theory and *testing* a theory?
3. **Grand vs. Middle-Range**: Why are middle-range theories (PSI, Framing) more useful for student research than grand theories (Marxism, Psychoanalysis)? When might grand theories be appropriate?

Chapter Summary

This chapter framed **theory as a lens** that focuses attention and generates predictions:

- **Grand theories** (Marxism, Psychoanalysis) explain broad human behavior but are hard to test
- **Middle-range theories** (PSI, Social Identity, Agenda Setting, Framing, Uses & Gratifications) are specific enough to test but broad enough to generalize
- **Parasocial Interaction** explains one-sided emotional relationships between audiences and media figures
- **Social Identity Theory** predicts that group membership shapes identity, leading to in-group favoritism and out-group derogation
- **Agenda Setting** examines how media coverage shapes public priorities
- **Framing Theory** studies how presenting information differently influences interpretation
- **Uses & Gratifications** asks what audiences *do* with media to satisfy needs
- **Variables** are the building blocks of hypotheses: **IVs** (causes), **DVs** (effects), **mediators** (explain relationships), and **moderators** (change relationships)
- Good theories generate **testable, falsifiable hypotheses**

Key Terms

- **Agenda Setting**: Media influence on what audiences think *about* (topic salience)
- **Concept**: An abstract idea within a theory (e.g., "intimacy," "identity")
- **Dependent Variable (DV)**: The outcome or effect being measured
- **Framing Theory**: How information presentation influences interpretation
- **Grand Theory**: Broad explanatory framework (hard to test empirically)
- **Hypothesis**: A testable prediction derived from theory
- **Independent Variable (IV)**: The cause or predictor being manipulated or measured
- **Mediator**: A variable that explains *how* or *why* an IV affects a DV
- **Middle-Range Theory**: Theory specific enough to test but broad enough to generalize
- **Moderator**: A variable that changes the *strength* or *direction* of an IV-DV relationship
- **Parasocial Interaction (PSI)**: One-sided emotional relationship between audience and media figure
- **Social Identity Theory**: Group membership shapes self-concept and intergroup behavior
- **Uses & Gratifications**: Audiences actively select media to satisfy specific needs
- **Variable**: A measurable characteristic that varies across observations

Looking Ahead

Chapter 6 synthesizes Chapters 1-5 into a **Project Prospectus**—a one-page roadmap that aligns your topic, theory, and method into a feasible research plan. You'll learn to manage scope, avoid common pitfalls, and write a prospectus that defends your research choices.

The Project Brief

Learning Objectives

- Synthesize previous chapters (story, tools, rules, data, theory) into a coherent project plan.
- Manage scope and avoid "scope creep" by narrowing focus systematically.
- Align topic, theory, and data source into a viable research design.
- Write a one-page research prospectus that defends your project's feasibility.

Overview

This is the synthesis chapter. You've gathered the pieces: curiosity (Ch 1), tools (Ch 2), discipline (Ch 3), data sources (Ch 4), and theoretical lens (Ch 5). Now you assemble them into a one-page roadmap—your **project prospectus**. This document answers three questions: What are you studying? Why does it matter? How will you do it?

The Scope Creep Villain

The greatest threat to student research is the "Everything Study":

- "I want to study the effect of the internet on society."
- "I'm interested in how social media influences politics."
- "I'm going to analyze all gaming platforms."

These are not research projects—they're entire academic careers. **Scope creep** happens when:

- You try to answer too many questions at once.
- You choose multiple theories that pull in different directions.
- You attempt to analyze datasets too large or complex for a semester timeline.

Solution: The Narrow Path. Drill down relentlessly until your project fits on one page.

Example progression:

1. Internet → Social Media
2. Social Media → TikTok
3. TikTok → Beauty Influencers
4. Beauty Influencers → Use of Filters
5. Use of Filters → Impact on Viewer Self-Esteem Among Gen Z Women (2024-2025)

By the fifth iteration, you have a manageable, focused study that can be completed in 14 weeks.

The Holy Trinity of Research Design

A viable project requires alignment across three elements:

Topic (What)

What phenomenon are you studying?

Bad example: "Social media"
Good example: "Toxicity in Twitch live streams during competitive gaming"

Theory (Why)

What explanatory framework guides your study?

Bad example: "I'm using lots of theories"
Good example: "Social Identity Theory predicts that competitive contexts increase in-group/out-group hostility, which manifests as toxicity in live chat."

Data Source (How)

What evidence will you analyze?

Bad example: "I'll look at some streams"
Good example: "I will collect chat logs from 50 Twitch streams (25 competitive ranked games, 25 casual games) and code toxic vs. neutral messages."

The Trinity must align. If your topic is Twitch toxicity, your theory is Social Identity, but your data source is YouTube comments, the project fails. Theory must predict patterns visible in your data.

The One-Page Prospectus

A research prospectus is a contract with yourself. It forces you to articulate:

- **Research Question**: What do you want to know?
- **Theoretical Framework**: What theory explains your phenomenon?
- **Data & Method**: What evidence will you analyze, and how?
- **Significance**: Why should anyone care?

Template: The Prospectus Structure

Title: [Concise, descriptive title]

Research Question: What is the relationship between [Variable X] and [Variable Y]?

Background & Significance: [2-3 sentences establishing why this question matters. Cite 2-3 sources.]

Theoretical Framework: [2-3 sentences summarizing the theory you're applying. Cite foundational study.]

Hypotheses/Research Questions: [State 1-3 testable predictions.]

Method: [Describe data source, sample size, and analytic approach in 3-4 sentences.]

Feasibility: [Explain why this project is doable in 14 weeks. Address potential obstacles.]

Example Prospectus

Title: Webcam Use and Viewer Donations on Twitch: A Parasocial Interaction Perspective

Research Question: Do Twitch streamers who use webcams receive higher average donations per stream than streamers without webcams?

Background & Significance: Live-streaming platforms like Twitch have created new economic models where audience donations sustain creators. Understanding what drives donations can inform both streamer strategies and platform design. Existing research on parasocial interaction (PSI) suggests that visual intimacy strengthens perceived relationships between audiences and media figures (Horton & Wohl, 1956; Dibble et al., 2016), but few studies examine PSI in live-streaming contexts.

Theoretical Framework: Parasocial Interaction Theory predicts that audiences form one-sided emotional bonds with media figures. Webcam use increases perceived intimacy by allowing viewers to see streamers' facial expressions and reactions, thus strengthening PSI intensity. Higher PSI should predict greater pseudo-reciprocity behaviors, including donations.

Hypotheses:

- H1: Streamers who use webcams will receive higher average donations per stream than streamers without webcams.
- H2: Viewer-reported PSI intensity will mediate the relationship between webcam use and donation behavior.

Method: I will observe 50 Twitch streams (25 with webcams, 25 without) across three game genres (FPS, RPG, strategy). For each stream, I will record: (1) webcam presence (yes/no), (2) total donations received during a 2-hour session, (3) viewer count, and (4) chat activity (messages per minute). I will also survey 200 viewers across these streams using the PSI-Process Scale (Schramm & Hartmann, 2008) to measure PSI intensity. Data will be analyzed using independent samples t-tests (H1) and mediation analysis (H2).

Feasibility: Twitch streams are publicly accessible, requiring no IRB approval for observational data. The PSI survey will require IRB exemption (anonymous, minimal risk). Data collection (50 streams $\times$ 2 hours = 100 hours of observation) can be distributed across 4 weeks. Surveys can be administered via Qualtrics and linked in Twitch chats with moderator permission.

This prospectus is clear, focused, and feasible. It passes the "explain your project in 60 seconds" test.

Common Pitfalls and How to Avoid Them

Pitfall 1: The Unfalsifiable Question

Bad RQ: "What is the essence of gaming culture?"
Why bad: "Essence" is philosophical, not empirical. No data could answer this.
Fix: "How do gamers construct identity through in-game avatars? (Content analysis of 100 player profiles)"

Pitfall 2: The Impossible Sample

Bad plan: "I will survey 10,000 Twitch viewers."
Why bad: Recruiting 10,000 participants requires months and a budget.
Fix: "I will survey 200 viewers recruited via Twitch chat over 4 weeks."

Pitfall 3: The Vague Method

Bad method: "I will analyze tweets about gaming."
Why bad: Which tweets? How many? What analysis?
Fix: "I will collect 1,000 tweets containing #ToxicGaming from January 2024 and code them for sentiment (positive, neutral, negative) and target (player, game, platform)."

Pitfall 4: The Theory-Free Study

Bad approach: "I'm just going to see what the data shows."
Why bad: Without theory, you can't interpret patterns. Correlation isn't explanation.
Fix: "Social Identity Theory predicts that competitive contexts increase out-group hostility. I will test whether toxic language is more frequent in ranked vs. casual games."

Defending Your Choices

A prospectus is an argument. You must defend:

Why This Topic?

Don't say "I like gaming." Say "Toxicity in online gaming has been linked to player attrition, mental health harms, and exclusion of marginalized groups (Consalvo, 2012). Understanding its causes can inform platform moderation policies."

Why This Theory?

Don't say "I read about this theory." Say "Social Identity Theory has been validated across dozens of studies on intergroup conflict (Tajfel & Turner, 1979). Applying it to live-streaming contexts extends the theory to a novel domain and addresses calls for research on digital tribalism (Kowert, 2020)."

Why This Method?

Don't say "Content analysis seemed easy." Say "Content analysis allows systematic measurement of observable behaviors (toxic language) at scale. Unlike surveys, which rely on self-reports, content analysis captures actual chat behavior, reducing social desirability bias."

The Feasibility Reality Check

Before committing to your project, ask:

1. **Can I access the data?** (Public vs. proprietary, API access, IRB approval)
2. **Can I analyze the data?** (Do I have the coding skills, software, or help?)
3. **Can I finish in 14 weeks?** (Data collection: 4 weeks, analysis: 4 weeks, writing: 4 weeks, buffer: 2 weeks)

If you answer "no" to any question, revise your plan.

From Prospectus to Execution

Once approved, your prospectus becomes your north star. When you're tempted to add variables, expand the sample, or pursue tangential findings—**refer back to the prospectus**. Scope discipline is what separates finished projects from abandoned drafts.

Practice: Crafting Your Prospectus

Exercise 6.1: Diagnosing Scope Creep

Evaluate these research questions for scope problems:

1. "How does social media affect democracy?"
   - **Problem**: _____
   - **Fix**: _____
2. "Why do people watch Twitch?"
   - **Problem**: _____
   - **Fix**: _____
3. "Do female streamers who play competitive FPS games in the 'Just Chatting' category on Twitch experience different types of harassment than male streamers?"
   - **Problem**: _____
   - **Fix**: _____

Exercise 6.2: The Holy Trinity Alignment Check

For each scenario, identify which element of the Trinity (Topic / Theory / Data) is misaligned:

Scenario A: - **Topic**: Instagram influencer authenticity - **Theory**: Agenda Setting Theory - **Data**: Instagram posts and captions - **Misalignment**: _____ - **Fix**: _____

Scenario B: - **Topic**: Toxicity in competitive gaming - **Theory**: Social Identity Theory - **Data**: Survey asking gamers to self-report their toxic behavior - **Misalignment**: _____ - **Fix**: _____

Exercise 6.3: Write Your Own Prospectus (Draft 1)

Use the template from section 6.5.2 to draft your prospectus. Include:

Title: _____

Research Question: _____

Background & Significance (2-3 sentences with 2-3 citations): _____

Theoretical Framework (2-3 sentences with foundational citation): _____

Hypotheses/Research Questions (1-3 predictions): _____

Method (data source, sample, analysis): _____

Feasibility (timeline, IRB considerations, obstacles): _____

Exercise 6.4: Peer Review Practice

Exchange prospectuses with a classmate (or review your own using these criteria):

Clarity Checklist: - [ ] Can you explain the project in 60 seconds? - [ ] Are the variables clearly defined? - [ ] Is the theory appropriately matched to the research question?

Feasibility Checklist: - [ ] Is the sample size realistic for 14 weeks? - [ ] Is the data accessible (public, or IRB strategy outlined)? - [ ] Are the analysis methods appropriate for the data type?

Scope Checklist: - [ ] Is the focus narrow enough to complete in one semester? - [ ] Does the research question avoid multiple unrelated questions? - [ ] Is the theory application focused (not "using all the theories")?

Feedback for revision: _____

Reflection Questions

1. **Your Scope Discipline**: What research question are you most excited about? Now narrow it down three times. How does narrowing improve feasibility without killing the interesting part?
2. **The Feasibility-Ambition Trade-off**: Reflect on the tension between "doing something meaningful" and "finishing on time." How do you balance intellectual ambition with practical constraints?
3. **Theory Justification**: Why is it not enough to say "I'm using Social Identity Theory because it seems relevant"? What counts as a strong theoretical justification?

Chapter Summary

This chapter taught you to synthesize previous chapters into a **focused, defensible research plan**:

- **Scope creep** kills student projects—narrow relentlessly until your project fits on one page
- The **Holy Trinity** (Topic, Theory, Data) must align: theory must predict patterns visible in your data
- A strong **prospectus** includes: research question, theoretical framework, hypotheses, method, and feasibility justification
- Common pitfalls include unfalsifiable questions, impossible samples, vague methods, and theory-free studies
- **Defending your choices** requires citing literature, explaining theoretical relevance, and justifying methodological decisions
- The **feasibility reality check**: Can you access the data? Analyze it? Finish in 14 weeks?
- Your prospectus is a **contract with yourself**—refer back to it when tempted by scope creep

Key Terms

- **Feasibility**: Whether a research project can realistically be completed given time, resources, and skills
- **Holy Trinity**: The alignment of Topic (what), Theory (why), and Data (how)
- **Prospectus**: A one-page research plan outlining question, theory, method, and feasibility
- **Scope**: The boundaries of a research project (population, timeframe, variables)
- **Scope creep**: The tendency for projects to expand beyond manageable limits

- **Scope discipline**: The practice of narrowing focus to ensure project completion
- **Theoretical justification**: Evidence-based reasoning for why a theory is appropriate for a research question

Looking Ahead

Chapter 7 introduces **Structured Listening**—the ethnographic practice of immersing yourself in digital communities before coding. You'll learn to ground your variables in the lived reality of the spaces you study, translating community norms into formal research questions.

Structured Listening

Learning Objectives

- Conduct structured, ethnographic-style observation in digital communities.
- Translate qualitative observations into formal Research Questions.
- Distinguish between descriptive and relational RQs.
- Recognize the epistemological value of "lurking" before coding.

Overview

Before you can measure something, you must **listen**. This chapter introduces **Structured Listening**—a systematic approach to qualitative observation that ensures your variables are grounded in the lived reality of the communities you study. The best content analysts don't impose external categories onto data; they derive categories from what communities actually say and do.

The Methodology of Listening

Structured Listening Defined

Structured Listening is "listening with a rubric." It's not passive consumption; it's active, framework-driven observation. You're not just browsing Reddit—you're documenting patterns, noting recurring themes, and testing your assumptions against what the community actually does.

Key principles:

1. **Suspension of judgment**: Resist labeling behavior as "good" or "bad" until you understand its function within the community.
2. **Pattern recognition**: Look for regularities (repeated phrases, rituals, norms).
3. **Documentation**: Keep field notes. What surprised you? What confused you? What terms does the community use that outsiders wouldn't understand?

Terminology note: This chapter uses terms from **ethnography** (the study of cultures). If terms like "non-participant observation" or "field notes" are unfamiliar, think of them this way: - **Ethnography** = studying how groups of people behave and interact - **Non-participant observation** = watching without joining in (like observing a Twitch chat without chatting yourself) - **Field notes** = your written observations, like a diary of what you noticed - **Immersion** = spending enough time in a community to understand insider perspectives

Trust and Context

To understand a community's norms, slang, and values, you must **immerse yourself**:

- Join Discord servers or subreddits related to your topic.
- Read old threads to understand inside jokes, historical conflicts, and community lore.
- Note who has status (moderators, long-time members, content creators).
- Observe without participating initially—this is called **"lurking"** or **non-participant observation** (watching from the sidelines before jumping in).

You cannot code for "toxicity" if you don't know the difference between playful banter and genuine harassment in that specific community. What looks like an insult to an outsider might be a term of endearment among friends.

Example: In fighting game communities, calling someone a "scrub" can mean:

- **Contextually hostile**: "You're trash, uninstall the game."
- **Playful ribbing**: "You got bodied, scrub!" (said after a close match among friends)
- **Self-deprecating**: "I'm such a scrub, I can't execute combos."

Without structured listening, you'll misclassify these messages.

The Translator Role

As a researcher, you act as a **Communication Liaison**—translating messy, organic community behavior into formal, testable propositions.

Example transformation:

- **Community Reality**: "Chat spams 'F' when the streamer dies."
- **Translation**: "Viewers use ritualized text (emotes, copypasta) to express collective emotional response."
- **Research Question**: "Do streamers who acknowledge chat rituals (e.g., respond to 'F' spam) receive higher engagement (measured by chat messages per minute)?"

Another example:

- **Community Reality**: "Valorant players flame teammates who don't use voice chat."
- **Translation**: "Communication norms in competitive gaming contexts privilege voice over text, creating exclusion for players who can't or won't use microphones (e.g., non-native speakers, women avoiding harassment)."
- **Research Question**: "Are players who don't use voice chat more likely to receive toxic messages in ranked matches compared to casual matches?"

Ethnographic Observation in Digital Spaces

Traditional ethnography involves physically entering a community and observing behavior. Digital ethnography adapts these methods for online contexts.

Where to Observe

- **Twitch/YouTube chats**: Real-time interaction during live streams.
- **Discord servers**: Asynchronous and synchronous conversation, voice channels, role hierarchies.
- **Reddit**: Threaded discussions, upvote/downvote dynamics, subreddit norms.
- **Twitter/X**: Public discourse, hashtag campaigns, quote-tweet culture.

What to Document

- **Language**: Slang, acronyms (GG, GG EZ, KEKW), in-jokes.
- **Norms**: Who gets respect? Who gets mocked? What behaviors are rewarded or punished?
- **Rituals**: Repeated behaviors (raid trains on Twitch, AMA threads on Reddit).
- **Power dynamics**: Who moderates? Who sets the tone? How is dissent handled?

Field Notes Template

Keep a structured log:

Date	Platform	Community	Observation	Potential Variable
2024-02-05	Twitch	Valorant_streamer_A	Streamer thanks every subscriber by name	"Acknowledgment frequency"
2024-02-05	Reddit	r/Valorant	Posts criticizing Riot Games get heavily upvoted	"Developer criticism = social capital"

Date	Platform	Community	Observation	Potential Variable
2024-02-06	Discord	CompetitiveFPS	Voice chat users dominate strategy discussions	"Communication modality hierarchy"

Formulating Research Questions (RQs)

Once you've listened, you can ask answerable questions. Research questions come in two types: **descriptive** and **relational**.

Descriptive RQs

Descriptive questions ask **"What?"** or **"How much?"** They map the landscape.

Examples:

- RQ1: "What proportion of Twitch chat messages during Valorant streams contain toxic language?"
- RQ2: "How frequently do news articles mention 'esports' in sports sections vs. entertainment sections?"
- RQ3: "What are the most common emotes used in high-engagement Twitch streams?"

Descriptive RQs are foundational. You can't explain patterns until you've documented their existence.

Relational RQs

Relational questions ask about **relationships between variables**: "Does X predict Y?" or "Is there a difference between Group A and Group B?"

Examples:

- RQ4: "Is there a relationship between streamer webcam use and average donation amounts?"
- RQ5: "Do competitive game streams (ranked matches) have higher rates of toxic chat compared to casual game streams?"
- RQ6: "Are female streamers more likely to experience gendered harassment than male streamers?"

Relational RQs test theories. They move from description ("What is happening?") to explanation ("Why is it happening?").

From Observations to Hypotheses

Research questions are exploratory. **Hypotheses** are predictions grounded in theory.

Observation: "I noticed that streamers who respond to chat frequently seem to get more donations."

Theory: Parasocial Interaction Theory predicts that interaction increases perceived intimacy, which strengthens emotional bonds.

Hypothesis: "Streamers who respond to at least 50% of chat messages will receive higher average donations per stream than streamers who respond to fewer than 25% of messages."

The hypothesis is **testable** (you can measure response rate and donations), **falsifiable** (you could find no difference or the opposite pattern), and **theoretically grounded** (it follows from PSI theory).

The Danger of Premature Coding

Many novice researchers skip structured listening and jump straight to coding. This leads to:

- **Misclassification**: You label behavior without understanding its meaning.
- **Missing categories**: Your codebook doesn't capture important distinctions.
- **Researcher bias**: You impose your assumptions onto the data.

Best practice: Spend 1-2 weeks listening before finalizing your codebook. Read 100 tweets, watch 10 streams, lurk in 3 Discord servers. Then, and only then, build your categories.

Case Study: Observing Twitch Raid Culture

Goal: Understand how Twitch "raids" (when one streamer sends their viewers to another channel) function as community-building rituals.

Structured Listening Process:

1. **Week 1**: Watch 20 raids across different streamers. Note what happens:
    - Raiding streamer announces: "We're raiding [name]! Let's show love!"
    - Chat floods new channel with emotes and greetings.
    - Receiving streamer thanks raiders and shouts out the raiding streamer.
2. **Week 2**: Look for variation:
    - Some raids are reciprocal (streamers raid each other regularly).
    - Some raids target small streamers (charity raids).
    - Some raids are hostile (raid with negative spam—rare but exists).

3. **Formulate RQs**:
    - RQ1: "What proportion of raids are reciprocal vs. one-directional?"
    - RQ2: "Do charity raids (large streamer to small streamer) increase the small streamer's follower count in the 24 hours post-raid?"
    - RQ3: "How do streamers linguistically frame raids (gift, obligation, business transaction)?"
4. **Build codebook**:
    - **Raid type**: Reciprocal, Charity, Random, Hostile
    - **Receiving streamer response**: Thanks + shoutout, Thanks only, No acknowledgment
    - **Chat sentiment**: Positive, Neutral, Negative

This systematic process ensures that your research questions and coding categories reflect the community's reality, not your assumptions.

Real Data: Bob's Burgers Episodes

To practice structured listening with actual television data, we'll examine **Bob's Burgers Episodes**, available via TidyTuesday (November 12, 2024).

Why Bob's Burgers? - 13+ seasons of consistent episode data (structured comparison across time) - Fandom is active and analyzable (fan theories, episode ratings) - Perfect for structured listening: episode structure is formulaic (you can notice deviations) - Demonstrates how media researchers analyze narrative patterns

Accessing the Data

Download from TidyTuesday: 1. Visit: https://github.com/rfordatascience/tidytuesday/blob/main/data/2024/2024-11-12/readme.md 2. Download the Bob's Burgers CSV (contains episode titles, ratings, air dates, season, episode number) 3. You'll use this for structured observation in Chapter 8

Structured Listening with Bob's Burgers Data

Research Question: How does the show's narrative structure evolve across seasons? Do episodes become more complex? Do character relationships deepen?

Listening Process (what you'd do):

1. **Week 1**: Watch 3-4 episodes from Season 1. Take field notes:

- What is the main problem/conflict in each episode?
- How is it resolved?
- What character growth (if any) happens?
- What community/family dynamics are established?

2. **Week 2**: Watch 3-4 episodes from Season 10. Compare:
   - Are problems more complex or simpler?
   - Do resolutions feel more satisfying?
   - Is character development deeper?
   - Have family relationships changed fundamentally?
3. **Pattern Recognition**:
   - Early seasons: "Self-contained problem solved by episode's end"
   - Later seasons: "Ongoing character arcs, relationships evolve"
4. **Formulate RQs**:
   - RQ1: "Do early Bob's Burgers episodes (Seasons 1-3) rely more on external conflict (customer complaints) vs. later episodes (Seasons 10-13) which focus on family conflict?"
   - RQ2: "How does Tina's character change across the series (meta-narrative through episode ratings)?"
   - RQ3: "Do fan ratings correlate with episodes featuring expanded character backstories?"

Building Your Field Notes (Obsidian Format)

As you watch, document in Obsidian using this template:

```
# Bob's Burgers: Season X, Episode Y - "[Title]"

## Episode Details
- **Season**: X
- **Episode**: Y
- **Air Date**: [date]
- **IMDb Rating**: [if available]

## Main Conflict
[Describe the central problem/conflict]

## Resolution Method
- Family problem-solving
- Customer crisis
```

```
- External event
- Character growth arc

## Character Focus
- Tina: [notes]
- Gene: [notes]
- Louise: [notes]

## Family Dynamics
[How do relationships show up in this episode?]

## Growth/Change
Does anyone grow? Learn something? Shift their perspective?

## Differences from Season 1
[If you've watched early episodes, how is this different?]
```

This practice teaches you how structured listening works: you're not randomly watching; you're systematically documenting patterns so you can eventually measure them.

Practice: Developing Structured Listening Skills

Exercise 7.1: Field Notes Practice

Choose a digital community you're interested in studying (Twitch chat, Reddit subreddit, Discord server, TikTok comments). Observe for 30-60 minutes and document:

Community: _____ **Platform**: _____ **Date/Time**: _____

Observations:

Observation	Community Term/Slang	Potential Variable
Ex: Users spam "F" when streamer dies	"Press F to pay respects" meme	Ritualized emotional expression
1.		
2.		
3.		
4.		
5.		

Norms observed: _____ **Who has status/power**: _____
Surprising behavior: _____

Exercise 7.2: Translating Observations to RQs

For each community observation, write both a **descriptive RQ** and a **relational RQ**:

Observation 1: "Gaming streamers often thank subscribers by name, but some ignore them."

- **Descriptive RQ**: _____
- **Relational RQ**: _____

Observation 2: "In competitive FPS streams, chat uses more aggressive language than in casual streams."

- **Descriptive RQ**: _____
- **Relational RQ**: _____

Your own observation:

- **Observation**: _____
- **Descriptive RQ**: _____
- **Relational RQ**: _____

Exercise 7.3: Distinguishing Context-Dependent Meaning

For each message, explain how an outsider might misinterpret it and what structured listening reveals:

1. **Message**: "You're such a bot lol"
   - **Outsider interpretation**: _____
   - **Community meaning**: _____
   - **How you know**: _____
2. **Message**: "Skill issue"
   - **Outsider interpretation**: _____
   - **Community meaning**: _____
   - **Context clues**: _____
3. **Find your own example** from observation:
   - **Message**: _____
   - **Outsider interpretation**: _____
   - **Community meaning**: _____

Exercise 7.4: From Listening to Theory

After observing your chosen community, connect your observations to potential theories:

Pattern observed: _____

Potential theories that might explain this (refer to Chapter 5): 1. _____ 2. _____

Which theory best fits? Why? _____

What hypothesis would you test? _____

Reflection Questions

1. **The Lurker's Advantage**: Why is it valuable to observe without participating initially? What might you miss if you immediately start interacting? How does "lurking" relate to traditional ethnography's "participant observation"?
2. **Your Assumptions Challenged**: Think of a digital community you're familiar with. What assumptions did you hold before closely observing it? How might those assumptions have led to coding errors?
3. **The Ethics of Listening**: You're observing public Twitch chat without announcing yourself as a researcher. Is this ethical? What's the difference between this and surveying users without consent?

Chapter Summary

This chapter introduced **structured listening** as the foundation for valid content analysis:

- **Structured listening** is systematic, documented observation with a framework—not casual browsing
- **Suspend judgment** to understand behavior within community context before labeling it
- **Immersion** in digital spaces (lurking) reveals norms, slang, status hierarchies, and rituals
- Researchers act as **communication liaisons**, translating organic community behavior into formal research questions
- **Descriptive RQs** ask "what" or "how much"; **relational RQs** ask about relationships between variables
- **Premature coding** (skipping listening phase) leads to misclassification, missing categories, and researcher bias

- Field notes should document language, norms, rituals, power dynamics, and surprising observations
- Hypotheses emerge from theory applied to observed patterns, not from assumptions alone

Key Terms

- **Descriptive research question**: A question asking "what" or "how much" about a phenomenon
- **Field notes**: Documented observations from ethnographic research
- **Immersion**: Extended engagement with a community to understand insider perspectives
- **Lurking**: Observing online communities without active participation (non-participant observation)
- **Non-participant observation**: Research method where the researcher observes without interacting
- **Relational research question**: A question about relationships between variables
- **Ritualized behavior**: Repeated, patterned actions with symbolic meaning (e.g., "F in chat")
- **Structured listening**: Systematic, documented observation using a framework
- **Suspension of judgment**: Temporarily withholding evaluation to understand behavior in context

Looking Ahead

Chapter 8 tackles the hardest cognitive leap: **operationalization**—translating abstract "vibes" (wholesome, toxic, authentic) into concrete, measurable variables. You'll learn to write conceptual and operational definitions, understand levels of measurement (NOIR), and navigate the reliability-validity trade-off.

From Vibes to Variables

Learning Objectives

- Bridge the gap between abstract concepts ("vibes") and measurable indicators ("variables").
- Write both conceptual and operational definitions for key variables.
- Understand levels of measurement: Nominal, Ordinal, Interval, Ratio (NOIR).
- Recognize the trade-offs between rich description and quantitative precision.

Overview

This chapter tackles the hardest cognitive leap in research methods: **operationalization**. You naturally perceive the world in "vibes"—intuitive, holistic impressions. Science requires precision. How do you convert the vibe of a "wholesome streamer" into a number? The answer lies in operationalization: the disciplined translation of abstract concepts into concrete, measurable indicators.

The Operationalization Gap

Vibes vs. Variables

- **Vibe**: "That streamer is wholesome."
- **Variable**: "Wholesome Behavior Index" = count of positive affirmations ("You got this!"), thank-yous to chat, and words of encouragement per stream hour.

The vibe is subjective and unmeasurable. The variable is explicit, countable, and reproducible.

The Phrase: "From Vibes to Variables"

Inspired by UX research podcasts, the phrase "vibes to variables" captures the translation process.

Example from product design:

- **Vibe**: "Users feel the app is slow."
- **Variable**: "Time to First Interaction" (measured in milliseconds from app launch to first user tap).

In mass communication:

- **Vibe**: "This news article is biased."
- **Variable**: "Source balance" = proportion of quotes from Democratic vs. Republican sources.

The variable doesn't capture the *fullness* of the vibe, but it captures something **measurable and testable**.

Conceptual vs. Operational Definitions

Every variable requires two definitions: conceptual (what does it mean in theory?) and operational (how do we measure it in practice?).

Conceptual Definition (Abstract)

The dictionary or theoretical definition. What does the concept mean in everyday language or scholarly discourse?

Example 1: **Toxicity** is hostile or harmful communication intended to demean, threaten, or exclude others.

Example 2: **Parasocial interaction** is a one-sided emotional relationship where a viewer feels connected to a media figure despite no reciprocal relationship.

Operational Definition (Concrete)

The instruction manual for measurement. How will you recognize and count this concept in your data?

Example 1 (Toxicity):

"Toxic messages are coded as any chat message containing:

1. Slurs (racial, homophobic, ableist—see Appendix A for full list)
2. Personal insults directed at individuals (e.g., 'you suck,' 'uninstall,' 'trash player')
3. Threats (e.g., 'I hope you get banned,' 'kill yourself')

Exclusions: Profanity directed at in-game events (e.g., 'that lag was f***ing awful') is NOT coded as toxic unless paired with personal attacks."

Example 2 (Parasocial Interaction):

"PSI intensity is measured using the PSI-Process Scale (Schramm & Hartmann, 2008), a 9-item Likert scale (1 = strongly disagree, 7 = strongly agree). Example items:

- 'I feel like I know the streamer personally.'
- 'If the streamer had problems, I would want to help them.'

PSI score = mean of all 9 items (range: 1-7, higher = stronger PSI)."

The Reliability-Validity Trade-off

Reliability: Can your measure be applied consistently? If two coders use your operational definition, do they get the same results?

Validity: Does your measure actually capture the concept you claim to measure?

The tension: Simple measures (e.g., counting swear words) are highly reliable but low validity (profanity toxicity). Complex measures (e.g., coding intent and context) are more valid but harder to apply reliably.

Solution: Pilot test your operational definitions. If inter-coder reliability is low (<.70), your definition is too vague. Refine it.

Levels of Measurement (NOIR)

Not all variables are created equal. The **level of measurement** determines what statistical operations are meaningful.

Nominal: Categories Without Order

Variables are labels or names. No ranking implied.

Examples:

- Game genre (FPS, RPG, Strategy, Sports)
- Streamer gender (Male, Female, Non-binary, Prefer not to say)
- Platform (Twitch, YouTube Gaming, Facebook Gaming)

Allowed operations: Count frequencies, calculate mode (most common category), test for association (Chi-Square).

Not allowed: Calculate means or say one category is "higher" than another.

Ordinal: Categories With Order (But Unequal Intervals)

Variables have a rank order, but the distance between ranks is undefined.

Examples:

- Likert scales: "How much do you agree?" (1 = Strongly Disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly Agree)
- Rank-ordered preferences: "Rank your top 3 games"
- Education level: (High School < Bachelor's < Master's < PhD)

Allowed operations: Calculate median, test for correlation (Spearman's rho).

Caution with means: Technically, you shouldn't calculate the mean of ordinal data (the "distance" between Disagree and Neutral isn't necessarily the same as Neutral to Agree), but in practice, researchers often treat Likert scales as interval data.

Interval: Equal Intervals, No True Zero

The distance between values is consistent, but zero is arbitrary (doesn't mean "absence").

Examples:

- Temperature in Fahrenheit (0°F doesn't mean "no temperature")
- IQ scores (100 is average, but 0 doesn't mean "no intelligence")

Allowed operations: Calculate mean, standard deviation, run t-tests and ANOVAs.

Not allowed: Say "100°F is twice as hot as 50°F" (because zero is arbitrary).

Rare in communication research: Most variables are nominal, ordinal, or ratio.

Ratio: Equal Intervals + True Zero

The gold standard. Zero means "none" or "absence."

Examples:

- Number of toxic messages (0 = no toxic messages)
- Stream duration (0 = no stream)
- Donations in dollars (0 = no donations)
- Follower count (0 = no followers)

Allowed operations: Everything—mean, median, ratios ("Streamer A has twice as many followers as Streamer B").

Why it matters: The more you know about your variable's level of measurement, the more you know about what statistical tests are appropriate.

Operationalizing Complex Concepts

Some concepts resist simple operationalization. How do you measure "authenticity," "community," or "identity"?

Strategy 1: Break it down into measurable components

Concept: Streamer authenticity

Components:

1. Self-disclosure (number of personal stories shared per stream)
2. Emotional expressiveness (coded facial expressions: genuine smiles, eye contact)
3. Consistency (alignment between stream persona and social media persona)

You can't measure "authenticity" directly, but you can measure behaviors associated with perceived authenticity.

Strategy 2: Use validated scales

If someone has already developed a reliable scale, use it. Don't reinvent the wheel.

Examples:

- **Parasocial Interaction**: PSI-Process Scale (Schramm & Hartmann, 2008)
- **Social Identity**: Social Identity Scale (Cameron, 2004)
- **Media Credibility**: Source Credibility Scale (Ohanian, 1990)

Strategy 3: Multi-method triangulation

Use multiple measures to capture different facets.

Concept: Community cohesion

Measures:

1. Survey: "I feel like I belong in this community" (Likert scale)
2. Behavioral: Frequency of repeat participation (returning to streams/threads)
3. Content analysis: Use of "we" language vs. "I" language in chat

Common Operationalization Mistakes

Mistake 1: Conflating Concepts

Bad: "I'll measure engagement by counting likes."

Problem: Likes measure **passive approval**, not engagement (which includes comments, shares, time spent).

Fix: Define engagement explicitly. If you mean "passive interaction," say so. If you mean "active participation," count comments and replies.

Mistake 2: Proxy Measures Without Justification

Bad: "I'll measure streamer popularity by subscriber count."

Problem: Subscriber count reflects **paying fans**, not total popularity (many viewers don't subscribe).

Fix: Justify your proxy: "While subscriber count doesn't capture all viewers, it represents **committed fans willing to pay**, which aligns with my definition of popularity as economic support."

Mistake 3: Over-simplification

Bad: "Toxicity = swear words."

Problem: This misses context. "This game is f***ing awesome" isn't toxic.

Fix: Code for context: "Toxicity = language directed negatively at individuals."

Mistake 4: Unmeasurable Definitions

Bad: "Wholesome content is content that makes viewers feel good."

Problem: You can't observe feelings directly.

Fix: "Wholesome content includes: positive affirmations, expressions of gratitude, supportive messages, and absence of profanity or negativity."

Case Study: Operationalizing "Streamer Authenticity"

Research Question: Are "authentic" streamers more successful than "performative" streamers?

Problem: "Authenticity" is a vibe, not a variable.

Step 1: Conceptual definition

Authenticity = the perception that a streamer is genuine, unscripted, and self-consistent (not playing a character).

Step 2: What observable behaviors signal authenticity?

- Sharing personal stories unrelated to gameplay
- Emotional expressiveness (laughing, frustration, excitement)
- Responding spontaneously to chat (not reading from a script)
- Consistency across platforms (Twitter persona matches stream persona)

Step 3: Operational definitions

1. **Self-disclosure**: Count of personal stories (defined as anecdotes about streamer's non-gaming life) per stream hour.
2. **Emotional expressiveness**: Coded presence/absence of facial webcam + number of audible emotional reactions (laughter, sighs, exclamations) per stream hour.
3. **Chat responsiveness**: Percentage of chat questions acknowledged within 2 minutes.
4. **Cross-platform consistency**: Analyze 20 tweets and 20 stream clips. Code tone as "casual," "professional," or "performative." Consistency = proportion of matching codes.

Step 4: Combine into Authenticity Index

Standardize each measure (z-scores), average them, and create an Authenticity Index (range: -3 to +3, higher = more authentic).

Step 5: Test

Correlate Authenticity Index with "success" measures (average concurrent viewers, subscriber count, donation revenue).

Practice: Mastering Operationalization

Exercise 8.1: Writing Operational Definitions

For each concept, write both a **conceptual definition** and an **operational definition**:

1. **Concept**: Streamer engagement with chat

- **Conceptual definition**: _____
- **Operational definition**: _____

2. **Concept**: News article bias

   - **Conceptual definition**: _____
   - **Operational definition**: _____

3. **Concept**: TikTok video virality

   - **Conceptual definition**: _____
   - **Operational definition**: _____

Exercise 8.2: Identifying Levels of Measurement (NOIR)

Classify each variable by its level of measurement:

1. Streamer's primary game genre (FPS, RPG, Sports, Strategy)

   - **Level**: Nominal / Ordinal / Interval / Ratio (circle one)
   - **Why**: _____

2. Viewer satisfaction rating (1 = Very Unsatisfied … 5 = Very Satisfied)

   - **Level**: Nominal / Ordinal / Interval / Ratio
   - **Why**: _____

3. Number of toxic messages per stream

   - **Level**: Nominal / Ordinal / Interval / Ratio
   - **Why**: _____

4. Streamer's rank (Bronze, Silver, Gold, Platinum, Diamond)

   - **Level**: Nominal / Ordinal / Interval / Ratio
   - **Why**: _____

5. Temperature of community sentiment (measured in arbitrary units where higher = warmer)

   - **Level**: Nominal / Ordinal / Interval / Ratio
   - **Why**: _____

Exercise 8.3: The Reliability-Validity Trade-off

For each scenario, identify whether the issue is **reliability** or **validity**, and suggest improvements:

Scenario A: Two coders disagree on 40% of cases about whether language is "sarcastic" because the operational definition says "code as sarcastic if it seems ironic."

- **Issue**: Reliability / Validity (circle one)
- **Problem**: _____
- **Fix**: _____

Scenario B: Your codebook defines "engagement" as "number of likes received." But likes might reflect algorithmic promotion, not genuine viewer interest.

- **Issue**: Reliability / Validity
- **Problem**: _____
- **Fix**: _____

Scenario C: You measure "toxicity" by counting swear words, which is easy to code consistently, but misses non-profane insults like "uninstall."

- **Issue**: Reliability / Validity
- **Problem**: _____
- **Fix**: _____

Exercise 8.4: From Vibes to Variables (Your Turn)

Choose a "vibe" you've observed in digital media and operationalize it:

Vibe/Concept: _____

Conceptual definition: _____

Observable indicators (3-5 behaviors or features that signal this vibe): 1. _____ 2. _____ 3. _____

Operational definition (step-by-step coding instructions): _____

Level of measurement: _____

Reliability concern: _____ **How to address**: _____

Real Data: American Idol Judge Feedback

Let's apply operationalization to a real dataset: American Idol episode data with judge feedback and contestant outcomes.

Research Question: Do judges provide more positive feedback to contestants from certain demographic backgrounds? Does positive feedback predict contestant advancement?

Step 1: Conceptual Definitions

- **Judge Feedback Tone**: The emotional valence (positive vs. negative) of judges' spoken comments to contestants
- **Contestant Demographics**: Visible characteristics including gender identity and apparent race/ethnicity
- **Advancement**: Whether a contestant progresses to the next round of competition

Step 2: Observable Indicators

Watch a single American Idol episode and notice: - Judges make comments like "You have incredible range" (positive) vs. "That wasn't your best" (negative) - Comments reference contestant's appearance, background, or identity - Comments precede judges' decisions to advance or eliminate contestants

Step 3: Operational Definitions

Judge Feedback Tone (Ordinal, coded per judge comment): - **Strongly Positive** (3): Compliments ability, predicts success, or uses superlatives ("incredible," "stunning," "perfect") - **Mildly Positive** (2): Appreciates effort or identifies strengths without superlatives ("nice job," "I liked the second verse") - **Neutral** (1): Provides constructive criticism without judgment ("try a different key," "slower tempo might work better") - **Negative** (0): Criticizes, expresses doubt, or challenges contestant ("that was off-key," "you seemed nervous")

Coding Decision Rules: - One comment = one code - If a comment contains both positive and negative elements, code the dominant impression - Code comments made during group rounds separately from individual performances

Contestant Gender Identity (Nominal): - **Male** - **Female** - **Non-binary/Other** (if identifiable)

Contestant Race/Ethnicity (Nominal): - Code based on visible appearance noted by judges or contestant introduction - Categories: White, Black, Latinx, Asian, Middle Eastern,

Multiracial, Other - **Reliability note**: Race/ethnicity assessment by outside coder is inherently subjective; consider obtaining demographic info directly from official data if available

Advancement (Nominal, binary): - **Yes**: Contestant proceeds to next round - **No**: Contestant is eliminated this round

Step 4: Data Structure

Example coded episode excerpt:

Judge	Contestant_ID	Gender	Race_Ethnicity	Comment_Tone	Advancement
Katy	C042	Female	Latinx	3 (Strongly Positive)	Yes
Luke	C042	Female	Latinx	2 (Mildly Positive)	Yes
Lionel	C042	Female	Latinx	2 (Mildly Positive)	Yes
Katy	C043	Male	White	1 (Neutral)	No
Luke	C043	Male	White	0 (Negative)	No
Lionel	C043	Male	White	1 (Neutral)	No

Step 5: Reliability Considerations

Challenge: Judge tone is subjective. Two coders might disagree: is "You have range, but pitch control needs work" positive or neutral?

Solution: 1. Create a **codebook** with clear decision rules and examples (Chapter 9) 2. Have two independent coders rate 5-10 sample episodes 3. Calculate **Cohen's kappa** to measure agreement (we'll do this in Chapter 10) 4. If kappa < 0.70, revise definitions and try again

Validity Consideration: - Does "feedback tone" actually capture judges' implicit biases, or does it capture contestants' actual singing quality? - This is unmeasurable from comments alone; you'd need audio analysis or comparison to amateur judges' evaluations - Accept this limitation: "This study measures *expressed* judge feedback tone, which may reflect both contestant quality and judge bias"

Accessing the Data

American Idol data is available from TidyTuesday (July 23, 2024):

```
# Download directly from TidyTuesday GitHub (no R code needed yet)
https://github.com/rfordatascience/tidytuesday/tree/master/data/2024/2024-07-
23
```

Files available: - `auditions.csv` - contestant audition data with demographics - `episodes.csv` - episode information and air dates - `judges.csv` - judge names and seasons

In Chapter 9, you'll build a complete codebook for coding judge feedback. In Chapter 11, you'll download this data and learn to import and organize it in R.

Reflection Questions

1. **Measurement Loss**: Every operationalization loses something. When you measure "authenticity" through self-disclosure frequency, what nuances do you miss? Is this loss acceptable? When is it not?
2. **The Quantification Dilemma**: Some scholars argue that quantifying human communication (reducing "wholesome" to a number) strips away meaning. Others argue it's the only way to make generalizable claims. Where do you stand? Can you appreciate both perspectives?
3. **Your Own Variables**: Think about your research question (from Chapter 6). What concepts do you need to operationalize? What will be hardest to measure? Why?

Chapter Summary

This chapter taught the cognitive leap from abstract concepts to concrete measurements:

- **Operationalization** translates "vibes" (intuitive impressions) into "variables" (measurable indicators)
- Every variable needs both a **conceptual definition** (what it means theoretically) and an **operational definition** (how to measure it)
- The **reliability-validity trade-off**: simple measures are reliable but may lack validity; complex measures capture nuance but are harder to apply consistently
- **Levels of measurement** (NOIR): **Nominal** (categories), **Ordinal** (ranked categories), **Interval** (equal intervals, no true zero), **Ratio** (equal intervals + true zero)
- Different measurement levels permit different statistical operations
- Complex concepts require **breaking down into components**, using **validated scales**, or **multi-method triangulation**
- Common mistakes: conflating concepts, unjustified proxies, oversimplification, unmeasurable definitions
- Operational definitions must be **specific, observable, and replicable**

Key Terms

- **Conceptual definition**: Abstract, theoretical meaning of a variable
- **Interval level**: Equal intervals between values, but no true zero (rare in communication research)
- **Level of measurement**: The mathematical properties of a variable (NOIR)
- **Nominal level**: Categories without order (e.g., gender, platform, genre)
- **Operational definition**: Concrete, step-by-step instructions for measuring a variable
- **Operationalization**: The process of translating concepts into measurable variables
- **Ordinal level**: Ranked categories with unequal intervals (e.g., Likert scales, education level)
- **Proxy measure**: An indirect indicator used when direct measurement is impossible
- **Ratio level**: Equal intervals plus true zero (e.g., follower count, message frequency)
- **Reliability**: Consistency of measurement across coders or time
- **Validity**: Whether a measure actually captures the intended concept
- **Variable**: A measurable characteristic that varies across observations

Looking Ahead

Chapter 9 transforms your operational definitions into a complete **Codebook**—the "source code" for human coders. You'll learn to ensure categories are exhaustive and mutually exclusive, write decision rules for edge cases, and understand how codebook quality determines inter-coder reliability.

The Rulebook

Learning Objectives

- Construct a complete codebook for content analysis.
- Ensure categories are exhaustive and mutually exclusive.
- Write coding rules explicit enough for a stranger to replicate your analysis.
- Understand the relationship between codebooks and inter-coder reliability.

Overview

The codebook is your scientific instrument. It's the "source code" for human coders. If your codebook is vague or incomplete, your data will be unreliable. This chapter teaches you to write coding rules that are **systematic, objective, and replicable**—the three pillars of content analysis.

Content Analysis Methodology

The Three Pillars

Content analysis is:

1. **Systematic**: Rules apply uniformly to all data. No cherry-picking favorable examples.
2. **Objective**: Coders follow the rulebook, not their feelings or intuitions.
3. **Quantitative**: Results are expressed as numbers (frequencies, percentages, correlations).

These principles distinguish content analysis from literary criticism or casual reading. You're not interpreting meaning—you're **counting patterns**.

The Codebook as Algorithm

Think of your codebook as an algorithm for a human processor:

```
IF message contains slur from list X:
  THEN code as "Toxic"
ELSE IF message contains personal insult from list Y:
  THEN code as "Toxic"
ELSE IF message contains threat:
  THEN code as "Toxic"
ELSE:
  THEN code as "Neutral"
```

The codebook must be explicit enough that two coders, working independently, produce nearly identical results. This is the essence of **inter-coder reliability**.

Anatomy of a Codebook

A complete codebook includes:

1. **Unit of analysis**: What are you coding? (tweets, articles, video frames, chat messages)
2. **Variables**: What features are you measuring?
3. **Categories**: What values can each variable take?
4. **Decision rules**: How do you handle edge cases?
5. **Examples**: Prototypical instances of each category.

Developing Categories

Exhaustive

Every unit of analysis (tweet, article, video frame) must fit into a category. No data should be unclassifiable.

Solution: Add an "Other" or "Unclear" category as a safety net, but minimize its use. If >10% of your data ends up in "Other," your categories are incomplete.

Example:

Variable: Message tone

Categories:

- Positive (praise, encouragement, celebration)

- Negative (criticism, insult, complaint)
- Neutral (factual statements, questions)
- **Other** (ambiguous, sarcastic, or unclear tone)

Mutually Exclusive

Each unit should fit into only ONE category per variable. Overlap creates ambiguity.

Bad example:

Variable: Streamer activity

Categories:

- Playing a game
- Talking to chat
- Showing gameplay

Problem: A streamer can play a game while talking to chat. These categories overlap.

Fix:

Variable: Primary streamer activity (code the dominant behavior)

Categories:

- Gameplay-focused (minimal chat interaction)
- Chat-focused (paused gameplay or pre-game/post-game talk)
- Mixed (equal balance of gameplay and chat)

Edge Cases and Decision Rules

Your codebook must anticipate ambiguity.

Example: You're coding Twitch chat for "toxicity."

Message: "GG EZ" (typed after winning a game)

Question: Is this toxic?

Decision rule:

" 'GG EZ' (good game, easy) is coded as **toxic** because it mocks the losing team. Rationale: While 'GG' alone is a neutral sportsmanship phrase, adding 'EZ' (easy) frames the opponent as weak, which constitutes disrespect."

Counterexample: "GG" alone is coded as **neutral**.

Document these decisions. Your codebook is a living document that evolves as you encounter new edge cases during pilot testing.

Writing Operational Definitions (Reprise)

Each variable needs an operational definition (see Chapter 8). The codebook is where those definitions become executable instructions.

Example:

Variable: Gender representation in esports news articles

Conceptual definition: The extent to which male, female, and non-binary players are mentioned, quoted, or featured.

Operational definition:

"For each article, count:

1. Total number of players mentioned by name.
2. Number of male players (use pronouns or prior knowledge to determine gender).
3. Number of female players.
4. Number of non-binary or gender-non-conforming players.
5. If gender is ambiguous, search the player's social media for self-identification. If unverifiable, code as 'Unknown.'

Calculate: % female = (female players / total players) × 100."

Example Codebook: Twitch Chat Toxicity

Unit of Analysis

Individual chat message (defined as one line of text sent by one user).

Variables

1. Message Type

- **Gameplay comment**: References in-game actions ("Nice shot!" "That play was terrible")
- **Social interaction**: Greetings, farewells, chatting with other viewers ("Hey everyone!" "Where's [username]?")
- **Emote/meme**: Only emotes or copypasta with no additional text (e.g., "KEKW")
- **Toxic**: See Variable 2 for subcategories

- **Other**: Meta-commentary about the stream itself ("Stream is lagging")

2. Toxicity (if applicable)

Code as toxic if the message contains:

- **Slurs**: Racial, homophobic, sexist, ableist slurs (see Appendix A for full list)
- **Personal insults**: Directed at streamer, other viewers, or players ("You're trash," "Uninstall," "[Player] is a bot")
- **Threats**: Explicit or implied harm ("I hope you get banned," "KYS" [kill yourself])
- **Harassment**: Repeated unwanted messages to one person, sexual harassment

Do NOT code as toxic:

- **Frustration with game**: "This game sucks," "The lag is terrible"
- **Criticism without insults**: "That strategy didn't work" (neutral analysis, not an attack)
- **Playful banter among known friends**: If context (prior messages, emotes) indicates joking, code as "Social interaction" (Note: This requires judgment—document your reasoning)

3. Target of Toxicity (if applicable)

- Streamer
- Other viewers
- In-game players (not present in chat)
- Game/developer
- Platform (Twitch itself)

Decision Rules

Rule 1: If a message contains both toxic and non-toxic content, code as toxic (toxicity takes precedence).

Example: "Nice shot, but you're still trash" → Toxic

Rule 2: Context matters for sarcasm, but default to literal interpretation when ambiguous.

Example: "Wow, great job throwing the game" → If emotes suggest sarcasm (KEKW), consider context. If unclear, code as toxic.

Rule 3: Copypasta (repeated memes) is coded based on content, not novelty.

Example: "You are an NPC" (repeated 10 times by different users) → Each instance coded as toxic (insult).

Handling Ambiguity

No codebook eliminates all ambiguity. What you CAN do:

1. **Pilot test**: Code 50-100 units before finalizing categories. Identify common edge cases.
2. **Refine iteratively**: After each pilot, revise the codebook to address new ambiguities.
3. **Document judgment calls**: If you make an interpretive decision, note it in the codebook so future coders make the same decision.
4. **Accept some noise**: Perfect reliability is impossible. Aim for .70 inter-coder reliability (Cohen's Kappa or Krippendorff's Alpha).

Codebook Format

Use a structured document:

```
# Codebook: Twitch Chat Toxicity Study

**Project**: Analysis of toxicity in competitive gaming streams
**Coder**: [Your Name]
**Date**: [Date]
**Version**: 1.3

---

## 1. Unit of Analysis
Individual chat message

## 2. Variables

### Variable 1: Message Type
- [Categories and definitions]

### Variable 2: Toxicity
- [Subcategories and examples]

## 3. Decision Rules
- Rule 1: [Description]
- Rule 2: [Description]

## 4. Examples

**Example 1**: "GG EZ"
```

```
- Message Type: Toxic
- Toxicity: Personal insult
- Target: In-game players
- Reasoning: "EZ" mocks opponents

**Example 2**: "This lag is unplayable"
- Message Type: Other
- Toxicity: N/A
- Reasoning: Criticism of technical issue, not a person

---

## Appendix A: Slur List
[Do not reproduce slurs in assignments-reference comprehensive lists maintained by organizati
```

Reliability and the Codebook

A good codebook produces **high inter-coder reliability**: when two coders independently code the same data, they agree most of the time.

Metrics:

- **Percent agreement**: Simple but flawed (doesn't account for chance agreement).
- **Cohen's Kappa** (2 coders): Agreement corrected for chance.
- **Krippendorff's Alpha**: Works for any number of coders, any level of measurement. Gold standard.

Benchmark:

- or .80: Excellent
- or = .70-.79: Acceptable
- or < .70: Unreliable (revise codebook and retrain coders)

Case Study: Coding Gender Representation in Esports News

Research Question: Are female esports players underrepresented in mainstream news coverage compared to their participation rates?

Unit of analysis: News article (from NYT, WSJ, ESPN, 2024)

Variables:

1. **Total players mentioned**: Count

2. **Female players mentioned**: Count
3. **Male players mentioned**: Count
4. **% female representation**: (Female / Total) $\times$ 100
5. **Quote presence**: Does the article quote a female player directly? (Yes/No)

Codebook excerpt:

Decision Rule 1: "Mentioned" means named explicitly (first and last name or gamertag). References like "a female competitor" without a name do NOT count.

Decision Rule 2: Gender determined by:

1. Pronouns used in the article
2. Player's verified social media profiles
3. If ambiguous, code as "Unknown" (do not guess)

Decision Rule 3: If an article mentions the same player multiple times, count only once.

Pilot test:

- Coder A and Coder B independently code 20 articles.
- Calculate Krippendorff's Alpha for "Total players mentioned" and "% female representation."
- If $< .70$, coders meet to discuss disagreements, refine codebook, and retest.

Practice: Building Reliable Codebooks

Exercise 9.1: Ensuring Exhaustive and Mutually Exclusive Categories

Evaluate these category sets. Identify problems and fix them:

Set A - Video game genre: - Action - RPG - Sports - Multiplayer

Problem: _____ **Fix**: _____

Set B - News article tone: - Positive - Somewhat positive - Negative - Critical

Problem: _____ **Fix**: _____

Set C - Streamer activity: - Playing game - Talking to chat - Taking break

Problem: _____ **Fix**: _____

Exercise 9.2: Writing Decision Rules

You're coding tweets for sentiment (Positive, Neutral, Negative). Write decision rules for these edge cases:

Tweet 1: "This game is trash but I love it" - **Ambiguity**: Contains both negative and positive - **Decision rule**: _____

Tweet 2: "lol this streamer is so bad" [followed by laughing emojis] - **Ambiguity**: Literal negative but tone suggests joking - **Decision rule**: _____

Tweet 3: "I can't even" [no other context] - **Ambiguity**: Unclear meaning - **Decision rule**: _____

Real Data: American Idol Codebook Example

Here's a complete, usable codebook for the American Idol judge feedback study (from Chapter 8):

Full Codebook: Judge Feedback Tone in American Idol

Project: Judge Feedback and Contestant Demographics in American Idol
Creator: [Your Name]
Version: 1.0
Date: [Date]

---

1. Unit of Analysis

Definition: A single verbal statement made by one judge about one contestant, during that contestant's performance evaluation.

What counts: - "You have incredible range" = 1 unit - "That was good, but the high notes were shaky" = 1 unit (judge's full turn speaking) - Judge nods without speaking = 0 units (skip)

Exclusions: - Banter between judges = skip - Comments about production/technical issues = skip - Comments about contestants not yet performed = skip

---

2. Variables

Variable 1: Judge Feedback Tone (Ordinal)

Conceptual Definition: The emotional valence (positive-to-negative spectrum) of a judge's verbal feedback.

Measurement Scale (code one category per unit):

- **3 = Strongly Positive**: Compliments contestant's ability with superlatives, predicts success, or expresses genuine enthusiasm. Uses words like "incredible," "stunning," "perfect," "one of the best," or direct encouragement ("you could win this").
- **2 = Mildly Positive**: Acknowledges strengths, appreciates effort, or identifies areas of excellence without superlatives. Uses phrases like "nice job," "I liked your interpretation," "strong vocals," "good control."
- **1 = Neutral/Constructive**: Provides constructive feedback, suggestions, or technical analysis without judgment. Uses phrases like "try a different approach," "slower tempo," "work on your breath control," "interesting choice." Can include both positive and negative elements in balanced proportion.
- **0 = Negative**: Expresses criticism, disappointment, doubt, or concern. Uses phrases like "that wasn't your best," "off-key," "you seemed nervous," "didn't connect," or implies contestant may be eliminated.

Decision Rules:

1. If a comment contains BOTH strong positive and strong negative elements, code the dominant emotion. (Example: "That was mostly great, but you lost it at the bridge" = code 2 (Mildly Positive), as praise dominates.)
2. If tone is ambiguous, look for qualifiers. "Good. But." often indicates Neutral (1), whereas "That was good and really shows your growth" indicates Mildly Positive (2).
3. Sarcasm counts as Negative (0). Example: "Oh wow, those were definitely high notes" = Negative.
4. Judge's facial expression (grimace, smile) can supplement coding only if words are ambiguous. **Primary coding is based on words alone**.
5. Personal comments ("You seem like a great person") that don't directly evaluate singing = Neutral (1).

Variable 2: Contestant Gender Identity (Nominal)

Conceptual Definition: Gender identity as presented by contestant or identified by judges.

Categories:

- **M**: Contestant presents as male or identified by pronouns/judges
- **F**: Contestant presents as female or identified by pronouns/judges
- **X**: Non-binary identity explicitly stated
- **U**: Unclear/not stated during episode

Decision Rules:

- Use judges' pronouns if mentioned ("He has great control")
- If judges use different pronouns, note in comments and mark U
- Do not assume based on appearance alone

Variable 3: Contestant Race/Ethnicity (Nominal)

Conceptual Definition: Visible racial/ethnic identity as noted during contestant introduction or inferred from appearance (see validity note below).

Categories:

- **W**: White/Caucasian
- **B**: Black/African American
- **L**: Latinx/Hispanic
- **A**: Asian/Asian American
- **M**: Middle Eastern/North African
- **R**: Multiracial (visibly multiple identities)
- **O**: Other/unclear

Decision Rules:

- Listen for contestant background statements ("I'm from Mexico," "My family is from Nigeria")
- If contestant self-identifies, use that description
- If no statement, code based on visible presentation (hair, skin tone, names mentioned)
- If truly ambiguous, mark U

VALIDITY CONCERN: Race/ethnicity coding by external coders is subjective and can perpetuate bias. Acceptable use cases: - Comparing to official contestant demographics if available from American Idol production - Testing for coder bias (do different coders see race differently?) - Analyzing *judge* statements about race/ethnicity, not contestant identity itself

Best Practice: Obtain actual demographic data from American Idol official records if this data is ever available. Do not rely on appearance-based coding for publication.

Variable 4: Advancement (Binary)

Conceptual Definition: Whether contestant progresses to the next round after this performance.

Categories:

- **Yes**: Judge votes to advance contestant (or contestant advances by audience vote)
- **No**: Judge votes against, or contestant is eliminated

Decision Rules:

- Code based on judge's immediate decision after feedback
- If judges are split, code each judge's decision separately
- In later rounds (live voting), code based on audience/judge consensus decision

---

3. Coding Example Walkthrough

Scenario: Contestant Maya (Female, Latinx) performs. Judge Luke says:

"Maya, that was really great. Your tone is beautiful, but I'd love to see you take more risks with your interpretation next time. You're definitely moving forward."

How to Code:

Variable	Value	Reasoning
Judge Feedback Tone	2 (Mildly Positive)	Compliments ("really great," "beautiful tone") dominate; constructive feedback ("take more risks") is supportive, not critical. "Moving forward" is positive outcome. Code as Mildly Positive.
Contestant Gender	F	Judge uses "your" (neutral), but previous introduction indicated female. Code F.
Contestant Race/Ethnicity	L	Contestant's self-introduction mentioned Latinx family background. Code L.
Advancement	Yes	Judge explicitly says "definitely moving forward." Code Yes.

---

4. Pilot Test Results

Before analyzing entire seasons, test your codebook on 2-3 episodes with a partner.

What to look for:

1. How many disagreements do you get on each variable?
2. Which decision rules are unclear?
3. Are any categories rarely used ($< 5\%$ of codings)?

Example pilot test (one episode, 6 contestants $\times$ 3 judges = 18 codings):

Variable	Agreement	Note
Judge Tone	16/18 (89%)	Two tone judgments disagreed. Refined "Mildly Positive" definition.
Gender	18/18 (100%)	Clear from introductions.

Variable	Agreement	Note
Race/Ethnicity	16/18 (89%)	Two coders disagreed on one mixed-race contestant. Revised rule: "If ambiguous, ask contestant directly."
Advancement	18/18 (100%)	Judges' decisions are explicit.

Decision: Codebook is ready to use (89% avg agreement > 70% threshold). Proceed with data collection.

Real Data Download

American Idol datasets are available from TidyTuesday:

`https://github.com/rfordatascience/tidytuesday/tree/master/data/2024/2024-07-23`

For coding judge feedback from actual episodes, you'll need to:

1. **Download**: Episode lists and contestant data from TidyTuesday
2. **Watch**: Episodes on streaming (Hulu, Disney+) or YouTube clips
3. **Code**: Use this codebook to record judge feedback
4. **In Chapter 10**: Learn sampling strategies (which episodes to code), and in Chapter 11: enter your codes into R for analysis

---

Exercise 9.3: Codebook Construction

Create a mini-codebook for analyzing Instagram post captions for "authenticity cues":

Unit of Analysis: _____

Variable 1: Self-Disclosure - Conceptual definition: _____ - **Categories**: - _____ - _____ - _____ - **Decision rules**: - _____

Variable 2: Filter Use - Conceptual definition: _____ - **Categories**: - _____ - _____ - **Decision rules**: - _____

Edge cases anticipated: _____

Exercise 9.4: Interpreting Reliability Metrics

Calculate percent agreement and identify the problem:

Two coders analyze 50 messages. Results:

Coder A	Coder B	Count
Toxic	Toxic	8
Toxic	Neutral	5
Neutral	Toxic	3
Neutral	Neutral	34

Percent agreement: _____ (Show calculation)

Interpretation: Is this acceptable? _____

Where do disagreements cluster?: _____

How would you revise the codebook?: _____

Reflection Questions

1. **The Algorithm Metaphor**: This chapter compared codebooks to algorithms. How far can this metaphor go? What do human coders do that algorithms can't (yet)? What advantages do algorithms have?

2. **Judgment vs. Objectivity**: Some edge cases require judgment (e.g., "is this sarcasm?"). Does this mean content analysis isn't truly "objective"? How do you balance the need for coder judgment with the goal of reliability?
3. **Your Codebook Anxiety**: If you're developing a codebook for your project, what keeps you up at night? Which categories will be hardest to define? How will you handle ambiguity?

Chapter Summary

This chapter taught you to construct codebooks that produce reliable, replicable content analysis:

- Content analysis must be **systematic** (rules apply uniformly), **objective** (coders follow rules, not intuitions), and **quantitative** (results expressed as numbers)
- A **codebook** is the "algorithm" human coders execute—it must be explicit enough for independent coders to produce nearly identical results
- Complete codebooks include: **unit of analysis**, **variables**, **categories**, **decision rules**, and **examples**
- Categories must be **exhaustive** (every unit fits somewhere) and **mutually exclusive** (each unit fits in only one category per variable)
- **Decision rules** handle edge cases and ambiguity systematically
- **Pilot testing** identifies vague definitions and common disagreements
- **Inter-coder reliability** (ICR) measures consistency: Cohen's Kappa (2 coders), Krippendorff's Alpha (any number of coders, any measurement level)
- Acceptable reliability: or .70 (adequate), .80 (excellent)
- Low reliability signals unclear definitions, insufficient training, or inherently ambiguous constructs

Key Terms

- **Cohen's Kappa ( )**: Inter-coder reliability metric for 2 coders, corrects for chance agreement
- **Codebook**: Systematic instructions for coding content, including variable definitions, categories, and decision rules
- **Content analysis**: Systematic, objective, quantitative analysis of communication content
- **Decision rule**: Explicit guideline for handling ambiguous or edge cases
- **Exhaustive categories**: Category set where every observation can be classified (no uncodeable cases)
- **Inter-coder reliability (ICR)**: Consistency between independent coders

- **Krippendorff's Alpha ( )**: ICR metric for any number of coders and any measurement level
- **Mutually exclusive categories**: Categories with no overlap (each unit fits in only one)
- **Operational definition**: Concrete instructions for identifying and measuring a variable
- **Percent agreement**: Simple reliability metric (agreements/total) that doesn't correct for chance
- **Pilot testing**: Preliminary coding phase to test and refine the codebook
- **Unit of analysis**: The object being coded (tweet, article, video, message)

Looking Ahead

Chapter 10 addresses **sampling**—defining your study's boundaries and verifying your instrument works. You'll learn sampling techniques, conduct pilot studies to test codebook reliability, calculate inter-coder agreement, and recognize how sampling bias affects generalizability.

The Sample

Learning Objectives

- Define sampling frames and select appropriate sampling techniques.
- Conduct a pilot study to test codebook reliability.
- Calculate and interpret inter-coder reliability metrics.
- Recognize sampling bias and its implications for generalizability.

Overview

You've built the instrument (codebook). Now you must define the boundaries of your study (the **sample**) and verify that your instrument works (**pilot testing and reliability**). This chapter addresses two critical questions: (1) What data will you analyze? (2) How do you know your analysis is reliable?

Populations and Sampling Frames

The Census vs. The Sample

A **census** analyzes every member of a population (e.g., all tweets ever posted about "Fortnite"). This is rarely feasible—there are billions of tweets.

A **sample** is a subset selected to represent the population. The key question: Is your sample **representative**, or is it biased?

Example:

- **Population**: All Twitch streams broadcast in 2024.
- **Sample**: 100 randomly selected Twitch streams from February 2024.

If your sample is representative, findings generalize to the population. If it's biased (e.g., only top streamers), findings apply only to that subset.

The Sampling Frame

The **sampling frame** is the specific list from which you draw your sample.

Example:

- **Population**: All Twitch streamers.
- **Sampling Frame**: The top 500 streamers by average concurrent viewers on TwitchTracker.com as of February 1, 2024.

Critical distinction: Your sampling frame is NOT the same as your population. The top 500 streamers don't represent **all** Twitch users—they represent **popular** streamers. Acknowledge this limitation in your report:

> "This study examines popular streamers (defined as those with >1,000 average viewers). Findings may not generalize to small or mid-tier streamers."

Sampling Techniques

Simple Random Sampling

Every unit in the sampling frame has an **equal chance** of selection. Use a random number generator.

Example: If your frame has 1,000 tweets, randomly select 100.

Pros: Unbiased. Easy to justify statistically.
Cons: May miss important subgroups if they're rare.

Systematic Sampling

Select every kth item. If you have 1,000 items and want 100, select every 10th item (k = 10).

Pros: Simple to execute.
Cons: If there's a hidden pattern (e.g., every 10th tweet is a bot), you'll introduce bias.

Use case: Systematic sampling is common when random sampling is impractical (e.g., selecting every 10th article from a chronological archive).

Stratified Sampling

Divide the population into **strata** (subgroups), then randomly sample from each stratum.

Example:

- **Population**: 1,000 esports news articles.
- **Strata**: Sports section (300 articles), Entertainment section (400 articles), Tech section (300 articles).
- **Sample**: Randomly select 50 from each stratum (ensuring representation from all three).

Pros: Ensures representation of important subgroups.
Cons: Requires knowledge of population structure.

Purposive (Theoretical) Sampling

Select cases based on **theoretical relevance**, not randomness.

Example: You're studying how streamers respond to harassment. You purposively select:

- 10 female streamers (who experience higher harassment rates)
- 10 male streamers (for comparison)
- 10 non-binary streamers (to examine intersectional experiences)

Pros: Efficient for exploratory research and case studies.
Cons: Not statistically representative. Can't generalize to the broader population.

Use case: Qualitative research, pilot studies, or when studying rare phenomena.

Convenience Sampling

Select whatever data is easily accessible.

Example: "I analyzed 50 streams from my Twitch following list."

Pros: Fast and cheap.
Cons: Highly biased. Your following list reflects your preferences, not the population.

When acceptable: Pilot studies, exploratory research, or when you explicitly acknowledge limitations ("This is a non-representative exploratory study").

Sample Size Considerations

How much data do you need?

General rules:

- **Qualitative/exploratory**: 20-50 units (e.g., 20 streams, 50 tweets) may suffice if richly analyzed.
- **Quantitative/descriptive**: 100-300 units provides reasonable precision for frequency estimates.
- **Inferential statistics**: The more subgroups you compare, the larger your sample. For Chi-Square tests, aim for expected cell counts 5.

Reality check: Student projects typically analyze 50-200 units. Larger samples require more time and resources.

Pilot Testing the Codebook

Before coding your full sample, you must **pilot test** your codebook to ensure it's reliable and feasible.

The Pilot Process

1. **Select a subsample**: 10% of your data (e.g., if your full sample is 200 tweets, pilot test on 20 tweets).
2. **Two coders independently code the pilot**: Use the codebook without discussion.
3. **Calculate inter-coder reliability** (see section 10.5).
4. **Identify disagreements**: Where did coders diverge? Were decision rules unclear?
5. **Revise the codebook**: Add clarifications, examples, or decision rules.
6. **Re-pilot if necessary**: If reliability is low ($< .70$), refine and test again.

What to Look For

- **High disagreement on specific categories**: One category may have vague definitions.
- **Coders using "Other" frequently**: Categories aren't exhaustive.
- **Systematic bias**: One coder consistently codes more strictly than the other (indicates training issue).

Inter-Coder Reliability (ICR)

ICR measures consistency between coders. High ICR means your codebook is clear and your categories are well-defined.

Percent Agreement

Formula:

$$\text{Agreement} = \frac{\text{Number of agreements}}{\text{Total units coded}}$$

Example: Two coders code 50 tweets. They agree on 40.

$$\text{Agreement} = \frac{40}{50} = 0.80 \ (80\%)$$

Problem: Percent agreement doesn't account for **chance agreement**. If you have only two categories (Toxic vs. Neutral), coders might agree 50% of the time just by guessing.

Cohen's Kappa ( )

Cohen's Kappa corrects for chance agreement.

Formula (simplified):

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

Where:

- P_o = observed agreement
- P_e = expected agreement by chance

Interpretation:

- = 1.0: Perfect agreement
- = 0.8-1.0: Excellent
- = 0.7-0.79: Acceptable
- $<$ 0.7: Unreliable (revise codebook)

Limitation: Cohen's Kappa works for **2 coders only**.

Krippendorff's Alpha ( )

Krippendorff's Alpha is the gold standard:

- Works for **any number of coders** (2, 3, 10, etc.)
- Works for **any level of measurement** (nominal, ordinal, interval, ratio)
- Accounts for chance agreement and sample size

Formula (conceptual):

$$\alpha = 1 - \frac{\text{Observed disagreement}}{\text{Expected disagreement by chance}}$$

Interpretation: Same as Cohen's Kappa (.80 excellent, .70-.79 acceptable, <.70 unreliable).

How to calculate: Use R, Python, or online calculators (don't do this by hand).

When Reliability is Low: Troubleshooting

If or $<$ .70:

1. **Identify problem variables**: Calculate reliability for each variable separately. Which has the lowest agreement?
2. **Examine edge cases**: Review disagreements. Are coders confused by specific examples?
3. **Clarify definitions**: Add decision rules or rephrase operational definitions.
4. **Retrain coders**: Meet with coders to discuss the codebook. Ensure shared understanding.
5. **Re-pilot**: Code a new subsample and recalculate reliability.

Example:

- **Problem**: Coders disagree on whether "GG EZ" is toxic (= .65).
- **Diagnosis**: One coder codes all "GG" variants as neutral (sportsmanship). The other codes "GG EZ" as toxic (mockery).
- **Solution**: Add decision rule: " 'GG EZ' is coded as toxic because 'EZ' mocks the opponent. 'GG' alone is neutral."
- **Re-pilot**: improves to .78.

Lone Coder Problem

Many student projects have only one coder (you). Without a second coder, you can't calculate ICR.

Solutions:

1. **Partner with a classmate**: Exchange datasets and code each other's samples.
2. **Code twice, separately**: Code your pilot, wait 1 week, code it again without looking at your original results. Compare the two rounds (intra-coder reliability).
3. **Acknowledge the limitation**: "This study was conducted by a single coder. While the codebook was designed for replicability, inter-coder reliability was not assessed. Future research should verify findings with multiple coders."

Sampling Bias and Generalizability

Every sample has limitations. Your responsibility is to **acknowledge** them, not eliminate them (which is often impossible).

Example biases:

- **Temporal bias**: Analyzing February 2024 tweets may not represent December 2024 (holiday effects, new game releases).
- **Platform bias**: Twitch users YouTube Gaming users Facebook Gaming users. Findings from one platform may not transfer.
- **Survivorship bias**: Analyzing only popular streamers ignores those who quit due to harassment or burnout.

How to address: In your limitations section, write:

> "This study sampled the top 100 Twitch streamers by viewership. These findings may not generalize to small or mid-tier streamers, who face different economic pressures and community dynamics. Future research should examine stratified samples across viewership tiers."

Case Study: Pilot Testing a Twitch Toxicity Codebook

Goal: Code 50 Twitch streams for toxic chat messages.

Step 1: Develop codebook (see Chapter 9).

Step 2: Select pilot sample:

- Randomly select 5 streams (10% of final sample).
- Each stream provides ~100 chat messages = 500 total pilot messages.

Step 3: Two coders independently code pilot.

Step 4: Calculate Krippendorff's Alpha:

- Variable: Message type (Gameplay comment, Social interaction, Toxic, Other)
- = .68 (below acceptable threshold)

Step 5: Diagnose:

- Coders disagree most on "Toxic" vs. "Gameplay comment."
- Example: "That play was trash" — Coder A: Toxic (insult). Coder B: Gameplay comment (criticism).

Step 6: Revise codebook:

- Add decision rule: "Criticism of gameplay ('That strategy didn't work') is **not toxic**. Criticism of the player ('You are trash') is **toxic**. If the message includes 'that' (referring to the action) rather than 'you' (referring to the person), code as Gameplay comment."

Step 7: Re-pilot:

- Code 5 new streams.
- = .76 (acceptable).

Step 8: Code full sample with confidence.

Real Data: American Idol Sampling and Reliability

Let's apply sampling and reliability testing to the American Idol judge feedback study (Chapters 8-9).

Research Setup

Research Question: Do judges provide more positive feedback to contestants from certain demographic backgrounds?

Population: All American Idol episodes (2002-present, ~400 episodes)

Feasibility challenge: Watching and coding 400 episodes is impossible for a student project. We must sample strategically.

Step 1: Define Sampling Frame

Decision: Focus on recent seasons with professional broadcast consistency.

Sampling Frame: American Idol Seasons 20-23 (2022-2024, $n = 128$ episodes total)

Why: Recent seasons have consistent judging panels, high video quality, and accessible data.

Limitation acknowledged: Findings won't generalize to early Idol seasons (2002-2010), which had different judging philosophies.

Step 2: Stratify by Round

Judges behave differently in auditions (mass eliminations, quicker feedback) vs. live shows (deeper critique).

Strata: - Audition episodes (Days 1-5, low-pressure feedback): 40 episodes - Hollywood week (mid-round, mixed pressure): 30 episodes
- Live performance shows (high-stakes, detailed feedback): 58 episodes

Step 3: Calculate Sample Size

Target: 60 episodes (50% sample for feasibility)

Stratified allocation (proportional to stratum): - Audition: 15 episodes ($40/128 \times 60$) - Hollywood: 14 episodes ($30/128 \times 60$) - Live shows: 31 episodes ($58/128 \times 60$)

Why stratified?: Ensures representation across judging contexts. Audition feedback ("Great!") differs from live show feedback ("You're fighting for your spot").

Step 4: Random Selection Within Strata

```r
# In R (Chapter 11+), you'd do:
set.seed(42)
audition_episodes <- c(1:40)
hollywood_episodes <- c(1:30)
live_episodes <- c(1:58)

sample_audition <- sample(audition_episodes, 15)
sample_hollywood <- sample(hollywood_episodes, 14)
sample_live <- sample(live_episodes, 31)
```

```r
selected_episodes <- c(
  paste0("Audition_", sample_audition),
  paste0("Hollywood_", sample_hollywood),
  paste0("Live_", sample_live)
)

# Document for reproducibility
write.csv(selected_episodes, "american_idol_sample.csv")
```

Step 5: Pilot Reliability Study

Pilot sample: 3 episodes (5% of 60-episode target), 1 from each stratum

Example pilot results (two coders, 3 episodes, ~40 judge comments total):

Episode 1 (Audition):

Judge	Contestant	Gender	Ethnicity	Tone	Agreement
Katy	C1	F	W	3 vs 3	Yes
Luke	C1	F	W	2 vs 2	Yes
Lionel	C1	F	W	2 vs 3	**Disagree**

Issue identified: Coders disagree on whether "You're really good, but you seem nervous" is Mildly Positive (Coder A) vs. Strongly Positive (Coder B).

Codebook revision (from Chapter 9): - Add example: "Nervousness comment is Mildly Positive if it identifies a fixable issue. Only code Strongly Positive if judge emphasizes strengths *without* criticism."

Step 6: Re-Pilot and Calculate Reliability

After revision, code 3 new episodes.

Krippendorff's Alpha calculation:

```r
library(irr)

# Data from 3 pilot episodes (40 comments)
judge_tone_coder_a <- c(3, 2, 2, 1, 0, 3, 2, 2, 1, 3, 2, 2, 3, 1, 2,
                        0, 2, 3, 1, 2, 3, 2, 2, 1, 0, 2, 3, 1, 2, 3,
                        2, 1, 3, 2, 1, 2, 3, 2, 1, 2)
```

```r
judge_tone_coder_b <- c(3, 2, 2, 1, 0, 3, 2, 2, 1, 3, 2, 2, 3, 1, 2,
                        0, 2, 3, 1, 2, 3, 2, 2, 1, 0, 2, 3, 1, 2, 3,
                        2, 1, 3, 2, 1, 2, 3, 2, 1, 2)

# Create matrix for Krippendorff's Alpha (observations × coders)
reliability_data <- matrix(c(judge_tone_coder_a, judge_tone_coder_b),
                           ncol = 2)

# Calculate Krippendorff's Alpha
alpha <- kripp.alpha(t(reliability_data), method = "ordinal")
print(alpha)
# Output: Krippendorff's alpha: 0.82
```

Interpretation: = 0.82 (excellent reliability, > 0.80 threshold). Codebook is clear enough for independent coders. Proceed to full sample.

Step 7: Calculate Demographics Reliability

Demographic coding should be simpler (gender/ethnicity more observable than tone):

Expected results: - Gender: 0.95+ (nearly perfect—judges usually state pronouns) - Race/Ethnicity: 0.72 (adequate—appearance assessment is subjective, see validity concerns in Chapter 9)

Decision rule addition: "If race/ethnicity disagree, coders discuss and reach consensus before full coding."

Step 8: Document Sampling and Reliability

Reporting template (for your final paper):

> "We conducted a stratified random sample of 60 American Idol episodes (2022-2024) across three performance rounds: auditions ($n = 15$), Hollywood week ($n = 14$), and live shows ($n = 31$). A pilot study ($n = 3$ episodes, 40 judge comments) with two independent coders yielded Krippendorff's = 0.82 for judge feedback tone, 0.95 for gender coding, and 0.72 for race/ethnicity assessment. The lower reliability for race/ethnicity reflects inherent subjectivity in appearance-based coding; we addressed this by resolving disagreements through coder discussion before analyzing the full sample. Based on pilot reliability, we proceeded to code all 60 episodes."

Practice: Sampling and Reliability

Exercise 10.1: Sampling Technique Selection

For each research question, select the MOST appropriate sampling technique and justify:

Techniques: Simple Random, Systematic, Stratified, Purposive, Convenience

RQ1: "What proportion of esports news articles mention female players?" - **Population**: All esports news articles in 2024 - **Sampling technique**: _____ - **Justification**: _____ - **Sample size recommendation**: _____

RQ2: "How do top streamers vs. mid-tier streamers engage with chat differently?" - **Population**: All Twitch streamers - **Sampling technique**: _____ - **Justification**: _____ - **Sample size**: _____

RQ3: "What themes emerge in Reddit discussions about gaming addiction?" - **Population**: r/gaming posts about addiction - **Sampling technique**: _____ - **Justification**: _____

Exercise 10.2: Calculating Inter-Coder Reliability

Two coders independently coded 50 Twitch chat messages for toxicity (Toxic vs. Neutral):

Coding results:

Message	Coder A	Coder B	Message	Coder A	Coder B
1	Toxic	Toxic	26	Neutral	Neutral
2	Neutral	Neutral	27	Toxic	Neutral
3	Toxic	Toxic	28	Neutral	Neutral
...	...	...	...	...	...

Summary of agreements: - Both coded "Toxic": 15 messages - Both coded "Neutral": 30 messages - Coder A said "Toxic," Coder B said "Neutral": 3 messages - Coder A said "Neutral," Coder B said "Toxic": 2 messages

Calculate:

1. **Total agreements** = 15 + 30 = ____
2. **Percent agreement** = (Total agreements / 50) × 100 = ____%
3. **Is this acceptable?** ____
4. **What does percent agreement NOT account for?** ____

Cohen's Kappa Interpretation (given = .78): - **Interpretation**: Acceptable / Excellent / Poor (circle one) - **Should you proceed with coding the full sample?**: Yes / No - **Why or why not?**: _____

Exercise 10.3: Sample Size Decisions

For each scenario, recommend an appropriate sample size and justify:

Scenario A: Exploring themes in YouTube comments about mental health - **Research type**: Qualitative thematic analysis - **Recommended sample size**: _____ - **Justification**: _____

Scenario B: Testing whether female streamers receive more donations than male streamers - **Research type**: Quantitative comparison (t-test) - **Recommended sample size**: _____ - **Justification**: _____

Scenario C: Describing the distribution of game genres on Twitch's front page - **Research type**: Descriptive frequencies - **Recommended sample size**: _____ - **Justification**: _____

Exercise 10.4: Diagnosing Low Reliability

You pilot-tested your codebook with a classmate. Krippendorff's Alpha = .62 (below threshold).

Here are your disagreements:

Message	Your Code	Partner's Code
"GG EZ"	Toxic	Neutral
"That play was trash"	Toxic	Gameplay comment
"Skill issue lol"	Toxic	Neutral
"You're such a bot"	Toxic	Neutral

Diagnosis:

1. **What pattern do you notice in the disagreements?** _____
2. **What's likely unclear in your codebook?** _____
3. **Proposed solution (new decision rule to add)**: _____

Exercise 10.5: Stratified Sampling in Practice

Population: All esports news articles published in 2024 (N = 1,200)

Strata breakdown: - Mainstream sports outlets (NYT, ESPN): 300 articles (25%) - Gaming-specific outlets (IGN, Kotaku): 600 articles (50%) - Tech outlets (Wired, TechCrunch): 300 articles (25%)

Task: Design a stratified sample of 120 articles (10% of population)

1. **How many from each stratum?**
   - Mainstream sports: _____
   - Gaming outlets: _____
   - Tech outlets: _____
2. **Why use stratified sampling instead of simple random?**
   - _____
3. **How would you randomly select articles within each stratum?**
   - _____

Working Example: Complete Sampling and Reliability Workflow

Let me walk you through a realistic example from start to finish.

Study: Gender Representation in Gaming News

Research Question: What percentage of esports players mentioned in mainstream news are female?

Step 1: Define Population and Frame

- **Population**: All esports news articles published in English-language mainstream media
- **Sampling Frame**: Articles from NYT, Washington Post, ESPN, BBC Sport mentioning "esports" in 2024 (accessed via Nexis Uni)
- **Frame size**: 450 articles

Limitation acknowledged: This frame excludes gaming-specific outlets (IGN, Kotaku), which may cover esports differently.

Step 2: Choose Sampling Method

Decision: Stratified random sampling

Rationale: We want representation across all four quarters of 2024 (seasonal variation).

Strata: - Q1 (Jan-Mar): 120 articles - Q2 (Apr-Jun): 105 articles - Q3 (Jul-Sep): 110 articles - Q4 (Oct-Dec): 115 articles

Step 3: Determine Sample Size

Target: 100 articles (22% of frame, provides ±10% margin of error)

Stratified allocation (proportional to stratum size): - Q1: 27 articles (120/450 × 100) - Q2: 23 articles (105/450 × 100) - Q3: 24 articles (110/450 × 100) - Q4: 26 articles (115/450 × 100)

Step 4: Select Sample

Using R:

```
# Create sampling frame
frame_q1 <- 1:120
frame_q2 <- 1:105
frame_q3 <- 1:110
frame_q4 <- 1:115

# Set seed for reproducibility
set.seed(12345)

# Random sample from each stratum
sample_q1 <- sample(frame_q1, 27)
sample_q2 <- sample(frame_q2, 23)
sample_q3 <- sample(frame_q3, 24)
sample_q4 <- sample(frame_q4, 26)

# Document which articles were selected
selected_articles <- c(paste0("Q1-", sample_q1),
                       paste0("Q2-", sample_q2),
                       paste0("Q3-", sample_q3),
                       paste0("Q4-", sample_q4))
```

Step 5: Develop Codebook

Variables: 1. Article ID 2. Outlet (NYT, Post, ESPN, BBC) 3. Publication date 4. Total players mentioned (count) 5. Female players mentioned (count) 6. % female representation

Operational definition: "Player mentioned" = named by full name or gamertag. Gender determined by pronouns in article or verified social media.

Step 6: Pilot Test (10 articles)

Two coders independently code 10 randomly selected articles from the sample.

Results:

Article	Coder A Total	Coder B Total	Coder A Female	Coder B Female
Q1-15	8	8	2	2
Q1-47	5	5	0	0
Q2-18	12	13	3	2
Q2-55	6	6	1	1
…	…	…	…	…

Issue identified: Coder B counted one ambiguous mention ("a player from Team Liquid") while Coder A did not.

Codebook revision: Add decision rule: "Only count players named explicitly (first/last name or gamertag). Generic references like 'a player' or 'the team's star' do NOT count."

Step 7: Re-Pilot and Calculate Reliability

After revision, re-code the same 10 articles.

Using R to calculate Krippendorff's Alpha:

```
library(irr)

# Create reliability matrix (each row = article, each column = coder)
total_players <- matrix(c(
  8, 8,
  5, 5,
  12, 12,
  6, 6,
  10, 10,
```

```
  15, 14,
  7, 7,
  9, 9,
  11, 11,
  4, 4
), ncol = 2, byrow = TRUE)

# Calculate Krippendorff's Alpha
kripp.alpha(t(total_players), method = "ratio")
```

Result: = .94 (excellent!)

Step 8: Code Full Sample

With reliable codebook, proceed to code all 100 articles.

Final findings: 847 total players mentioned; 127 were female (15% representation).

Report limitation: "This sample represents mainstream sports coverage in 2024. Gaming-specific outlets (e.g., IGN) may show different patterns."

Two coders analyze 40 tweets. Calculate percent agreement and interpret:

Coder A	Coder B	Count
Positive	Positive	12
Positive	Neutral	3
Positive	Negative	1
Neutral	Positive	2
Neutral	Neutral	15
Neutral	Negative	2
Negative	Positive	0
Negative	Neutral	1
Negative	Negative	4

Total agreements: _____ (show work) **Total cases**: 40 **Percent agreement**: _____

Interpretation: Is this acceptable? _____ **What additional metric should you calculate?**: _____ **Why is percent agreement potentially misleading here?**: _____

Exercise 10.3: Identifying Sampling Bias

For each study, identify the **type of sampling bias** and explain how it limits generalizability:

Study A: A researcher analyzes toxicity in 100 Twitch streams—all from the researcher's "Following" list. - **Bias type**: _____ - **Limitation**: _____ - **How to improve**: _____

Study B: A researcher codes news articles about esports from January 2024 only (ignoring that a major championship happened in March). - **Bias type**: _____ - **Limitation**: _____ - **How to improve**: _____

Study C: A researcher studies "successful streamers" by only analyzing channels with >10,000 concurrent viewers, ignoring those who quit due to harassment. - **Bias type**: _____ - **Limitation**: _____ - **Impact on conclusions**: _____

Exercise 10.4: Planning Your Pilot Study

For your research project (or a hypothetical one):

Research question: _____

Sampling frame: _____

Sampling technique: _____

Target sample size: _____

Pilot study plan: - **Pilot sample size** (10% of full sample): _____ - **Number of coders**: _____ - **Expected challenges**: _____ - **Success criteria** (minimum): _____ - **If reliability is low, I will**: _____

Reflection Questions

1. **Generalizability Trade-offs**: Think about your research question. What population do you *want* to generalize to (all streamers? all gamers? all people)? What population can you *actually* sample given time and resource constraints? Is the gap acceptable?
2. **The Lone Coder Problem**: If you're coding alone (no partner for ICR), how can you ensure your coding is reliable? What strategies might help (coding twice with time between, extreme clarity in codebook, transparency about limitations)?

3. **Sampling as Argument**: Your sampling choices shape your findings. If you study only top streamers, you might conclude "streaming is lucrative." If you study failed streamers, you might conclude "streaming is unsustainable." How do you avoid cherry-picking samples that confirm your expectations?

Chapter Summary

This chapter taught you to define study boundaries and verify your codebook's reliability:

- **Census** (entire population) is rarely feasible; **samples** must represent the population or acknowledge limitations
- **Sampling frame** is the specific list from which you draw your sample (often narrower than the ideal population)
- **Sampling techniques**: Simple random (unbiased), systematic (every kth item), stratified (ensuring subgroup representation), purposive (theory-driven), convenience (easy but biased)
- **Sample size** depends on research goals: 20-50 for exploratory, 100-300 for descriptive, larger for inferential statistics
- **Pilot testing** (typically 10% of sample) identifies codebook ambiguities and estimates reliability
- **Inter-coder reliability** measures consistency: percent agreement (simple but flawed), Cohen's Kappa (2 coders), Krippendorff's Alpha (any number of coders, gold standard)
- **Acceptable reliability**: or .70 (adequate), .80 (excellent)
- **Low reliability troubleshooting**: identify problem variables, examine edge cases, clarify definitions, retrain coders, re-pilot
- **Sampling bias** (temporal, platform, survivorship) limits generalizability—acknowledge explicitly in limitations section

Key Terms

- **Census**: Analysis of every member of a population (rarely feasible)
- **Convenience sampling**: Selecting easily accessible cases (quick but biased)
- **Generalizability**: The extent to which sample findings apply to the broader population
- **Inter-coder reliability (ICR)**: Consistency between independent coders
- **Krippendorff's Alpha ( )**: Reliability metric for any number of coders and measurement levels
- **Pilot study**: Preliminary coding phase to test codebook and estimate reliability
- **Population**: The complete set of cases of interest
- **Purposive sampling**: Theory-driven selection of specific cases (not statistically representative)

- **Sample**: A subset of the population selected for analysis
- **Sampling bias**: Systematic over- or under-representation of certain population characteristics
- **Sampling frame**: The specific list from which a sample is drawn
- **Simple random sampling**: Every unit has equal probability of selection (unbiased)
- **Stratified sampling**: Random sampling within defined subgroups to ensure representation
- **Systematic sampling**: Selecting every kth item (e.g., every 10th article)

Looking Ahead

Chapter 11 introduces **data wrangling**—transforming messy, raw coding sheets into clean, analysis-ready datasets using R. You'll learn the tidy data philosophy, essential dplyr verbs, how to handle missing values and inconsistencies, and why documenting every cleaning decision matters for reproducibility.

Wrangling the Data

Learning Objectives

- Apply the tidy data philosophy in R.
- Import, clean, and transform raw coding sheets into analysis-ready datasets.
- Handle common data problems: missing values, inconsistent categories, duplicates.
- Document data cleaning decisions for reproducibility.

Overview

You've collected your data (coded your sample). Now it's time to **wrangle** it into shape for analysis. Data wrangling is the unsexy but essential work of transforming messy, raw data into clean, analysis-ready datasets. This chapter introduces the technical skills of data management in R, emphasizing reproducibility and the **tidy data philosophy**.

The Tidy Data Philosophy

Tidy data has three rules (Wickham, 2014):

1. Each **row** is an observation (one tweet, one article, one video).
2. Each **column** is a variable (sentiment, game, author).
3. Each **cell** contains one value (no lists or embedded tables).

Example of Messy Data:

Respondent	Games
Alice	Fortnite, Valorant, Overwatch

Problem: The "Games" column contains three values. You can't analyze "favorite game" without splitting this string.

Same Data, Tidied:

Respondent	Game
Alice	Fortnite
Alice	Valorant
Alice	Overwatch

Now each row is one observation (one person-game pair). Each cell has one value. This is tidy.

Why it matters: Tidy data works seamlessly with R's tidyverse tools (`dplyr`, `ggplot2`, `tidyr`). Messy data requires manual reshaping before every analysis.

Importing Data into R

Using Base R

```
raw_data <- read.csv("data_raw/coding_sheet.csv")
```

Problems:

- Column names may have spaces or special characters: "Message Type" becomes `Message.Type`.
- R may misinterpret data types (numbers as text, dates as strings).

Using `readr` (Tidyverse)

```
library(readr)

raw_data <- read_csv("data_raw/coding_sheet.csv")
```

Advantages:

- Preserves column names (spaces allowed).
- Better type detection (automatically parses dates and numbers).
- Faster for large files.

Pro tip: Always inspect your data after import:

```
glimpse(raw_data)  # Shows column names and types
head(raw_data)     # Shows first 6 rows
```

Common Data Problems and Solutions

Missing Values

Problem: Some cells are blank or contain "NA", "N/A", "Unknown".

Solution: Standardize missing values:

```
library(dplyr)

clean_data <- raw_data %>%
  mutate(game = na_if(game, "Unknown")) %>%  # Convert "Unknown" to NA
  mutate(game = na_if(game, ""))              # Convert empty strings to NA
```

Decision: Do you:

- **Exclude rows with missing data** (listwise deletion)?
- **Impute values** (fill in with the mode, median, or a logical guess)?
- **Keep NA and analyze only complete cases** (pairwise deletion)?

Document your choice and justify it in your methods section.

Inconsistent Categories

Problem: Coders entered the same category in different ways:

- "Valorant", "valorant", "VALORANT", "Val"

Solution: Standardize with `case_when()` or `recode()`:

```
clean_data <- raw_data %>%
  mutate(game = tolower(game)) %>%  # Convert to lowercase
  mutate(game = case_when(
    game %in% c("valorant", "val") ~ "Valorant",
    game %in% c("overwatch", "ow") ~ "Overwatch",
    game == "cs:go" ~ "CS:GO",
    TRUE ~ game  # Keep everything else as is
  ))
```

Duplicate Rows

Problem: Some observations were coded twice accidentally.

Solution: Remove duplicates:

```
clean_data <- raw_data %>%
  distinct()  # Keeps only unique rows
```

Caution: If duplicates are legitimate (e.g., two coders coded the same data for reliability), don't delete them—merge them after calculating ICR.

Incorrect Data Types

Problem: "Donation amount" is stored as text ("$5.00") instead of numeric.

Solution: Convert types:

```
clean_data <- raw_data %>%
  mutate(donation = parse_number(donation))  # Extracts numbers from "$5.00"
```

Essential `dplyr` Verbs

Before diving into complex tidyverse code, let's compare **base R** (what you've already learned) with **tidyverse** (what makes analysis easier). Understanding both helps you appreciate why we use tidyverse.

Side-by-Side: Base R vs. Tidyverse

Filtering data (keep only certain rows):

```
# BASE R approach (what you know from Chapter 2)
valorant_base <- clean_data[clean_data$game == "Valorant", ]

# TIDYVERSE approach (clearer to read)
valorant_tidy <- clean_data %>%
  filter(game == "Valorant")
```

Creating new columns:

```r
# BASE R
clean_data$is_toxic <- ifelse(clean_data$toxicity == "Toxic", 1, 0)

# TIDYVERSE
clean_data <- clean_data %>%
  mutate(is_toxic = if_else(toxicity == "Toxic", 1, 0))
```

Selecting specific columns:

```r
# BASE R
minimal_base <- clean_data[, c("id", "game", "toxicity")]

# TIDYVERSE
minimal_tidy <- clean_data %>%
  select(id, game, toxicity)
```

Why use tidyverse? As operations get more complex, tidyverse code remains readable. Compare:

```r
# BASE R: calculate average viewers for webcam streamers (hard to read)
mean(clean_data[clean_data$uses_webcam == TRUE, ]$viewers)

# TIDYVERSE: same operation (reads like English)
clean_data %>%
  filter(uses_webcam == TRUE) %>%
  summarize(avg_viewers = mean(viewers))
```

The pipe operator (`%>%`) lets you read code **left-to-right** instead of **inside-out**.

The Five Core `dplyr` Verbs

Now that you see the comparison, let's master the five tidyverse functions you'll use constantly:

1. `filter()`: Keep rows that meet a condition

```r
# Keep only Valorant streams
valorant_data <- clean_data %>%
  filter(game == "Valorant")
```

```r
# Multiple conditions (AND)
competitive_valorant <- clean_data %>%
  filter(game == "Valorant", ranked == TRUE)

# OR conditions
fps_games <- clean_data %>%
  filter(game == "Valorant" | game == "Overwatch")
```

2. `select()`: Keep or drop columns

```r
# Keep only specific columns
minimal_data <- clean_data %>%
  select(id, game, toxicity)

# Drop columns
clean_data %>%
  select(-coder_id, -notes)  # Remove these columns
```

3. `mutate()`: Create or modify columns

```r
# Create a binary toxicity variable
clean_data <- clean_data %>%
  mutate(is_toxic = if_else(toxicity == "Toxic", 1, 0))

# Create multiple columns at once
clean_data <- clean_data %>%
  mutate(
    is_toxic = if_else(toxicity == "Toxic", 1, 0),
    viewer_category = case_when(
      viewers < 100 ~ "Small",
      viewers < 1000 ~ "Medium",
      viewers >= 1000 ~ "Large"
    )
  )
```

4. `group_by()` + `summarize()`: Aggregate data

```r
# Calculate toxicity rate by game
summary_stats <- clean_data %>%
```

```
  group_by(game) %>%
  summarize(
    total_messages = n(),
    toxic_messages = sum(is_toxic),
    toxicity_rate = toxic_messages / total_messages
  )
```

Common mistake: Forgetting to use `group_by()` before `summarize()`. Without grouping, you get one summary for the entire dataset instead of one per group.

5. `arrange()`: Sort rows

```
# Sort games by toxicity rate (descending)
summary_stats %>%
  arrange(desc(toxicity_rate))

# Sort ascending (lowest to highest)
summary_stats %>%
  arrange(toxicity_rate)
```

Chaining Operations Together

The real power comes from **combining verbs**:

```
# Start with raw data, end with a sorted summary table
result <- clean_data %>%
  filter(game %in% c("Valorant", "Overwatch")) %>%  # Only FPS games
  mutate(is_toxic = if_else(toxicity == "Toxic", 1, 0)) %>%  # Create binary
  group_by(game, uses_webcam) %>%  # Group by game AND webcam
  summarize(
    total = n(),
    toxic = sum(is_toxic),
    rate = toxic / total
  ) %>%
  arrange(desc(rate))  # Sort by toxicity rate
```

Read this aloud: "Take clean_data, THEN filter for FPS games, THEN create a toxicity indicator, THEN group by game and webcam use, THEN calculate summaries, THEN sort by rate."

This is why tidyverse exists: **code that reads like instructions**.

Reshaping Data

Pivoting Wider (Long → Wide)

Use case: You have multiple rows per person, and you want one row per person with columns for each variable.

Example: Transform this:

Person	Variable	Value
Alice	Age	22
Alice	Game	Valorant
Bob	Age	25
Bob	Game	Overwatch

Into this:

Person	Age	Game
Alice	22	Valorant
Bob	25	Overwatch

Code:

```
library(tidyr)

wide_data <- long_data %>%
  pivot_wider(names_from = Variable, values_from = Value)
```

Pivoting Longer (Wide → Long)

Use case: You have one row per person with multiple variables as columns, and you want one row per person-variable pair.

Code:

```
long_data <- wide_data %>%
  pivot_longer(cols = c(Age, Game), names_to = "Variable", values_to = "Value")
```

Factors: Taming Categorical Data

Factors are R's way of storing categorical data with defined levels and ordering.

Why Factors Matter

By default, R treats categories as character strings and orders them alphabetically. This is often wrong:

- **Education level**: "High School" < "Bachelor's" < "Master's" < "PhD" (ordinal, not alphabetical)
- **Likert scales**: "Strongly Disagree" < "Disagree" < "Neutral" < "Agree" < "Strongly Agree"

Creating Factors

```
library(forcats)

clean_data <- clean_data %>%
  mutate(game = factor(game, levels = c("Valorant", "Overwatch", "CS:GO")))
```

Now "game" has defined levels. Visualizations and tables will respect this order.

Reordering Factors by Frequency

```
clean_data <- clean_data %>%
  mutate(game = fct_infreq(game))  # Orders by frequency (most common first)
```

Documenting Your Cleaning Steps

Every transformation you make is a **methodological decision**. Document it.

Example cleaning log:

```r
# Data Cleaning Log
# Author: [Your Name]
# Date: 2024-10-15

# 1. Imported raw data
raw_data <- read_csv("data_raw/twitch_chat_coding.csv")

# 2. Standardized game names
clean_data <- raw_data %>%
  mutate(game = tolower(game)) %>%
  mutate(game = case_when(
    game %in% c("valorant", "val") ~ "Valorant",
    game %in% c("overwatch", "ow") ~ "Overwatch",
    TRUE ~ game
  ))

# 3. Converted toxicity to binary
clean_data <- clean_data %>%
  mutate(is_toxic = if_else(toxicity == "Toxic", 1, 0))

# 4. Removed rows with missing game data (n = 12 rows excluded)
clean_data <- clean_data %>%
  filter(!is.na(game))

# 5. Saved cleaned data
write_csv(clean_data, "data_clean/twitch_chat_clean.csv")
```

Case Study: Cleaning a Twitch Chat Dataset

Raw data: 500 chat messages coded for game, message type, and toxicity.

Problem 1: Game names inconsistent ("Valorant" vs. "valorant" vs. "Val").

Solution: Standardize with `tolower()` and `case_when()`.

Problem 2: Toxicity column has values: "Toxic", "Neutral", "Other", "???"

Solution:

- Recode "???" as NA (ambiguous).
- Collapse "Other" into "Neutral" (both non-toxic).

Problem 3: Coder ID is stored as numeric (1, 2) but should be categorical ("Coder A", "Coder B").

Solution: Convert to factor with labels.

Final cleaned dataset:

- 488 rows (12 excluded for missing game data)
- 4 variables: `id`, `game`, `message_type`, `is_toxic`
- Ready for analysis

This is why tidyverse exists: **code that reads like instructions**.

Real Data: American Idol Data Wrangling

Let's apply data wrangling to the American Idol judge feedback dataset (Chapters 8-10).

Scenario: Raw Coding Sheet to Analysis-Ready Data

Raw data: 60 episodes × 3 judges = ~180 judge feedback entries coded in a spreadsheet.

Raw structure (messy):

Episode	Judge	Contestant_Name	Gender	Race_Ethnicity	Feedback_Comment	Tone_Code	Advancement_Decision
S22E01	Katy	Marcus	M	W	"That was great work"	2	Advanced
S22E01	Katy	Taylor	F	B	"lovely performance"	3	Advanced
S22E01	Luke	Marcus	M	W	"Impressive range"	3	Advanced

Problems to solve:

1. **Episode ID confusion**: "S22E01" vs. "Season 22 Ep 1" vs. "22-01"
2. **Tone codes as text**: "2" should be numeric; some cells say "Mildly Positive" instead of "2"
3. **Advancement has text**: "Advanced" vs. "Yes" vs. "1"
4. **Missing data**: Some cells blank, some say "N/A", some say "Unclear"
5. **Contestant ID missing**: Can't match across rows (if a contestant appears in multiple episodes)
6. **Non-tidy format**: Comments stored as text; should be separate from codes

Step 1: Set Up the Cleaning Script

```
library(tidyverse)
library(readr)

# Import raw data
raw_idol <- read_csv("data_raw/idol_raw_coding.csv")

# Inspect
glimpse(raw_idol)
head(raw_idol)
```

Step 2: Fix Episode IDs

```
# Standardize episode codes to "S##E##" format
clean_idol <- raw_idol %>%
  mutate(episode_id = str_replace(Episode, " ", ""),  # Remove spaces
         episode_id = str_replace(episode_id, "-", "E"),  # Convert hyphen
         episode_id = tolower(episode_id))  # Lowercase for consistency

# Check results
clean_idol %>% select(episode_id) %>% distinct()
```

Step 3: Standardize Judge Tone Coding

```
# Convert text descriptions to numeric codes
clean_idol <- clean_idol %>%
  mutate(
    tone = case_when(
      Tone_Code == 3 | Tone_Code == "Strongly Positive" |
        Tone_Code == "3" ~ 3,
      Tone_Code == 2 | Tone_Code == "Mildly Positive" |
        Tone_Code == "2" ~ 2,
      Tone_Code == 1 | Tone_Code == "Neutral" |
        Tone_Code == "Constructive" | Tone_Code == "1" ~ 1,
      Tone_Code == 0 | Tone_Code == "Negative" |
        Tone_Code == "0" ~ 0,
      is.na(Tone_Code) ~ NA_real_,
```

```
      TRUE ~ NA_real_  # Unknown codes become NA
    ),
    tone = as.numeric(tone)
  ) %>%
  select(-Tone_Code)  # Drop old column

# Check
clean_idol %>% count(tone)
```

Step 4: Convert Advancement to Binary

```
clean_idol <- clean_idol %>%
  mutate(
    advancement = case_when(
      Advancement_Decision %in% c("Advanced", "Yes", "1", "Advanved") ~ 1,  # Note typo
      Advancement_Decision %in% c("Eliminated", "No", "0") ~ 0,
      is.na(Advancement_Decision) ~ NA_real_,
      TRUE ~ NA_real_
    ),
    advancement = as.numeric(advancement)
  ) %>%
  select(-Advancement_Decision)

# Check
clean_idol %>% count(advancement)
```

Step 5: Standardize Demographics

```
clean_idol <- clean_idol %>%
  mutate(
    # Standardize gender
    gender = tolower(str_trim(Gender)),  # Lowercase and remove extra spaces
    gender = case_when(
      gender %in% c("m", "male", "man") ~ "Male",
      gender %in% c("f", "female", "woman") ~ "Female",
      gender %in% c("nb", "non-binary", "other") ~ "Non-Binary",
      TRUE ~ NA_character_
    ),
```

```r
    # Standardize race/ethnicity
    ethnicity = tolower(str_trim(Race_Ethnicity)),
    ethnicity = case_when(
      ethnicity %in% c("w", "white", "caucasian") ~ "White",
      ethnicity %in% c("b", "black", "african american") ~ "Black",
      ethnicity %in% c("l", "latinx", "hispanic", "latin") ~ "Latinx",
      ethnicity %in% c("a", "asian", "asian american") ~ "Asian",
      ethnicity %in% c("m", "middle eastern", "mena") ~ "Middle Eastern",
      ethnicity %in% c("r", "multiracial", "mixed") ~ "Multiracial",
      TRUE ~ NA_character_
    )
  ) %>%
  select(-Gender, -Race_Ethnicity)

# Check
clean_idol %>% count(gender, ethnicity)
```

Step 6: Handle Missing Data

```r
# Document missingness before excluding
missing_summary <- clean_idol %>%
  summarize(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing_Count")

print(missing_summary)

# Decision: For tone and advancement, missing = ambiguous coding
# Exclude these rows (they represent coder uncertainty)
clean_idol <- clean_idol %>%
  filter(!is.na(tone) & !is.na(advancement))

# For gender/ethnicity, missing is acceptable (some contestants not identified)
# Keep these as NA and handle in analysis (use drop_na() only when needed)
```

Step 7: Add Judge as Factor

```r
clean_idol <- clean_idol %>%
  mutate(judge = factor(Judge,
```

```
                          levels = c("Katy", "Luke", "Lionel"),
                          labels = c("Katy Perry", "Luke Bryan", "Lionel Richie")))
```

Step 8: Create Analysis Variables

```
clean_idol <- clean_idol %>%
  mutate(
    # Binary tone indicator: positive (2-3) vs. non-positive (0-1)
    tone_positive = if_else(tone >= 2, 1, 0),

    # Judge-specific indicators
    judge_name = as.character(judge)
  ) %>%
  select(episode_id, judge, judge_name, Contestant_Name,
         gender, ethnicity, tone, tone_positive, advancement)
```

Step 9: Remove Duplicates

```
# Check for exact duplicates
clean_idol %>%
  duplicated() %>%
  sum()

# If duplicates found, investigate:
# clean_idol %>% get_dupes(episode_id, judge, Contestant_Name)

# Remove duplicates (keep first occurrence)
clean_idol <- clean_idol %>%
  distinct(episode_id, judge, Contestant_Name, .keep_all = TRUE)
```

Step 10: Save Cleaned Data

```
write_csv(clean_idol, "data_clean/idol_judge_feedback.csv")

# Final summary
glimpse(clean_idol)
nrow(clean_idol)  # Should be ~180, minus rows with missing tone/advancement
```

Step 11: Create Data Dictionary

```
# Document your dataset for reproducibility
data_dictionary <- tibble(
  Variable = names(clean_idol),
  Type = map_chr(clean_idol, class),
  Description = c(
    "Episode ID (Season##Episode## format)",
    "Judge (factor: Katy, Luke, Lionel)",
    "Judge full name",
    "Contestant name",
    "Contestant gender (Male/Female/Non-Binary)",
    "Contestant race/ethnicity",
    "Judge feedback tone (0-3 ordinal scale)",
    "Binary positive tone (1 = tone >= 2)",
    "Did contestant advance? (1 = Yes, 0 = No)"
  )
)

write_csv(data_dictionary, "data_clean/idol_data_dictionary.csv")
```

Final cleaned dataset: - **Rows**: 174 (6 excluded for missing tone or advancement) - **Columns**: 9 variables, analysis-ready - **Data types**: Episode ID (character), Judge (factor), Demographics (character), Tone (numeric), Advancement (numeric)

Practice: Data Wrangling in R

Exercise 11.1: Tidy Data Diagnosis

For each dataset structure, identify whether it's **tidy** or **messy**. If messy, explain what needs to change:

Dataset A: | PersonID | Q1_Response | Q2_Response | Q3_Response | |———-|————-|————-|————-| | 1 | Agree | Disagree | Neutral | | 2 | Disagree | Agree | Agree |

- **Tidy or Messy?**: _____
- **Problem (if any)**: _____
- **How to tidy**: _____

Dataset B: | Tweet_ID | Sentiment_Emotion | |———-|————————-| | 001 | Positive_Joy | | 002 | Negative_Anger |

- **Tidy or Messy?**: _____
- **Problem (if any)**: _____
- **How to tidy**: _____

Dataset C: | Streamer | Date | Viewers | |———-|——|———| | StreamerA | 2024-01-15 | 1250 | | StreamerA | 2024-01-16 | 1340 |

- **Tidy or Messy?**: _____
- **Why or why not?**: _____

Exercise 11.2: dplyr Practice

Write the R code (or pseudocode if you're still learning) for these data manipulations:

Given this dataset: `twitch_data` with variables `streamer`, `game`, `viewers`, `toxicity`

Task 1: Keep only streams with more than 1000 viewers

```
# Your code:
```

Task 2: Create a new variable `high_toxicity` (1 if toxicity > 10%, 0 otherwise)

```
# Your code:
```

Task 3: Calculate average viewers by game

```
# Your code:
```

Task 4: Find the top 5 games by average viewers

```
# Your code:
```

Exercise 11.3: Handling Missing Data

You have a dataset with 200 rows. Variable `donation_amount` has 30 NA values.

Strategy options: A) Delete all 30 rows with missing donations (listwise deletion) B) Replace NAs with 0 (assuming no donation = $0) C) Replace NAs with the median donation amount (imputation) D) Keep NAs and note them in analysis

For each scenario, choose the best strategy and justify:

Scenario 1: You're calculating "what percentage of viewers donate?" NAs represent viewers who didn't donate. - **Best strategy**: _____ - **Justification**: _____

Scenario 2: You're calculating "average donation amount among those who donated." NAs represent missing data (tech error). - **Best strategy**: _____ - **Justification**: _____

Scenario 3: NAs are randomly distributed and you're unsure why they're missing. - **Best strategy**: _____ - **Justification**: _____

Exercise 11.4: Data Cleaning Log

Document your cleaning decisions for a hypothetical dataset:

Raw data file: `raw_survey_data.csv` **Observations**: 300 **Variables**: `id`, `platform_use`, `age`, `satisfaction`

Problems discovered: 1. Platform names inconsistent ("YouTube", "youtube", "YT", "Youtube") 2. Age column has text entries: "25", "thirty", "prefer not to say" 3. 15 duplicate responses (same ID, same timestamp) 4. Satisfaction scale mislabeled (1-5 should be 1-7)

Your cleaning log (document each decision):

Problem 1: _____ **Solution**: _____ **R code**: _____

Problem 2: _____ **Solution**: _____ **R code**: _____

Problem 3: _____ **Solution**: _____ **R code**: _____

Problem 4: _____ **Solution**: _____ **R code**: _____

Reflection Questions

1. **The Cleaning Iceberg**: It's often said that 80% of data analysis time is spent on cleaning. Why is this invisible work so time-consuming? How does documenting your cleaning protect against accusations of data manipulation?
2. **Judgment Calls**: When you decide to exclude rows with missing data or recode ambiguous categories, you're making analytical choices that affect results. How do you balance cleaning pragmatism with intellectual honesty? When should you report sensitivity analyses ("results with vs. without excluded cases")?
3. **Reproducibility Check**: Imagine someone emails you six months after your project: "Your data had an error—one coder accidentally swapped 'Positive' and 'Negative' for 20 cases." If your analysis is fully scripted, how quickly can you fix this and regenerate results? What if you'd used point-and-click software?

Chapter Summary

This chapter taught you to transform messy data into analysis-ready datasets:

- **Tidy data** principle: each row = one observation, each column = one variable, each cell = one value
- **Importing data**: Use `readr::read_csv()` for better type detection and column name handling
- Common data problems: **missing values** (NA, blanks), **inconsistent categories** (spelling variations), **duplicates**, **incorrect data types**
- **dplyr verbs**: `filter()` (keep rows), `select()` (keep columns), `mutate()` (create/modify columns), `group_by() + summarize()` (aggregate), `arrange()` (sort)
- **Reshaping**: `pivot_wider()` (long to wide), `pivot_longer()` (wide to long)
- **Factors**: Categorical data with defined levels and ordering; use `forcats` package for manipulation
- **Document every decision**: Cleaning logs ensure reproducibility and defend against accusations of p-hacking
- **Missing data strategies**: Listwise deletion, imputation, keeping NAs (depends on context and reasons for missingness)

Key Terms

- **Cleaning log**: Documentation of all data transformations and decisions
- **dplyr**: R package providing verbs for data manipulation
- **Factor**: R data structure for categorical variables with defined levels

- **Imputation**: Filling in missing values with estimated values (mean, median, model-based)
- **Listwise deletion**: Removing any observation with missing data on any variable
- **Long format**: Each row represents one measurement (tidy for analysis)
- **Missing data**: Absent values (NA in R) requiring handling decisions
- **Mutate**: dplyr verb for creating or modifying variables
- **Pairwise deletion**: Using all available data for each calculation (missing data for some variables doesn't exclude observation entirely)
- **Piping**: Chaining operations with `%>%` operator
- **Pivot**: Reshaping data between long and wide formats
- **Tidy data**: Data structure where each row = observation, column = variable, cell = value
- **Wide format**: Each row represents one entity with multiple measurement columns

Looking Ahead

Chapter 12 shifts from data preparation to **data visualization**—telling your data's story through charts and descriptive statistics. You'll learn the grammar of graphics (ggplot2), create publication-quality visualizations, generate frequency tables, and apply principles of effective visual communication.

Visualizing the Narrative

Learning Objectives

- Create clear, publication-quality visualizations using `ggplot2`.
- Generate frequency tables and descriptive statistics.
- Use data visualization to answer descriptive research questions.
- Apply principles of effective visual communication.

Overview

This chapter introduces **descriptive statistics** and **data visualization**. After 11 weeks of setup, you finally get to see what your data says. This is the first payoff moment—the reveal of patterns you've worked systematically to uncover.

The Grammar of Graphics

`ggplot2` is built on a consistent logic called the **Grammar of Graphics** (Wilkinson, 2005). Before jumping into complex charts, let's build one **step-by-step** so you understand each piece.

Building Your First ggplot2 Chart (Step-by-Step)

Goal: Create a bar chart showing the count of toxic vs. neutral messages.

Step 1: Start with data

```
library(ggplot2)

# Just the data layer (nothing plots yet)
ggplot(data = clean_data)
```

This creates a blank canvas. You've told ggplot "use this dataset," but not what to plot.

Step 2: Add aesthetic mappings (aes)

```
ggplot(data = clean_data, aes(x = message_type))
```

Now ggplot knows "put message_type on the x-axis," but you still haven't said **how** to visualize it (bars? points? lines?).

Step 3: Add a geometry layer

```
ggplot(data = clean_data, aes(x = message_type)) +
  geom_bar()
```

Now you have a chart! The `+` adds layers (like Photoshop layers). `geom_bar()` says "draw bars."

Step 4: Customize with labels and colors

```
ggplot(clean_data, aes(x = message_type)) +
  geom_bar(fill = "steelblue") +
  labs(
    title = "Distribution of Message Types in Twitch Chat",
    x = "Message Type",
    y = "Count"
  )
```

Final result: A clean, labeled bar chart ready for publication.

The Grammar of Graphics Formula

Every ggplot2 chart follows this template:

```
ggplot(data = YOUR_DATA, aes(x = X_VARIABLE, y = Y_VARIABLE)) +
  geom_CHART_TYPE() +
  labs(title = "TITLE", x = "X LABEL", y = "Y LABEL")
```

Components:

- **data**: Your cleaned dataset (from Chapter 11)
- **aes()**: Aesthetic mappings (which variables map to x, y, color, size)
- **geom_*()**: Geometry (bar, line, point, histogram, boxplot)

- **labs()**: Labels and titles
- **+ (plus sign)**: Adds layers (NOT the pipe `%>%`)

Common beginner mistake: Using `%>%` instead of `+` with ggplot2. Remember: - `%>%` for dplyr (data manipulation) - `+` for ggplot2 (visualization)

Why Layering Matters

Think of `ggplot2` as building a sandwich:

1. **Bread (data)**: `ggplot(clean_data)`
2. **Filling (aesthetics)**: `aes(x = game, y = viewers)`
3. **Toppings (geometry)**: `+ geom_col()`
4. **Garnish (labels)**: `+ labs(title = "...")`

Each `+` adds another layer. You can stack as many as needed:

```
ggplot(clean_data, aes(x = game, y = viewers)) +
  geom_col(fill = "coral") +
  labs(title = "Average Viewers by Game", x = "Game", y = "Viewers") +
  theme_minimal() +  # Clean background
  coord_flip()  # Horizontal bars
```

Visualization as Storytelling

Connecting back to Chapter 1, a chart is a **visual story**:

- The **X-axis** is the setting (context: which groups or categories?).
- The **Y-axis** is the plot (what changes or varies?).
- The **bars/points** are the characters (your data).

Your job: make the story clear, not cluttered.

Bad visualization: 12 colors, overlapping labels, no title.
Good visualization: One focal point, clear labels, interpretable at a glance.

Common Visualizations

Bar Chart (Categorical Data)

Use for comparing frequencies or proportions across categories.

Example: How many messages were coded as toxic vs. neutral?

```
library(ggplot2)

ggplot(clean_data, aes(x = message_type)) +
  geom_bar(fill = "steelblue") +
  labs(
    title = "Distribution of Message Types in Twitch Chat",
    x = "Message Type",
    y = "Count"
  )
```

Interpretation: If the "Toxic" bar is much shorter than "Neutral," toxicity is relatively rare.

Grouped Bar Chart

Compare categories across groups.

Example: Toxicity by game (Valorant vs. Overwatch).

```
ggplot(clean_data, aes(x = game, fill = message_type)) +
  geom_bar(position = "dodge") +  # Side-by-side bars
  labs(
    title = "Message Types by Game",
    x = "Game",
    y = "Count",
    fill = "Message Type"
  ) +
  scale_fill_manual(values = c("Toxic" = "red", "Neutral" = "gray"))
```

Interpretation: If Valorant's "Toxic" bar is taller than Overwatch's, Valorant chat is more toxic.

Proportional (Stacked) Bar Chart

Show proportions within each category.

```
ggplot(clean_data, aes(x = game, fill = message_type)) +
  geom_bar(position = "fill") +  # Stacked as proportions
  labs(
    title = "Proportion of Toxic vs. Neutral Messages by Game",
    x = "Game",
    y = "Proportion",
    fill = "Message Type"
  )
```

Interpretation: If Valorant's red slice is larger than Overwatch's, a higher proportion of Valorant messages are toxic (even if the total count is similar).

Histogram (Continuous Data)

Show the distribution of a numeric variable.

Example: Distribution of streamer follower counts.

```
ggplot(streamer_data, aes(x = followers)) +
  geom_histogram(bins = 30, fill = "purple", color = "black") +
  labs(
    title = "Distribution of Follower Counts",
    x = "Followers",
    y = "Frequency"
  )
```

Interpretation: If the histogram is right-skewed, most streamers have few followers, with a long tail of popular streamers.

Boxplot

Visualize the spread and outliers of a numeric variable across groups.

Example: Donations per stream by game.

```r
ggplot(streamer_data, aes(x = game, y = donations)) +
  geom_boxplot(fill = "lightblue") +
  labs(
    title = "Donation Amounts by Game",
    x = "Game",
    y = "Donations ($)"
  )
```

Interpretation: The median (middle line) shows typical donations. Outliers (dots) show extreme values.

Frequency Tables

Before visualizing, **tabulate** your data. Frequency tables answer: "How many observations fall into each category?"

Simple Frequency Table

```r
table(clean_data$message_type)
```

Output:

```
Toxic  Neutral
 120     380
```

Interpretation: 120 messages were toxic, 380 were neutral.

Two-Way Frequency Table (Crosstab)

```r
table(clean_data$game, clean_data$message_type)
```

Output:

```
           Toxic  Neutral
Valorant    80     170
Overwatch   40     210
```

Interpretation: Valorant has twice as many toxic messages as Overwatch (80 vs. 40).

Proportions

```r
prop.table(table(clean_data$game, clean_data$message_type), margin = 1)
```

Output (row proportions):

```
           Toxic  Neutral
Valorant   0.32   0.68
Overwatch  0.16   0.84
```

Interpretation: 32% of Valorant messages are toxic vs. 16% of Overwatch messages.

Descriptive Statistics

Central Tendency

Mean: Average (sensitive to outliers).

```r
mean(streamer_data$followers)
```

Median: Middle value (robust to outliers).

```r
median(streamer_data$followers)
```

When to use:

- **Mean**: Symmetric distributions (normal-ish).
- **Median**: Skewed distributions (income, follower counts).

Spread

Standard Deviation: Average distance from the mean.

```r
sd(streamer_data$followers)
```

Range: Min to max.

```r
range(streamer_data$followers)
```

Interquartile Range (IQR): Middle 50% of data (25th to 75th percentile).

```r
IQR(streamer_data$followers)
```

Combining Tables and Visualizations

Use `dplyr` to create summary tables, then visualize.

Example: Calculate toxicity rate by game, then visualize.

```r
library(dplyr)

summary_data <- clean_data %>%
  group_by(game) %>%
  summarize(
    total_messages = n(),
    toxic_messages = sum(message_type == "Toxic"),
    toxicity_rate = toxic_messages / total_messages
  )

# Visualize
ggplot(summary_data, aes(x = game, y = toxicity_rate)) +
  geom_col(fill = "coral") +
  labs(
    title = "Toxicity Rate by Game",
    x = "Game",
    y = "Toxicity Rate (Proportion)"
  ) +
  scale_y_continuous(labels = scales::percent)  # Format as percentages
```

Principles of Effective Visualization

Clarity Over Decoration

Avoid:

- 3D effects (distort perception)
- Excessive colors (confusing)

- Unlabeled axes (audience has to guess)

Embrace:

- Minimalism (remove non-data ink)
- Consistent color schemes (e.g., red = toxic, blue = neutral)
- Clear titles and labels

Accessibility

Color blindness: Avoid red-green combinations. Use colorblind-friendly palettes (e.g., `viridis` package).

```
library(viridis)

ggplot(clean_data, aes(x = game, fill = message_type)) +
  geom_bar(position = "dodge") +
  scale_fill_viridis(discrete = TRUE)  # Colorblind-safe palette
```

Text size: Make labels large enough to read (minimum 10pt).

```
theme_set(theme_minimal(base_size = 14))  # Increase base font size
```

Context

Always include:

- **Title**: What does this chart show?
- **Axis labels**: What are the units?
- **Legend**: What do colors/shapes represent?
- **Caption** (optional): Data source, sample size, notes.

Answering Descriptive Research Questions

Recall Chapter 7: Descriptive RQs ask "What?" or "How much?"

Example RQs and corresponding visualizations:

RQ1: "What proportion of Twitch chat messages during Valorant streams contain toxic language?"

- **Answer**: Frequency table + bar chart. Result: 32% of messages are toxic.

RQ2: "How do toxicity rates compare between competitive and casual games?"

- **Answer**: Grouped bar chart or proportional bar chart comparing the two contexts.

RQ3: "What is the distribution of streamer follower counts?"

- **Answer**: Histogram showing right-skewed distribution (most streamers have <10,000 followers, a few have >100,000).

Real Data: American Idol Judge Feedback Visualization

Let's visualize the cleaned American Idol dataset (from Chapter 11).

Research Questions:

1. Do judges give more positive feedback to contestants from certain demographic backgrounds?
2. Does positive feedback predict advancement?

Question 1: Judge Tone by Contestant Demographics

Data prep (in R):

```
library(tidyverse)
library(ggplot2)

# Load cleaned data
idol_data <- read_csv("data_clean/idol_judge_feedback.csv")

# Summarize: Average judge tone by contestant gender
tone_by_gender <- idol_data %>%
  group_by(gender) %>%
  summarize(
    mean_tone = mean(tone, na.rm = TRUE),
    sd_tone = sd(tone, na.rm = TRUE),
    n = n()
  )

print(tone_by_gender)
```

Expected output (example):

```
gender       mean_tone  sd_tone    n
Female          2.18    0.73     54
Male            2.35    0.68     91
Non-Binary      2.12    0.82     12
```

Visualization 1: Judge Tone by Gender

```
ggplot(tone_by_gender, aes(x = gender, y = mean_tone, fill = gender)) +
  geom_col() +
  geom_errorbar(aes(ymin = mean_tone - sd_tone,
                    ymax = mean_tone + sd_tone),
                width = 0.2) +
  labs(
    title = "Average Judge Feedback Tone by Contestant Gender",
    x = "Contestant Gender",
    y = "Average Tone Score (0 = Negative, 3 = Strongly Positive)",
    fill = "Gender"
  ) +
  ylim(0, 3) +
  theme_minimal() +
  theme(legend.position = "none")
```

Interpretation: - Male contestants receive slightly higher average tone (M = 2.35) vs. female (M = 2.18) - Difference is small (~0.17 points on 0-3 scale) - Error bars show substantial variability within each gender group

Visualization 2: Judge Tone by Race/Ethnicity

```
tone_by_ethnicity <- idol_data %>%
  group_by(ethnicity) %>%
  summarize(
    mean_tone = mean(tone, na.rm = TRUE),
    n = n()
  ) %>%
  filter(!is.na(ethnicity))  # Remove missing data

ggplot(tone_by_ethnicity, aes(x = reorder(ethnicity, mean_tone),
                              y = mean_tone,
                              fill = mean_tone)) +
  geom_col() +
  scale_fill_viridis_c(option = "plasma") +
  labs(
```

```
    title = "Average Judge Feedback Tone by Contestant Race/Ethnicity",
    x = "Race/Ethnicity",
    y = "Average Tone Score",
    fill = "Avg Tone"
  ) +
  coord_flip() +  # Horizontal bars for readability
  ylim(0, 3) +
  theme_minimal()
```

Interpretation: - Visual inspection shows tone scores vary by ethnicity (ranging ~2.0-2.4) - Horizontally-oriented chart makes category labels easier to read - Viridis color palette is colorblind-friendly

Question 2: Does Positive Feedback Predict Advancement?

Data prep:

```
# Create contingency table
advancement_table <- idol_data %>%
  mutate(tone_category = if_else(tone >= 2, "Positive", "Non-Positive")) %>%
  group_by(tone_category, advancement) %>%
  summarize(count = n())

print(advancement_table)
```

Expected output:

```
tone_category    advancement    count
Non-Positive          No           38
Non-Positive          Yes          15
Positive              No           21
Positive              Yes          82
```

Visualization 3: Advancement by Judge Feedback Tone

```
ggplot(advancement_table, aes(x = tone_category, y = count, fill = factor(advancement))) +
  geom_col(position = "dodge") +
  labs(
    title = "Contestant Advancement by Judge Feedback Tone",
    x = "Judge Feedback Tone",
```

```
    y = "Number of Contestants",
    fill = "Advanced?"
  ) +
  scale_fill_manual(values = c("0" = "lightcoral", "1" = "lightgreen"),
                    labels = c("0" = "Eliminated", "1" = "Advanced")) +
  theme_minimal()
```

Interpretation: - Contestants receiving positive feedback: 82 advanced, 21 eliminated (79.6% advancement rate) - Contestants receiving non-positive feedback: 15 advanced, 38 eliminated (28.3% advancement rate) - Strong relationship: positive feedback → higher advancement probability

Visualization 4: Judge-Specific Patterns

```
judge_patterns <- idol_data %>%
  filter(!is.na(gender)) %>%
  group_by(judge_name, gender) %>%
  summarize(
    mean_tone = mean(tone, na.rm = TRUE),
    n = n()
  )

ggplot(judge_patterns, aes(x = judge_name, y = mean_tone, fill = gender)) +
  geom_col(position = "dodge") +
  labs(
    title = "Judge Feedback Tone by Contestant Gender (by Judge)",
    x = "Judge",
    y = "Average Tone Score",
    fill = "Contestant Gender"
  ) +
  ylim(0, 3) +
  scale_fill_manual(values = c("Male" = "#1f77b4", "Female" = "#ff7f0e")) +
  theme_minimal()
```

Interpretation: - Compare across judges: Do some judges give systematically higher/lower scores? - Compare within judges: Does each judge's feedback differ by contestant gender? - Useful for exploring potential bias patterns

Descriptive Statistics Summary

```r
# Overall summary
idol_summary <- idol_data %>%
  summarize(
    Total_Codings = n(),
    Avg_Judge_Tone = mean(tone, na.rm = TRUE),
    SD_Judge_Tone = sd(tone, na.rm = TRUE),
    Advancement_Rate = mean(advancement, na.rm = TRUE),
    Positive_Feedback_Pct = mean(tone_positive, na.rm = TRUE) * 100
  )

print(idol_summary)
```

Output example:

```
Total_Codings: 174
Avg_Judge_Tone: 2.18 (on 0-3 scale)
SD_Judge_Tone: 0.79
Advancement_Rate: 65.5% (114/174 advanced)
Positive_Feedback_Pct: 68.4%
```

Case Study: Visualizing Twitch Toxicity Data

Dataset: 500 chat messages from 50 Twitch streams (10 messages per stream).

Variables:

- `game`: Valorant, Overwatch, CS:GO
- `message_type`: Toxic, Neutral
- `streamer_gender`: Male, Female

Goal: Explore toxicity patterns.

Step 1: Frequency Table

```r
table(clean_data$game, clean_data$message_type)
```

Finding: Valorant has the most toxic messages (80/250 = 32%).

Step 2: Visualize Toxicity by Game

```
ggplot(clean_data, aes(x = game, fill = message_type)) +
  geom_bar(position = "fill") +
  labs(
    title = "Toxicity Rates by Game",
    x = "Game",
    y = "Proportion",
    fill = "Message Type"
  ) +
  scale_fill_manual(values = c("Toxic" = "darkred", "Neutral" = "lightgray"))
```

Interpretation: Valorant has the highest toxicity rate (32%), followed by CS:GO (24%) and Overwatch (16%).

Step 3: Explore Gender Differences

```
summary_gender <- clean_data %>%
  group_by(streamer_gender, message_type) %>%
  summarize(count = n()) %>%
  group_by(streamer_gender) %>%
  mutate(proportion = count / sum(count))

ggplot(summary_gender, aes(x = streamer_gender, y = proportion, fill = message_type)) +
  geom_col(position = "dodge") +
  labs(
    title = "Toxicity by Streamer Gender",
    x = "Streamer Gender",
    y = "Proportion",
    fill = "Message Type"
  )
```

Interpretation: Female streamers' chats have a higher proportion of toxic messages (hypothesis: gendered harassment).

Practice: Creating Effective Visualizations

Exercise 12.1: Chart Type Selection

For each research question, select the MOST appropriate visualization type:

Chart types: Bar chart, Grouped bar chart, Stacked bar chart, Histogram, Boxplot, Line graph, Scatter plot

RQ1: "What is the distribution of follower counts among 200 streamers?" - **Best chart**: _____ - **Why**: _____

RQ2: "Do competitive games have higher toxicity rates than casual games?" - **Best chart**: _____ - **Why**: _____

RQ3: "How has esports news coverage changed from 2020 to 2024 (articles per month)?" - **Best chart**: _____ - **Why**: _____

RQ4: "Is there a relationship between stream duration and viewer count?" - **Best chart**: _____ - **Why**: _____

Exercise 12.2: Interpreting Descriptive Statistics

Given this summary for streamer donations (n=100): - Mean: $45.50 - Median: $12.00 - SD: $89.30 - Range: $0 - $500 - IQR: $5 - $35

Questions: 1. Which measure of central tendency (mean or median) better represents "typical" donations? Why? - **Answer**: _____

2. What does the large difference between mean and median tell you about the distribution?
   - **Answer**: _____
3. Why is the standard deviation so large relative to the mean?
   - **Answer**: _____
4. If you were to report one number as "typical donation amount," which would you choose? Why?
   - **Answer**: _____

Exercise 12.3: Building a Frequency Table

You coded 150 YouTube video titles for emotional appeal. Results: - Fear: 35 - Humor: 48 - Inspiration: 22 - Curiosity: 28 - Anger: 17

Create a frequency table with percentages:

Emotional Appeal	Frequency	Percentage
Fear	35	___%
Humor	48	___%
Inspiration	22	___%
Curiosity	28	___%
Anger	17	___%
Total	150	100%

Interpretation questions: 1. Which emotional appeal is most common? _____
2. What percentage of videos use negative emotions (Fear + Anger)? _____
3. Write one sentence summarizing this table for a Results section: _____

Exercise 12.4: Improving Visualizations

Critique and improve this poorly designed chart description:

Bad Chart: A pie chart with 12 slices showing game genres, using 12 different colors including multiple shades of green and blue that look similar. No labels on slices, only a legend. Title: "Data"

Problems identified: 1. _____ 2. _____ 3. _____

How to improve: 1. **Chart type**: Switch to _____ 2. **Color scheme**: _____ 3. **Labels**: _____ 4. **Title**: _____

Exercise 12.5: ggplot2 Practice (Pseudocode)

Write the ggplot2 code (or describe what each layer does) for this visualization:

Goal: Create a bar chart showing the count of news articles by source (NYT, WSJ, ESPN) with bars colored by source.

```
# Your code:
ggplot(data = ______________, aes(x = ______________, fill = ______________)) +
  geom_______() +
  labs(
    title = \"______________\",
    x = \"______________\",
    y = \"______________\"
  ) +
  theme_________()
```

Reflection Questions

1. **The Lie Factor**: Misleading visualizations can distort perception (e.g., y-axis starting at 50 instead of 0 makes small differences look huge). Have you encountered misleading data visualizations in news or social media? What made them deceptive?
2. **Accessibility Trade-offs**: Making visualizations accessible (colorblind-friendly palettes, alt-text, high contrast) sometimes reduces aesthetic appeal. How do you balance beauty with accessibility? Should research prioritize one over the other?
3. **Visualization as Argument**: Your choice of chart type, color, and axis scaling tells a story. How do you ensure you're using visualization to *clarify* rather than *manipulate*? What ethical obligations do data visualizers have?

Chapter Summary

This chapter introduced descriptive statistics and data visualization as storytelling tools:

- **ggplot2** uses the Grammar of Graphics: data + aesthetic mappings (aes) + geometry (geom) + labels
- Common visualizations: **bar charts** (categorical counts), **grouped/stacked bars** (comparing categories across groups), **histograms** (continuous distributions), **boxplots** (spread and outliers)
- **Frequency tables** answer "how many in each category?" and support crosstabs (two-way tables)
- **Descriptive statistics**: Central tendency (**mean**, **median**, **mode**) and spread (**SD**, **range**, **IQR**)
- **Mean vs. median**: Use median for skewed distributions (outliers), mean for symmetric distributions
- Visualizations are **stories**: x-axis = setting, y-axis = plot, data points = characters

- **Effective visualization principles**: Clarity over decoration, minimize non-data ink, use accessible color palettes, make interpretations easy
- Combine **dplyr** (data summarization) with **ggplot2** (visualization) for powerful analysis pipelines

Key Terms

- **Bar chart**: Visualization for comparing counts or proportions across categories
- **Boxplot**: Visualization showing median, quartiles, and outliers (five-number summary)
- **Central tendency**: Typical or average value (mean, median, mode)
- **Frequency table**: Count of observations in each category
- **geom**: Geometry layer in ggplot2 (bar, line, point, histogram, etc.)
- **ggplot2**: R package for data visualization using Grammar of Graphics
- **Grammar of Graphics**: Framework treating visualizations as layers (data, aesthetics, geometry)
- **Histogram**: Visualization showing distribution of continuous variable
- **IQR (Interquartile Range)**: Middle 50% of data (Q3 - Q1)
- **Mean**: Arithmetic average (sum/n), sensitive to outliers
- **Median**: Middle value when sorted, robust to outliers
- **Standard deviation (SD)**: Average distance from the mean
- **Spread**: Variability in data (range, SD, IQR)

Looking Ahead

Chapter 13 moves from description to **inference**—using your sample data to make claims about the broader population. You'll learn the logic of hypothesis testing, interpret p-values, conduct Chi-Square tests of independence, and distinguish between statistical and practical significance.

Making the Call

Learning Objectives

- Understand the logic of inferential statistics and hypothesis testing.
- Interpret p-values and the null hypothesis.
- Select the appropriate inferential test for different research questions.
- Conduct Chi-Square tests, t-tests, ANOVA, and correlation analyses in R.
- Calculate and interpret effect sizes for each test.
- Recognize the difference between statistical significance and practical significance.

Overview

Descriptive statistics (Chapter 12) summarize your **sample**. Inferential statistics answer a bigger question: **Do the patterns in my sample likely exist in the broader population, or are they just random noise?**

This chapter introduces the logic of hypothesis testing and several foundational inferential methods:

- **Chi-Square test**: For associations between two categorical variables
- **t-test**: For comparing means between two groups
- **ANOVA**: For comparing means across three or more groups
- **Correlation**: For measuring strength of linear relationships between continuous variables

Each method applies to different research questions and data structures. By the end, you'll understand when to use each and how to interpret results.

Before you start: This chapter assumes you understand basic probability concepts. If terms like "probability," "random chance," or "sampling distribution" are unfamiliar, read the "Statistical Foundations Primer" section below before proceeding.

Statistical Foundations Primer

If you've never taken statistics, start here. These concepts underpin all inferential tests.

Population vs. Sample

- **Population**: Everyone/everything you want to make claims about (e.g., "all Twitch chat messages")
- **Sample**: The subset you actually study (e.g., "200 messages from 10 streams")

The problem: You can't study the entire population (too large, too expensive). So you study a sample and **infer** whether patterns hold for the population.

Random Chance and Sampling Variability

Imagine flipping a fair coin 10 times. You might get 6 heads and 4 tails—not exactly 50/50. That's **random chance**.

Now imagine you sample 100 Twitch messages and find 30% are toxic. If you took a different sample of 100 messages, you might get 28% or 33%. That variation is called **sampling variability**.

Key insight: Even if there's NO real difference in toxicity between games, random sampling might make it LOOK like there's a difference. Inferential statistics help you decide: "Is this difference real, or just random noise?"

What Is a P-Value? (Simple Version)

The p-value answers: **"If there's really no difference (null hypothesis is true), how often would I see a difference THIS big just by random chance?"**

- **p = 0.03**: "Only 3% of the time would random chance produce this result. Probably not random!"
- **p = 0.45**: "45% of the time random chance could produce this. Probably just noise."

The threshold: By convention, if $p < 0.05$ (less than 5%), we say the result is **statistically significant**—unlikely to be random.

Probability Refresher

If you flip a fair coin once: - Probability of heads = 0.5 (50%) - Probability of tails = 0.5 (50%)

If you roll a fair six-sided die: - Probability of rolling a 3 = 1/6 0.167 (16.7%)

Independence: The coin doesn't "remember" previous flips. Each flip has the same 50% chance.

Why this matters: Inferential tests assume your samples are independent (one observation doesn't influence another). If you violate this (e.g., coding the same streamer's chat 20 times), your tests are invalid.

The Logic of Inference

The Null Hypothesis (H_0)

The null hypothesis is the assumption that "nothing is happening."

Example: H_0: "There is no difference in toxicity rates between Valorant and Overwatch chat."

Your job as a researcher: try to prove the null wrong.

The Alternative Hypothesis (H_1 or H_A)

The alternative is what you suspect is true.

Example: H_1: "Valorant chat has a higher rate of toxic messages than Overwatch chat."

The P-Value: A "Surprisal Score"

The p-value is **not** the probability that your hypothesis is true. It's the probability of observing your data (or something more extreme) **if the null hypothesis were true**.

- $\mathbf{p < 0.05}$: The data is surprising under the null. We reject H_0 (tentatively accept H_1).
- $\mathbf{p \quad 0.05}$: The data is not surprising under the null. We fail to reject H_0 (we don't "accept" it; we just lack evidence against it).

Analogy: You're a detective. The null hypothesis is "the suspect is innocent." The p-value tells you how surprising the evidence is if the suspect is innocent. If it's very surprising ($p < 0.05$), you have grounds to charge them (reject the null). If it's not surprising, you don't have a case.

Type I and Type II Errors

Type I Error (False Positive): Rejecting the null when it's actually true.

- Example: You conclude Valorant is more toxic, but the difference was just sampling error.
- Probability = (significance level, typically 0.05).

Type II Error (False Negative): Failing to reject the null when it's actually false.

- Example: Valorant IS more toxic, but your sample was too small to detect it.
- Probability = (depends on sample size and effect size).

The trade-off: Lower (stricter threshold) reduces false positives but increases false negatives. Science conventionally accepts 5% Type I error rate (= 0.05).

Exposing the Sacred Flaw

Recall Chapter 1: the null hypothesis is the "sacred flaw"—the universe's current story. Rejecting the null means you've forced the universe to admit its story was wrong and adopt a new one (your alternative hypothesis). This is the climax of your research narrative.

The Chi-Square Test of Independence

When to Use Chi-Square

Use Chi-Square when you have **two categorical variables** and want to test whether they're associated.

Example research questions:

- Is there a relationship between game type (Valorant vs. Overwatch) and message toxicity (Toxic vs. Neutral)?
- Is there a relationship between streamer gender (Male vs. Female) and chat toxicity?

The Null Hypothesis for Chi-Square

H_0: The two variables are independent (no relationship).

H_1: The two variables are associated (there IS a relationship).

How Chi-Square Works

Chi-Square compares **observed frequencies** (what you actually counted) to **expected frequencies** (what you'd expect if the null were true).

Formula:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Where:

- O = observed frequency
- E = expected frequency (calculated assuming independence)

If observed and expected are very different, χ^2 is large → small p-value → reject H_0.

Assumptions

1. **Independent observations**: Each message is independent (not the same person posting 100 times).
2. **Expected cell counts 5**: Each cell in your crosstab should have at least 5 expected observations. If not, consider combining categories or using Fisher's Exact Test.

Running Chi-Square in R

Step 1: Create a Contingency Table

```
# Crosstab of game and message type
contingency_table <- table(clean_data$game, clean_data$message_type)
contingency_table
```

Output:

```
           Toxic  Neutral
Valorant    80     170
Overwatch   40     210
```

Step 2: Run the Chi-Square Test

```r
chisq_result <- chisq.test(contingency_table)
chisq_result
```

Output:

```
Chi-squared test for independence

X-squared = 12.34, df = 1, p-value = 0.0004
```

Step 3: Interpret

- **X-squared (²)**: Test statistic (12.34). Larger values = stronger evidence against the null.
- **df (degrees of freedom)**: (rows - 1) × (columns - 1). Here: (2-1) × (2-1) = 1.
- **p-value**: 0.0004 < 0.05 → Reject H_0.

Conclusion: "There is a statistically significant association between game type and message toxicity, ²(1) = 12.34, p < .001. Valorant chat messages are more likely to be toxic than Overwatch messages."

Effect Size: Beyond Significance

A significant p-value tells you there's *probably* a real effect. But how **big** is the effect?

Cramér's V

Cramér's V measures the strength of association (0 = no association, 1 = perfect association).

Formula:

$$V = \sqrt{\frac{\chi^2}{n \times (k-1)}}$$

Where:

- n = total sample size
- k = smaller of (rows, columns)

Interpretation:

- V = 0.10: Small effect
- V = 0.30: Medium effect
- V = 0.50: Large effect

R code:

```r
library(effectsize)

cramers_v(contingency_table)
```

Output:

```
Cramér's V = 0.18 (small to medium effect)
```

Interpretation: The association is statistically significant but relatively weak. Game type explains 18% of the variance in toxicity.

Reporting Results

Follow APA style for reporting Chi-Square:

Template:

> A Chi-Square test of independence was conducted to examine the relationship between [Variable 1] and [Variable 2]. The relationship was statistically significant, ^{2}(df) = X.XX, p < .05, V = X.XX. [Describe the pattern].

Example:

> A Chi-Square test of independence examined the relationship between game type (Valorant, Overwatch) and message toxicity (Toxic, Neutral). The relationship was statistically significant, ^{2}(1) = 12.34, p < .001, V = 0.18. Valorant chat messages were more likely to be toxic (32%) compared to Overwatch messages (16%).

Statistical Significance vs. Practical Significance

A **statistically significant** result means the pattern probably isn't due to chance. But is it **meaningful**?

Example: You find that female streamers' chats have 1% more toxic messages than male streamers' chats. Chi-Square is significant ($p = 0.04$) because your sample is huge ($n = 10{,}000$).

Question: Does 1% matter in the real world? Probably not. This is statistically significant but practically trivial.

Best practice: Report both p-values (statistical significance) and effect sizes (practical significance). Discuss whether the effect is large enough to matter.

Common Misinterpretations

Misinterpretation 1: "$p < 0.05$ means the hypothesis is true"

Correction: $p < 0.05$ means the data are *surprising* if the null is true. It's evidence against the null, not proof of your hypothesis.

Misinterpretation 2: "$p = 0.051$ means no effect"

Correction: $p = 0.051$ is arbitrary. The effect might be real; you just lack strong evidence. Don't treat 0.05 as a magic threshold.

Misinterpretation 3: "Non-significant = no difference"

Correction: Failing to reject the null doesn't mean the null is true. It means you don't have enough evidence to reject it (possibly due to small sample size or weak effect).

When Chi-Square Isn't Appropriate

Use Fisher's Exact Test instead if:

- Expected cell counts < 5 (Chi-Square loses accuracy).
- Very small sample sizes ($n < 20$).

R code:

```
fisher.test(contingency_table)
```

Use other tests if:

- **Both variables are continuous**: Use correlation (Pearson's r) or regression.
- **One categorical, one continuous**: Use t-test or ANOVA.
- **More than two variables**: Use logistic regression or multivariate analysis.

Real Data: American Idol Judge Feedback Analysis

Let's apply inferential tests to answer research questions using the cleaned American Idol dataset (from Chapters 11-12).

Research Question 1: Does Judge Tone Differ by Contestant Gender?

Research Question: Do judges provide different feedback (tone) to male vs. female contestants?

Data Types: - Judge Tone: Continuous (0-3 scale) - Contestant Gender: Categorical (Male, Female)

Appropriate Test: Independent samples t-test (comparing means of continuous variable across 2 groups)

Hypotheses: - H_0: Male and female contestants receive equal average judge tone. - H_1: Male and female contestants receive different average judge tone.

Step 1: Descriptive Statistics

```
library(tidyverse)

idol_data <- read_csv("data_clean/idol_judge_feedback.csv")

# Summary by gender
tone_by_gender <- idol_data %>%
  filter(!is.na(gender), !is.na(tone)) %>%
  group_by(gender) %>%
  summarize(
    mean_tone = mean(tone),
```

```r
    sd_tone = sd(tone),
    n = n()
  )

print(tone_by_gender)
```

Output:

```
gender    mean_tone  sd_tone    n
Female        2.12    0.81    54
Male          2.35    0.75    91
```

Interpretation: Male contestants receive slightly higher average tone (2.35 vs. 2.12), a difference of about 0.23 points on a 0-3 scale.

Step 2: Conduct t-test

```r
library(rstatix)  # For easier t-test reporting

# Prepare data (remove missing values)
idol_clean <- idol_data %>%
  filter(!is.na(gender), !is.na(tone))

# Run independent samples t-test
t_test_result <- t_test(
  idol_clean,
  tone ~ gender,
  var.equal = TRUE  # Assume equal variances
)

print(t_test_result)

# Get detailed output with effect size
t.test(idol_clean$tone[idol_clean$gender == "Female"],
       idol_clean$tone[idol_clean$gender == "Male"],
       var.equal = TRUE)
```

Output:

```
t-test for independent samples
```

```
  Welch Two Sample t-test

data:  tone by gender
t = -1.87, df = 143, p-value = 0.063
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.473  0.013
sample estimates:
mean in group Female mean in group Male
              2.12              2.35
```

Step 3: Calculate Effect Size (Cohen's d)

```r
library(effectsize)

cohens_d(idol_clean$tone ~ idol_clean$gender)
```

Output:

```
Cohen's d |  95% CI
        0.29 | [0.00, 0.58]
```

Step 4: Interpretation and Reporting

- **p-value = 0.063**: Marginally non-significant (just above the 0.05 threshold)
- **Cohen's d = 0.29**: Small effect size
- **Conclusion**: "An independent samples t-test examined whether judge feedback tone differed by contestant gender. While males received slightly higher tone scores (M = 2.35, SD = 0.75) compared to females (M = 2.12, SD = 0.81), this difference was marginally non-significant, t(143) = -1.87, p = .063, d = 0.29. The effect size is small. This suggests judges' feedback tone may not systematically differ by contestant gender, although a larger sample might reveal a genuine effect."

Research Question 2: Does Advancement Differ by Judge Feedback Tone?

Research Question: Is positive judge feedback associated with contestant advancement?

Data Types: - Judge Tone: Categorical (Positive = tone 2 vs. Non-positive = tone < 2) - Advancement: Categorical (Advanced = 1 vs. Eliminated = 0)

Appropriate Test: Chi-Square Test of Independence (association between two categorical variables)

Hypotheses: - H_0: Judge tone and advancement are independent. - H_1: Judge tone and advancement are associated.

Step 1: Create Contingency Table

```
# Create binary tone variable
idol_chi <- idol_data %>%
  filter(!is.na(tone), !is.na(advancement)) %>%
  mutate(
    tone_category = if_else(tone >= 2, "Positive", "Non-Positive"),
    advancement_label = if_else(advancement == 1, "Advanced", "Eliminated")
  )

# Contingency table
contingency <- table(idol_chi$tone_category, idol_chi$advancement_label)
print(contingency)
```

Output:

```
              Advanced Eliminated
Non-Positive        15         38
Positive            82         21
```

Step 2: Run Chi-Square Test

```
chi_result <- chisq.test(contingency)
chi_result
```

Output:

```
Chi-squared test for independence

data:  contingency
X-squared = 46.78, df = 1, p-value = 7.2e-12
```

Step 3: Calculate Effect Size (Cramér's V)

```
cramers_v(contingency)
```

Output:

```
Cramér's V |  95% CI
       0.49 | [0.37, 0.59]
```

Step 4: Report Proportions

```r
# Calculate proportions
prop_advancement <- idol_chi %>%
  group_by(tone_category) %>%
  summarize(
    advanced = sum(advancement_label == "Advanced"),
    eliminated = sum(advancement_label == "Eliminated"),
    total = n(),
    advancement_rate = advanced / total
  )

print(prop_advancement)
```

Output:

```
tone_category  advanced eliminated total advancement_rate
Non-Positive        15         38    53             0.283
Positive            82         21   103             0.796
```

Step 5: Interpretation and Reporting

- **p-value = 7.2e-12**: Highly significant ($p < 0.001$)
- **Cramér's V = 0.49**: Large effect size
- **Advancement rates**: 79.6% with positive feedback vs. 28.3% with non-positive feedback

"A Chi-Square test of independence examined whether judge feedback tone predicted contestant advancement. The relationship was highly significant, $^2(1) = 46.78$, $p < .001$, $V = 0.49$. Contestants receiving positive feedback advanced at a rate of 79.6%, while those receiving non-positive feedback advanced at a rate of 28.3%. This large effect indicates that judge feedback tone is a strong predictor of contestant advancement."

Research Question 3: Do Multiple Judges Show Consistent Patterns?

Research Question: Does judge feedback tone vary by judge, or is the tone-advancement relationship consistent across all three judges?

Data Types: - Judge: Categorical (3 categories: Katy, Luke, Lionel) - Judge Tone: Continuous (0-3 scale)

Appropriate Test: One-Way ANOVA (comparing means across 3+ groups)

Hypotheses: - H_0: All three judges give equal average tone scores. - H_1: Judge tone scores differ among judges.

Step 1: Descriptive Statistics

```
tone_by_judge <- idol_data %>%
  filter(!is.na(judge_name), !is.na(tone)) %>%
  group_by(judge_name) %>%
  summarize(
    mean_tone = mean(tone),
    sd_tone = sd(tone),
    n = n()
  )

print(tone_by_judge)
```

Output:

```
judge_name   mean_tone  sd_tone    n
Katy Perry       2.15    0.79    58
Lionel Richie    2.22    0.76    57
Luke Bryan       2.28    0.82    59
```

Step 2: Run One-Way ANOVA

```
anova_result <- aov(tone ~ judge_name, data = idol_data)
summary(anova_result)
```

Output:

```
            Df Sum Sq Mean Sq F value Pr(>F)
judge_name   2  0.85   0.426   0.674  0.511
Residuals  171 108.34   0.634
```

Step 3: Calculate Effect Size (Eta-squared)

```
library(effectsize)

eta_squared(anova_result)
```

Output:

```
Eta2_partial |  95% CI
         0.0078 | [0.00, 0.04]
```

Step 4: Interpretation and Reporting

- **p-value = 0.511**: Not significant (well above 0.05 threshold)
- **Eta-squared = 0.008**: Negligible effect size
- **Conclusion**: "A one-way ANOVA examined whether judge feedback tone differed among the three judges (Katy Perry, Luke Bryan, Lionel Richie). The analysis revealed no significant differences in tone across judges, F(2, 171) = 0.67, p = .511, ^{2} = .008. All three judges provided similar average tone scores (M = 2.15-2.28 on a 0-3 scale), suggesting consistency in their feedback patterns across contestants."

Case Study: Testing Toxicity by Game

Research Question: Is there a relationship between game type and message toxicity?

Hypotheses:

- H_0: Game type and toxicity are independent.
- H_1: Game type and toxicity are associated.

Data: 500 messages from Valorant (250) and Overwatch (250).

Step 1: Contingency table

```r
table(clean_data$game, clean_data$message_type)
```

```
           Toxic  Neutral
Valorant    80     170
Overwatch   40     210
```

Step 2: Chi-Square test

```r
chisq.test(table(clean_data$game, clean_data$message_type))
```

```
 ²(1) = 12.34, p = 0.0004
```

Step 3: Effect size

```r
cramers_v(table(clean_data$game, clean_data$message_type))
```

```
V = 0.18
```

Step 4: Report

> A Chi-Square test revealed a significant association between game type and message toxicity, $^2(1) = 12.34$, $p < .001$, $V = 0.18$. Valorant messages were more likely to be toxic (32%) than Overwatch messages (16%), suggesting that competitive game contexts may foster higher rates of hostile communication.

Choosing the Right Test: A Decision Guide

Before diving into individual tests, let's establish a decision framework. The appropriate inferential test depends on:

1. **Your research question type**: Comparing groups? Testing associations? Measuring relationships?
2. **Number of groups**: Two? Three or more?
3. **Data types**: Both categorical? One categorical, one continuous? Both continuous?

Test Selection Matrix

Research Question	Data Types	Groups	Test
Do two games differ in toxicity **rate**?	Cat, Cat	2	Chi-Square
Do female & male streamers differ in toxicity **rate**?	Cat, Cat	2	Chi-Square
Do Twitch, YouTube, TikTok differ in toxicity **rate**?	Cat, Cat	3+	Chi-Square
Do two games differ in **average** chat sentiment?	Cont, Cat	2	t-test
Do female & male creators differ in **average** donations?	Cont, Cat	2	t-test

Research Question	Data Types	Groups	Test
Do three games differ in **average** viewer engagement?	Cont, Cat	3+	ANOVA
What's the **strength of relationship** between views and donations?	Cont, Cont	—	Correlation
How much do hours streamed and game type together predict earnings?	Cont, Cat, Cat	—	Regression

Key insight: Your research question determines the test, not the other way around.

The Independent Samples t-Test

When to Use t-Test

Use an **independent samples t-test** when you want to **compare means between two groups**.

Example research questions:

- Do male and female streamers receive different average donations per stream?
- Is the average chat sentiment higher on Twitch than YouTube?
- Do competitive games have higher average viewer counts than casual games?

The Null Hypothesis for t-Test

H_0: The two groups have equal means: $\mu_1 = \mu_2$

H_1: The two groups have different means: $\mu_1 \neq \mu_2$ (two-tailed) or $\mu_1 > \mu_2$ (one-tailed)

How t-Test Works (Conceptually)

The t-test compares the **difference between group means** to the **variability within groups**:

$$t = \frac{\text{Difference between means}}{\text{Pooled standard error}}$$

Intuition:

- If groups are far apart and within-group variation is small → large t → small p-value → reject H_0
- If groups are close together or within-group variation is large → small t → large p-value → fail to reject H_0

Assumptions

1. **Independent observations**: Observations in one group don't influence the other.
2. **Continuous data**: Your dependent variable is measured on a continuous scale (not categories).
3. **Approximately normal**: Data in each group should be roughly normally distributed (less critical with large n).
4. **Equal variances** (for standard t-test): The two groups should have similar variability. (Welch's t-test relaxes this).

Running t-Test in R

Step 1: Prepare Data

Imagine you've coded streamer donations and gender:

```
# Create sample data
streamer_data <- data.frame(
  gender = c("Male", "Male", "Female", "Female", "Male", "Female"),
  donations = c(250, 310, 280, 340, 270, 360)
)
```

Step 2: Run t-Test

```r
# Independent samples t-test
t_result <- t.test(donations ~ gender, data = streamer_data)
t_result
```

Output:

```
Two Sample t-test

data:  donations by gender
t = -1.89, df = 4, p-value = 0.144
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -172.8  30.1
sample estimates:
mean in group Female  mean in group Male
              326.7               276.7
```

Step 3: Interpret

- **t = -1.89**: Test statistic. Negative means the first group has a lower mean.
- **df = 4**: Degrees of freedom (n + n - 2 = 3 + 3 - 2 = 4).
- **p-value = 0.144**: Not significant at = 0.05. We fail to reject H_0.
- **95% CI [-172.8, 30.1]**: The true difference in means is probably between -172.8 and 30.1 donations (includes zero).
- **Means**: Female streamers averaged $326.70; male streamers averaged $276.70 (a $50 difference, but not statistically significant).

Conclusion: "There was no significant difference in average donations between female (M = $326.70) and male (M = $276.70) streamers, t(4) = -1.89, p = .144."

Effect Size for t-Test: Cohen's d

Cohen's d measures the **standardized difference** between means:

$$d = \frac{M_1 - M_2}{SD_{\text{pooled}}}$$

Interpretation:

- d = 0.20: Small effect

- d = 0.50: Medium effect
- d = 0.80: Large effect

R code:

```
library(effectsize)

cohens_d(donations ~ gender, data = streamer_data)
```

Output: d = 0.32 (small to medium effect)

Analysis of Variance (ANOVA)

When to Use ANOVA

Use **ANOVA (Analysis of Variance)** when you want to compare means across **three or more groups**.

Example research questions:

- Do toxicity rates differ across game genres (FPS, MOBA, MMO, Casual)?
- Is average engagement different for morning, afternoon, and evening streams?
- Do three social media platforms (TikTok, Instagram, YouTube) differ in average video length?

Why Not Just Run Multiple t-Tests?

If you have 4 groups, you might compare: Group 1 vs. 2, 1 vs. 3, 1 vs. 4, 2 vs. 3, 2 vs. 4, 3 vs. 4. That's **6 t-tests**.

Problem: Each t-test has a 5% Type I error rate. Running 6 tests inflates the overall error rate to ~26%! This is **multiple comparisons problem**.

Solution: ANOVA controls the overall Type I error rate while testing all groups simultaneously.

The Null Hypothesis for ANOVA

H_0: All group means are equal: $\mu_1 = \mu_2 = \mu_3 = ... = \mu_k$

H_1: At least one group mean differs from the others.

How ANOVA Works (Conceptually)

ANOVA partitions variance into two sources:

$$F = \frac{\text{Variance between groups}}{\text{Variance within groups}}$$

- **Between-group variance**: How different are the group means from each other?
- **Within-group variance**: How much do individuals vary within their own group?

If between-group variance is large compared to within-group variance → large F → small p-value → reject H_0.

Running One-Way ANOVA in R

Step 1: Prepare Data

```
# Game type and toxicity rate
game_data <- data.frame(
  game = c("FPS", "FPS", "MOBA", "MOBA", "Casual", "Casual"),
  toxicity = c(35, 42, 38, 45, 18, 22)
)
```

Step 2: Run ANOVA

```
anova_result <- aov(toxicity ~ game, data = game_data)
summary(anova_result)
```

Output:

```
            Df Sum Sq Mean Sq F value Pr(>F)
game         2  834.0   417.0   20.85  0.009
Residuals    3   60.0    20.0

Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretation:

- **Df**: Degrees of freedom (groups - 1 = 3 - 1 = 2; residuals = n - groups = 6 - 3 = 3).
- **F = 20.85**: Test statistic.
- **p-value = 0.009**: Significant at = 0.05. At least one group differs from the others.

Conclusion: "Game type significantly predicted toxicity rates, F(2, 3) = 20.85, p = .009."

Step 3: Follow-Up: Which Groups Differ?

ANOVA tells you *that* groups differ, but not *which* groups differ from each other. Use **post-hoc comparisons** (Tukey HSD) to find out:

```r
TukeyHSD(anova_result)
```

Output:

```
  Tukey multiple comparisons of means
    95% family-wise confidence level

$game
         diff     lwr     upr   p adj
MOBA-FPS    1   -10.8   12.8   0.961
Casual-FPS -22   -33.8  -10.2   0.003
Casual-MOBA -23   -34.8  -11.2   0.002
```

Interpretation:

- **MOBA vs. FPS**: No significant difference (p = 0.961).
- **Casual vs. FPS**: Significant difference (p = 0.003). Casual games are **22% less toxic** than FPS.
- **Casual vs. MOBA**: Significant difference (p = 0.002). Casual games are **23% less toxic** than MOBA.

Effect Size for ANOVA: Eta-Squared (^{2})

Eta-squared measures the proportion of variance in the dependent variable explained by the grouping variable:

$$\eta^2 = \frac{\text{Sum of Squares Between}}{\text{Total Sum of Squares}}$$

Interpretation:

- $^2 = 0.01$: Small effect
- $^2 = 0.06$: Medium effect
- $^2 = 0.14$: Large effect

R code:

```
library(effectsize)

eta_squared(anova_result)
```

Correlation: Measuring Linear Relationships

When to Use Correlation

Use **Pearson's r correlation** when you want to measure the **strength and direction of a linear relationship between two continuous variables**.

Example research questions:

- Is there a relationship between hours streamed per week and total monthly earnings?
- Do videos with more hashtags receive more views?
- Is average viewer engagement correlated with stream length?

Note: Correlation **does not imply causation**. Both variables might be influenced by a third variable, or the relationship might be reverse.

The Null Hypothesis for Correlation

H_0: There is no linear relationship: $r = 0$

H_1: There is a linear relationship: $r \neq 0$

How Correlation Works (Conceptually)

Pearson's r measures how closely two variables **co-vary**:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

Interpretation:

- **r = +1.0**: Perfect positive correlation (as x increases, y increases).
- **r = 0.0**: No linear relationship.
- **r = -1.0**: Perfect negative correlation (as x increases, y decreases).

General guidelines:

- |r| = 0.10-0.29: Weak
- |r| = 0.30-0.49: Moderate
- |r| = 0.50-1.00: Strong

Running Correlation in R

Step 1: Prepare Data

```
# Stream hours and monthly earnings
stream_data <- data.frame(
  hours_per_week = c(10, 20, 15, 25, 30, 18, 22),
  earnings_per_month = c(500, 1200, 800, 1500, 2000, 1000, 1300)
)
```

Step 2: Calculate Correlation

```
cor_result <- cor.test(stream_data$hours_per_week, stream_data$earnings_per_month)
cor_result
```

Output:

```
Pearson's product-moment correlation

data:  hours_per_week and earnings_per_month
t = 8.57, df = 5, p-value = 0.0005
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.848 0.998
sample estimates:
     cor
    0.966
```

Interpretation:

- **r = 0.966**: Very strong positive correlation.
- **p-value = 0.0005**: Significant at = 0.05.
- **95% CI [0.848, 0.998]**: The true population correlation is likely between 0.85 and 0.998.

Conclusion: "There was a strong positive correlation between hours streamed per week and monthly earnings, r(5) = .97, p < .001. Streamers who spent more hours streaming earned significantly more per month."

Step 3: Visualize

Always plot bivariate relationships:

```r
library(ggplot2)

ggplot(stream_data, aes(x = hours_per_week, y = earnings_per_month)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = TRUE, color = "blue") +
  labs(
    title = "Stream Hours and Monthly Earnings",
    x = "Hours Streamed per Week",
    y = "Monthly Earnings ($)",
    caption = "r = .97, p < .001"
  ) +
  theme_minimal()
```

Why visualize? Correlation assumes a **linear** relationship. You might have a curved relationship, clusters, or outliers that numbers alone don't reveal.

When Correlation Breaks Down

Example: The Spurious Correlation

Claim: "Ice cream sales are correlated with drowning deaths, r = 0.88!"

Truth: Both are correlated with **temperature** (a third variable). Ice cream and drowning aren't causally related; they're confounded by season.

Lesson: Always think about alternative explanations. Correlation Causation.

Exercise 13.1: Interpreting P-Values

For each scenario, interpret the p-value correctly:

Scenario A: You test whether streamers with webcams receive more donations. p = 0.03

Wrong interpretation: "There's a 3% chance the hypothesis is true."

Correct interpretation: _____

Conclusion: Reject null? Yes / No (circle one) **Why**: _____

Scenario B: You test whether female streamers experience more harassment. p = 0.12

Correct interpretation: _____ **Conclusion**: Reject null? Yes / No **Does this mean female streamers DON'T experience more harassment?**: _____

Scenario C: You test game genre and toxicity. p = 0.001

Correct interpretation: _____ **Strength of evidence**: Weak / Moderate / Strong (circle one)

Exercise 13.2: Chi-Square Assumptions and Calculations

Given this contingency table for Platform (Twitch vs. YouTube) and Content Type (Gaming vs. Just Chatting):

	Gaming	Just Chatting	Total
Twitch	85	65	150
YouTube	45	55	100
Total	130	120	250

Questions:

1. State the null and alternative hypotheses:
   - **H** : _____
   - **H** : _____
2. Check assumptions:
   - Are observations independent? _____
   - Calculate expected counts for each cell (show work for one):
     - Expected (Twitch, Gaming) = _____
   - Are all expected counts 5? _____
3. If ² = 6.83, df = 1, p = 0.009:
   - **Interpretation**: _____
   - **Reject null?**: Yes / No
   - **Write one-sentence conclusion**: _____
4. Calculate Cramér's V (given ² = 6.83, n = 250):
   - **V** = _____
   - **Effect size interpretation**: Small / Medium / Large

Exercise 13.3: Statistical vs. Practical Significance

Study Results: A researcher finds that Game A has a mean toxicity rate of 15.2% and Game B has 15.8%. With n = 10,000 messages per game, this difference is statistically significant (p = 0.04).

Questions:

1. Is this result statistically significant? _____
2. Is the effect **practically** meaningful? _____
3. Why or why not? _____
4. What additional information would help you assess practical significance? _____

Exercise 13.4: Null Hypothesis Formulation

Write appropriate null and alternative hypotheses for each research question:

RQ1: "Is there a relationship between streamer gender and chat toxicity?" - **H** : _____ - **H** : _____

RQ2: "Do tweets about esports generate more engagement than tweets about traditional sports?" - **H** : _____ - **H** : _____

RQ3: "Are Reddit posts with sensational headlines more likely to be upvoted?" - **H** : _____ - **H** : _____

Exercise 13.5: Type I and Type II Errors

A researcher tests whether violent video games increase aggression (H : they do).

Scenario A: The researcher rejects the null (concludes games increase aggression), but actually they don't. - **Error type**: Type I / Type II (circle one) - **Consequence**: _____

Scenario B: The researcher fails to reject the null (concludes no effect), but games actually DO increase aggression. - **Error type**: Type I / Type II - **Consequence**: _____

Trade-off question: If you had to choose, which error is worse in this context? Why? _____

Reflection Questions

1. **The p = 0.05 Threshold**: Why is 0.05 the conventional cutoff for significance? Is it arbitrary? Should different fields or research questions use different thresholds? When might p = 0.10 be acceptable?
2. **Negative Results**: You hypothesized that streamer interactivity would predict donations, but p = 0.67 (not significant). Is this a "failed" study? What can you learn from null results? Should journals publish more non-significant findings?
3. **Sacred Flaw Revisited**: Return to Chapter 1's concept of the null hypothesis as the "sacred flaw." How does exposing this flaw (rejecting the null) feel different after actually conducting statistical tests? Did the process change your understanding of scientific knowledge?

Chapter Summary

This chapter introduced inferential statistics, hypothesis testing, and a toolkit of foundational statistical tests:

Foundational Concepts: - **Inferential statistics** assess whether sample patterns likely exist in the population or are just random noise - The **null hypothesis (H)** assumes no effect or relationship; the **alternative hypothesis (H)** predicts an effect - **P-value**: Probability of observing data this extreme if the null were true—NOT the probability the hypothesis is true - **Significance threshold**: $p < 0.05$ conventionally means "reject the null" (evidence against no effect) - **Type I error** (= 0.05): False positive (rejecting a true null) - **Type II error** ( ): False negative (failing to reject a false null) - **Statistical significance practical significance**: Always report effect sizes

Choosing Tests by Research Question:

Question	Data Types	Test	Effect Size
Is there an **association** between two categories?	Cat-Cat	Chi-Square	Cramér's V
Do two groups differ in **average**?	Cont-Cat (2 groups)	t-test	Cohen's d
Do 3+ groups differ in **average**?	Cont-Cat (3+ groups)	ANOVA	Eta-squared
How **strong** is the relationship between two continuous variables?	Cont-Cont	Correlation	Pearson's r

Chi-Square Test of Independence: - Compares **observed vs. expected frequencies** for categorical associations - Assumptions: Independent observations, expected counts 5 - Report: ^{2}(df) = X.XX, p = .XXX, V = X.XX

Independent Samples t-Test: - Compares **means between two groups** - Assumptions: Independent observations, continuous data, normal distribution - Report: t(df) = X.XX, p = .XXX, d = X.XX

One-Way ANOVA: - Compares **means across three or more groups** - Controls for multiple comparisons problem - Use Tukey HSD post-hoc test to identify which groups differ - Report: F(df , df) = X.XX, p = .XXX, ^{2} = X.XX

Pearson's Correlation: - Measures **strength of linear relationship** between continuous variables - r ranges from -1 (perfect negative) to +1 (perfect positive) - Always visualize correlations; correlation causation - Report: r(df) = X.XX, p = .XXX

Key Terms

- **Alternative hypothesis (H)**: Prediction that an effect or relationship exists
- **ANOVA (Analysis of Variance)**: Inferential test for comparing means across three or more groups
- **Chi-Square test (^{2})**: Inferential test for association between two categorical variables
- **Cohen's d**: Effect size for t-test (0.2 = small, 0.5 = medium, 0.8 = large)
- **Correlation (Pearson's r)**: Measure of linear relationship between two continuous variables (-1 to +1)
- **Cramér's V**: Effect size measure for Chi-Square (0 to 1, indicating association strength)
- **Degrees of freedom (df)**: Number of values free to vary in a calculation

- **Effect size**: Magnitude of an effect, independent of sample size
- **Eta-squared (²)**: Effect size for ANOVA (0.01 = small, 0.06 = medium, 0.14 = large)
- **Expected frequency**: Count predicted if the null hypothesis were true
- **Fisher's Exact Test**: Alternative to Chi-Square for small samples (expected counts < 5)
- **F-statistic**: Test statistic for ANOVA (between-group variance / within-group variance)
- **Independent samples t-test**: Inferential test for comparing means between two groups
- **Inferential statistics**: Statistical methods for generalizing from sample to population
- **Null hypothesis (H)**: Assumption that no effect or relationship exists
- **Observed frequency**: Actual count in each cell of a contingency table
- **P-value**: Probability of observing data this extreme if the null were true
- **Practical significance**: Whether an effect is large enough to matter in the real world
- **Post-hoc test (Tukey HSD)**: Follow-up comparison after significant ANOVA to identify which groups differ
- **Regression**: Statistical method predicting a continuous outcome from one or more predictors
- **Statistical significance**: Result unlikely due to chance alone (typically $p < 0.05$)
- **t-statistic**: Test statistic for t-test (difference between means / standard error)
- **Type I error**: False positive (rejecting a true null hypothesis)
- **Type II error**: False negative (failing to reject a false null hypothesis)

Looking Ahead

Chapter 14 brings everything together into **The One-Click Report**—assembling your complete research narrative using the IMRaD structure (Introduction, Methods, Results, Discussion), rendering dynamic documents that integrate code and output, and deploying your final portfolio to GitHub Pages as a live, publishable website. This is the culmination of your research journey.

The One-Click Report

Learning Objectives

- Assemble a complete research report using the IMRaD structure.
- Render dynamic documents that weave narrative text and executable code.
- Deploy your final portfolio to GitHub Pages as a live website.
- Reflect on the research journey from curiosity to published findings.

Overview

This is the synthesis and publication chapter. You've completed the research cycle: from curiosity (Ch 1) to evidence (Ch 13). Now you package it into a professional, reproducible portfolio that demonstrates your ability to conduct rigorous, data-driven research.

The final deliverable is not a static PDF—it's a **live, interactive website** hosted on GitHub Pages, where your code, visualizations, and narrative coexist in one reproducible document.

The One-Click Philosophy

Recall Chapter 2: Quarto documents are **recipes**. Your final report is not "written"—it's **rendered**. Text, code, tables, and figures are woven together. If your data changes, you click "Render" and a fresh, updated report appears instantly.

This is the power of **reproducible research**: others (including your future self) can verify, extend, or challenge your findings because every step is documented and executable.

The IMRaD Structure

IMRaD is the standard format for scientific papers:

- **I**: Introduction
- **M**: Methods
- **R**: Results
- **a**: and

- **D**: Discussion

This structure mirrors the research process and guides readers through your logic.

14.2.1 Introduction (The Story)

Reintroduce the narrative from Chapter 1:

- **What is the research problem?** (The inciting incident)
- **Why does it matter?** (Stakes)
- **What is your research question or hypothesis?** (The protagonist's quest)

Components:

1. **Hook**: Open with a compelling anecdote, statistic, or question.

Example:

> "In 2023, Twitch averaged 2.5 million concurrent viewers daily, with millions of chat messages exchanged every hour. But beneath the emotes and inside jokes lies a darker pattern: toxic language, harassment, and exclusion—particularly in competitive gaming spaces."

2. **Literature Review** (2-3 paragraphs): What have others discovered? Where is the gap your study fills?

Example:

> "Prior research has documented toxicity in multiplayer games (Gray, 2014; Consalvo, 2012), but few studies examine live-streaming contexts where real-time moderation and parasocial relationships complicate community dynamics. This study addresses that gap by analyzing chat behavior across competitive and casual game streams."

3. **Research Question or Hypothesis**: State it explicitly.

Example:

> **RQ**: Is there a relationship between game type (competitive vs. casual) and the prevalence of toxic chat messages?

14.2.2 Methods (The Recipe)

Document **every decision** so someone else could replicate your study.

Components:

1. **Data Source**: Archive, platform, date range, search terms.

Example:

> "Data were collected from 50 Twitch streams broadcast between February 1-28, 2024. Streams were selected from the top 100 channels by average concurrent viewers on TwitchTracker.com, stratified by game genre (25 competitive FPS streams, 25 casual simulation/RPG streams)."

2. **Sample**: Sampling frame, technique, final n.

Example:

> "From each stream, 10 consecutive chat messages were coded, yielding 500 total messages (250 from competitive games, 250 from casual games). Messages consisting solely of emotes were excluded."

3. **Variables**: Operational definitions (pull from your codebook, Chapter 9).

Example:

> "**Message toxicity** was coded as a binary variable: Toxic (messages containing personal insults, slurs, or threats) vs. Neutral (all other messages). See Appendix A for the full codebook."

4. **Reliability**: Report pilot ICR (Chapter 10).

Example:

> "Two independent coders analyzed 50 pilot messages. Inter-coder reliability was acceptable (Krippendorff's = .76). Disagreements were resolved through discussion, and the codebook was revised before final coding."

5. **Analytic Plan**: What statistical tests did you use?

Example:

> "Descriptive statistics (frequencies, proportions) summarized the distribution of toxic messages. A Chi-Square test of independence assessed the relationship between game type and toxicity."

14.2.3 Results (The Data Speak)

Present findings **objectively**. No interpretation yet—just report what the numbers say.

Components:

1. **Descriptive Statistics**: Summarize the sample.

Example:

> "Of the 500 coded messages, 120 (24%) were classified as toxic. Competitive game streams accounted for 80 toxic messages (32% of competitive messages), while casual game streams accounted for 40 toxic messages (16% of casual messages)."

2. **Inferential Statistics**: Report test results.

Example:

> "A Chi-Square test revealed a statistically significant association between game type and message toxicity, $^2(1) = 12.34$, $p < .001$, Cramér's $V = 0.18$. Competitive game chats exhibited higher rates of toxicity compared to casual game chats."

3. **Tables and Figures**: Visualize key findings.

Table 1: Toxicity by Game Type

Game Type	Toxic	Neutral	Total	% Toxic
Competitive	80	170	250	32%
Casual	40	210	250	16%
Total	120	380	500	24%

Figure 1: Toxicity Rates by Game Type

```r
ggplot(summary_data, aes(x = game_type, y = toxicity_rate)) +
  geom_col(fill = "coral") +
  labs(
    title = "Toxicity Rates in Competitive vs. Casual Game Streams",
    x = "Game Type",
    y = "Toxicity Rate (%)"
  )
```

14.2.4 Discussion (The Interpretation)

Now you interpret. What do the findings **mean**?

Components:

1. **Summary of Findings**: Restate the main result in plain language.

Example:

> "This study found that competitive game streams on Twitch exhibit significantly higher rates of toxic chat messages compared to casual game streams, supporting the hypothesis that competitive contexts amplify hostile communication."

2. **Theoretical Implications**: Connect back to your theory (Chapter 5).

Example:

> "These findings align with Social Identity Theory (Tajfel & Turner, 1979), which predicts that competitive contexts heighten in-group/out-group dynamics, leading to increased hostility toward perceived rivals. Competitive gaming may activate tribal identities, normalizing aggressive language."

3. **Practical Implications**: Why should anyone care?

Example:

> "Platform designers and moderators should recognize that competitive game streams may require stricter moderation policies and automated toxicity detection tools. Streamers in competitive genres may benefit from training in de-escalation techniques."

4. **Limitations**: Be transparent about weaknesses.

Example:

> "This study analyzed only the top 100 Twitch channels, limiting generalizability to smaller streamers. Additionally, coding relied on manifest content (observable language) and may have missed subtle forms of toxicity such as sarcasm or coded language. Future research should incorporate sentiment analysis and examine longitudinal patterns."

5. **Future Directions**: What comes next?

Example:

> "Future studies could examine how streamer responses to toxic chat (bans, timeouts, verbal reprimands) affect subsequent toxicity rates. Additionally, comparative analysis across platforms (Twitch vs. YouTube Gaming) could reveal whether platform affordances shape community norms."

Writing in Quarto: Integrating Code and Narrative

Quarto allows you to embed R code directly into your report. When you render the document, the code executes and outputs (tables, plots) appear inline.

Example: Embedding a Table

```
# Descriptive statistics
summary_table <- clean_data %>%
  group_by(game_type) %>%
  summarize(
    total = n(),
    toxic = sum(message_type == "Toxic"),
    toxicity_rate = toxic / total
  )

knitr::kable(summary_table, caption = "Toxicity by Game Type")
```

Rendered output: A formatted table appears in your report.

Example: Embedding a Plot

```
ggplot(clean_data, aes(x = game_type, fill = message_type)) +
  geom_bar(position = "fill") +
  labs(
    title = "Message Toxicity by Game Type",
    x = "Game Type",
    y = "Proportion"
  )
```

Rendered output: The plot appears in your report, generated directly from your data.

Real Data: American Idol Research Report Example

Let's assemble a complete IMRaD report using the American Idol judge feedback data (Chapters 8-13).

American Idol Study: Full Report Outline

Title: "Judge Feedback and Contestant Demographics in American Idol: A Content Analysis of Bias and Advancement Patterns"

Introduction

American Idol, debuting in 2002, has become a cultural phenomenon with over 500 episodes and millions of viewers. A critical element of the show is judges' verbal feedback to contestants—comments that express evaluation, encouragement, or critique. Prior research demonstrates that judges' feedback influences audience perception and, likely, advancement outcomes (Shelly et al., 2015). However, whether judges' feedback differs by contestant demographic characteristics (gender, race/ethnicity) remains understudied in the context of reality television talent competition. This study addresses this gap by analyzing judge feedback tone and its relationship to contestant advancement.

Research Questions: 1. Do judges provide systematically different feedback (tone) to contestants of different genders? 2. Does positive judge feedback predict contestant advancement? 3. Are feedback patterns consistent across judges, or do individual judges show different patterns?

Methods

Data Source and Sample: Data were collected from American Idol Seasons 20-23 (2022-2024). A stratified random sample of 60 episodes was selected across three performance rounds: auditions (n=15), Hollywood week (n=14), and live performance shows (n=31). This stratification ensured representation across judging contexts, as feedback differs between elimination-heavy auditions and emotionally charged live shows.

Each episode features three judges (Katy Perry, Luke Bryan, Lionel Richie) providing feedback to multiple contestants. Judge comments range from brief acknowledgment ("Nice!") to detailed critiques lasting 30+ seconds. All judge utterances directed at a specific contestant were coded (n=174 judge comments from the 60-episode sample).

Variables:

Judge Feedback Tone (Ordinal, 0-3 scale): The emotional valence of a judge's feedback comment. - 3 = Strongly Positive (compliments ability, predicts success, uses superlatives) - 2 = Mildly Positive (appreciates effort, identifies strengths) - 1 = Neutral/Constructive (provides technical feedback without judgment) - 0 = Negative (expresses criticism, doubt, or concern)

Contestant Gender (Nominal): Contestant's gender identity as presented during introduction or identified by judges' pronouns. Categories: Male, Female, Non-Binary.

Contestant Race/Ethnicity (Nominal): Race/ethnicity coded from contestant introduction statements or appearance, including: White, Black, Latinx, Asian, Middle Eastern, Multiracial.

Advancement (Binary): Whether contestant progressed to the next round (Yes=1, No=0) based on judge decision.

Reliability Testing: A pilot study with two independent coders examined 3 episodes (approximately 40 judge comments). Inter-coder reliability was assessed using Krippendorff's Alpha: Judge Tone (=.82), Gender (=.95), Advancement (=1.00). These acceptable levels (all >0.70) indicated codebook clarity. The codebook was finalized and applied to the full sample.

Analysis Plan: Descriptive statistics (means, frequencies) summarized sample characteristics. An independent samples t-test compared judge tone between male and female contestants. A Chi-Square test examined the association between feedback tone and advancement. A one-way ANOVA assessed whether judges differed in feedback patterns.

Results

Descriptive Statistics (n=174 judge comments):

Mean judge tone across all comments was 2.18 (SD=0.79) on the 0-3 scale, indicating predominantly positive feedback. Contestants advanced at a rate of 65.5% (114/174). Gender distribution: Male (52.3%), Female (31.0%), Non-Binary (6.9%), Unknown (9.8%).

Research Question 1: Judge Tone by Contestant Gender

An independent samples t-test examined whether judge tone differed by contestant gender (excluding Unknown category, n=157).

Female contestants (M=2.12, SD=0.81) received marginally lower tone scores than male contestants (M=2.35, SD=0.75), but this difference was marginally non-significant, t(143)=-1.87, p=.063, Cohen's d=0.29 (small effect). The 95% confidence interval for the difference ranged from -0.473 to 0.013.

Research Question 2: Judge Tone and Advancement

A Chi-Square test examined the association between judge feedback tone and contestant advancement. Judge tone was recoded as binary: Positive (tone 2) vs. Non-Positive (tone < 2).

The relationship was highly significant, $^{2}(1)=46.78$, $p<.001$, Cramér's $V=0.49$ (large effect). Contestants receiving positive feedback advanced at a rate of 79.6% (82/103), while those receiving non-positive feedback advanced at only 28.3% (15/53).

Feedback Tone	Advanced	Eliminated	Total	Advancement Rate
Positive	82	21	103	79.6%
Non-Positive	15	38	53	28.3%
Total	**97**	**59**	**156**	**62.2%**

Research Question 3: Consistency Across Judges

A one-way ANOVA tested whether mean judge tone differed among the three judges.

Judges showed remarkably similar feedback patterns: Katy Perry (M=2.15, SD=0.79), Luke Bryan (M=2.28, SD=0.82), Lionel Richie (M=2.22, SD=0.76). The differences were not statistically significant, $F(2,171)=0.67$, $p=.511$, $^{2}=.008$ (negligible effect). This suggests judges apply consistent standards when evaluating contestants.

Discussion

This study examined judge feedback tone and its relationship to contestant demographics and advancement in American Idol Seasons 20-23. Three key findings emerged.

First, we found no significant differences in judge tone by contestant gender, though males received slightly higher tone scores than females (difference of 0.23 points). While this difference was marginally non-significant ($p=.063$) with a small effect size ($d=0.29$), it suggests judges may provide marginally warmer feedback to male contestants, consistent with prior literature documenting gender bias in entertainment industries. However, the modest effect size and marginal significance indicate this pattern warrants replication with larger samples.

Second, positive judge feedback strongly predicted contestant advancement. Contestants receiving positive feedback advanced at a rate of 79.6%, compared to 28.3% for those receiving non-positive feedback. This large effect ($V=0.49$) suggests judge feedback tone is a powerful signal of advancement likelihood, though causality cannot be inferred from this observational study.

Third, all three judges exhibited similar feedback patterns, suggesting institutional consistency in evaluation standards. This is notable for reality television, where judges' idiosyncratic

personalities often drive viewer engagement. The consistency may reflect producers' influence on judge behavior or genuine alignment in evaluative criteria.

Limitations:

This study focused on recent seasons (2022-2024), limiting generalizability to earlier Idol eras with different judging panels and production approaches. Race/ethnicity coding relied on appearance assessment, introducing potential coder bias despite acceptable inter-rater reliability. Additionally, feedback tone is measured through verbal cues alone, missing non-verbal communication (facial expressions, body language) that might convey different messages. Future research should incorporate multi-modal analysis and expand to additional seasons and international Idol franchises.

Conclusions:

While judge feedback tone shows marginal differences by contestant gender, it strongly predicts advancement. The consistency across judges suggests evaluation standards transcend individual judge personalities. Future research should examine whether audience voting behavior aligns with judge feedback and investigate the mechanisms through which feedback influences outcomes.

Deploying to GitHub Pages

Once your report is complete, deploy it as a live website.

Step 1: Render the Site

In RStudio, click **Render** or run:

```
quarto::quarto_render()
```

This generates HTML files in the `_site/` directory.

Step 2: Commit and Push

```
git add .
git commit -m "Final report complete"
git push origin main
```

Step 3: Enable GitHub Pages

1. Go to your GitHub repo → **Settings** → **Pages**.
2. Under "Source," select **Deploy from a branch**.
3. Choose the **main** branch and the **_site** folder.
4. Click **Save**.

GitHub will generate a URL (e.g., `https://sim-lab-siue.github.io/vibes-to-variables/`).

Step 4: Share Your Work

Your research is now live. Share the link with peers, instructors, or potential employers.

The Narrative Arc Completed

Let's revisit the journey:

- **Chapter 1**: You had curiosity—a vibe that something was worth studying.
- **Chapters 2-3**: You learned tools and discipline.
- **Chapters 4-6**: You mapped the territory, chose a lens, and defined your project.
- **Chapters 7-10**: You operationalized variables, built a codebook, sampled data, and tested reliability.
- **Chapters 11-13**: You wrangled, visualized, and analyzed your data.
- **Chapter 14**: You published your findings.

You began as a journalist (curious observer) and ended as a scientist (evidence-driven narrator). You've told a story—not a fictional one, but one grounded in systematic inquiry. You've contributed a new chapter to the collective story of knowledge.

Reflection: What You've Built

This course taught you more than statistics. You've learned:

- **Epistemology**: How to evaluate claims and distinguish evidence from intuition.
- **Transparency**: How to document decisions so others can verify or challenge your work.
- **Humility**: How to acknowledge limitations and resist overclaiming.
- **Communication**: How to translate technical findings for diverse audiences.

These are the skills of a **research literate citizen**: someone who can critically evaluate information, design studies to answer questions, and communicate findings responsibly.

Beyond the Semester

This textbook is not the end—it's a foundation. You now have:

- A portfolio website demonstrating your research skills.
- Familiarity with R, Quarto, Git, and reproducible workflows.
- Conceptual fluency in content analysis, hypothesis testing, and data visualization.

Next steps:

- Extend your project: Collect more data, test new hypotheses.
- Explore advanced methods: Sentiment analysis, network analysis, regression.
- Contribute to open science: Share your data, code, and codebook on OSF or GitHub.

Practice: Assembling Your Research Portfolio

Exercise 14.1: Writing IMRaD Sections

Draft one paragraph for each IMRaD section for your research project (or a hypothetical one):

Introduction (The Hook + Gap)

Write an opening paragraph that: - Opens with a compelling hook (anecdote, statistic, or question) - Establishes why the topic matters - Identifies the research gap your study fills

Your paragraph: _____

Methods (The Recipe)

Write a methods paragraph describing: - Your data source and sampling strategy - Your sample size and date range - Your variables and how you operationalized them

Your paragraph: _____

Results (The Findings)

Write a results paragraph that: - Reports descriptive statistics (frequencies, percentages) - Reports your inferential test result with proper APA format - Describes the pattern without interpreting it yet

Your paragraph: _____

Discussion (The Interpretation)

Write a discussion paragraph that: - Restates the main finding in plain language - Connects to your theoretical framework - Discusses practical implications or limitations

Your paragraph: _____

Exercise 14.2: Integrating Code and Narrative

Practice embedding code in narrative text. Write the Quarto syntax:

Task 1: Insert a code chunk that creates a frequency table

```
Your Quarto code:
```

Task 2: Insert a code chunk that creates a bar chart with a caption

```
Your Quarto code:
```

Task 3: Insert inline R code to report a calculated percentage

```
Your sentence: "The toxicity rate was `r _______________`%."
```

Exercise 14.3: GitHub Pages Deployment Checklist

Prepare your project for deployment by completing this checklist:

Pre-Deployment: - [ ] All chapters render without errors - [ ] Figures have descriptive captions - [ ] Tables are properly formatted - [ ] Citations are complete and properly formatted - [ ] Codebook is included as appendix - [ ] Data files are in the repository (if ethically shareable) - [ ] README.md explains the project

Deployment Steps: 1. [ ] Run `quarto render` successfully 2. [ ] Verify `_site/` directory contains HTML files 3. [ ] Commit all changes to Git 4. [ ] Push to GitHub repository 5. [ ] Enable GitHub Pages in repository settings 6. [ ] Select source: branch (main) and folder (_site) 7. [ ] Wait 2-5 minutes for deployment 8. [ ] Test the live URL

Post-Deployment: - [ ] All internal links work - [ ] Images load correctly - [ ] Navigation menu functions - [ ] Search feature works (if enabled) - [ ] Share URL on social media or portfolio

Exercise 14.4: Peer Review of Complete Project

Exchange final reports with a classmate (or self-review using these criteria):

Content Quality: - [ ] Research question is clear and focused - [ ] Literature review establishes gap - [ ] Theory application is appropriate - [ ] Methods are replicable (another researcher could follow) - [ ] Results are presented clearly (tables + text) - [ ] Discussion connects findings to theory and implications - [ ] Limitations are acknowledged honestly

Technical Quality: - [ ] Code runs without errors - [ ] Visualizations are clear and interpretable - [ ] Tables follow APA format - [ ] Statistical reporting is complete (test statistic, df, p, effect size) - [ ] Citations are properly formatted

Writing Quality: - [ ] Writing is clear and concise - [ ] No jargon without definition - [ ] Logical flow between sections - [ ] No typos or grammatical errors

Feedback for author: _____

Exercise 14.5: Final Reflection Essay

Write a 300-500 word reflection addressing:

1. **The Journey**: How did your understanding of research change from Chapter 1 to now?
2. **Challenges Overcome**: What was the hardest part of this project (conceptual, technical, or logistical)? How did you solve it?
3. **Unexpected Discoveries**: Did your findings surprise you? Did they support or challenge your initial assumptions?

4. **Skills Gained**: Which skills (R coding, writing codebooks, theory application, visualization) do you feel most confident about? Which need more development?
5. **Future Applications**: How might you use these research skills beyond this course (career, grad school, civic engagement)?

Your reflection: _____

Reflection Questions

1. **Reproducibility in Practice**: Now that you've created a fully reproducible report, how has your perception of "good research" changed? Will you be more skeptical of studies that don't share code and data?
2. **The Portfolio as Identity**: Your GitHub portfolio demonstrates competence to future employers, grad schools, or collaborators. How does this differ from traditional assessments (exams, papers locked in LMS)? What opportunities does public scholarship create?
3. **Research Ethics Revisited**: Reflect on the ethical dimensions of your project. Did you collect data responsibly? Represent findings honestly? Acknowledge limitations? What ethical dilemmas did you face?

Chapter Summary

This chapter synthesized the entire research journey into a publishable portfolio:

- The **one-click report** philosophy: Quarto documents regenerate all analysis from raw data with one command
- **IMRaD structure**: Introduction (story + gap), Methods (recipe), Results (data speak), Discussion (interpretation)
- **Introduction** components: Hook, literature review, research question/hypothesis
- **Methods** components: Data source, sample, variables (operational definitions), reliability, analytical plan
- **Results** present findings objectively with tables, figures, and descriptive/inferential statistics
- **Discussion** interprets findings theoretically, discusses practical implications, acknowledges limitations, suggests future research
- **Embedding code**: Quarto weaves narrative text and executable R code; outputs appear inline when rendered
- **GitHub Pages deployment**: Renders site locally, pushes to GitHub, enables Pages, publishes live URL

- **Narrative arc completed**: From curiosity (Ch 1) through tools (Ch 2), discipline (Ch 3), planning (Ch 4-6), execution (Ch 7-13), to publication (Ch 14)
- **Skills gained**: Epistemology, transparency, humility, communication, technical literacy, reproducible workflows

Key Terms

- **Discussion section**: IMRaD component interpreting findings, connecting to theory, and discussing implications
- **GitHub Pages**: Free hosting service for static websites directly from GitHub repositories
- **IMRaD**: Standard research paper structure (Introduction, Methods, Results, and Discussion)
- **Introduction section**: IMRaD component establishing research problem, reviewing literature, stating hypotheses
- **Limitations section**: Part of Discussion acknowledging study weaknesses and boundaries
- **Methods section**: IMRaD component documenting data, sample, variables, and analytical procedures
- **One-click report**: Reproducible document that regenerates all analysis with a single render command
- **Portfolio website**: Public demonstration of research skills, hosted online
- **Reproducible research**: Research where findings can be regenerated from raw data using shared code
- **Results section**: IMRaD component objectively presenting findings without interpretation

The End (and the Beginning)

Congratulations. You've completed the journey from vibes to variables, from intuition to evidence, from journalist to scientist.

The skills you've developed—curiosity disciplined by method, storytelling grounded in data, humility paired with rigor—are rare and valuable. Use them responsibly. Question everything, but question with evidence. Tell stories, but back them with systematic inquiry.

Research is not about having all the answers. It's about asking better questions, gathering honest evidence, and contributing to collective knowledge with transparency and humility.

Your research journey doesn't end here. It evolves. May your questions remain sharp, your methods remain sound, and your stories remain true.

Now go forth and make evidence-driven narratives.

Appendix A: R Quick Reference Guide

This quick reference provides essential R code snippets for common research tasks. Bookmark this page for rapid lookup during analysis.

Data Import and Export

```
# Read CSV file
data <- read.csv("path/to/file.csv")

# Read with tidyverse (better)
library(readr)
data <- read_csv("path/to/file.csv")

# Write CSV file
write.csv(data, "output/cleaned_data.csv", row.names = FALSE)

# Write with tidyverse
write_csv(data, "output/cleaned_data.csv")
```

Basic Data Inspection

```
# View first 6 rows
head(data)

# View last 6 rows
tail(data)

# See structure (variable types)
str(data)

# Summary statistics
```

```r
summary(data)

# Check for missing values
sum(is.na(data$variable_name))

# Count unique values
length(unique(data$variable_name))
```

Data Manipulation (dplyr)

```r
library(dplyr)

# Filter rows
data %>% filter(toxicity == "Toxic")
data %>% filter(viewers > 1000)
data %>% filter(game == "Valorant" & uses_webcam == TRUE)

# Select columns
data %>% select(id, game, toxicity)
data %>% select(-coder_notes)  # Remove column

# Create/modify columns
data %>% mutate(viewer_category = case_when(
  viewers < 100 ~ "Small",
  viewers < 1000 ~ "Medium",
  viewers >= 1000 ~ "Large"
))

# Summarize data
data %>% summarize(
  total = n(),
  avg_viewers = mean(viewers),
  median_viewers = median(viewers)
)

# Group and summarize
data %>%
  group_by(game) %>%
  summarize(
```

```r
    total_streams = n(),
    avg_viewers = mean(viewers),
    toxic_rate = sum(toxicity == "Toxic") / n()
  )

# Sort data
data %>% arrange(viewers)  # Ascending
data %>% arrange(desc(viewers))  # Descending

# Chain operations
data %>%
  filter(game %in% c("Valorant", "Overwatch")) %>%
  mutate(is_toxic = ifelse(toxicity == "Toxic", 1, 0)) %>%
  group_by(game) %>%
  summarize(toxicity_rate = mean(is_toxic)) %>%
  arrange(desc(toxicity_rate))
```

Data Visualization (ggplot2)

```r
library(ggplot2)

# Bar chart
ggplot(data, aes(x = category)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Count by Category", x = "Category", y = "Count")

# Grouped bar chart
ggplot(data, aes(x = game, fill = toxicity)) +
  geom_bar(position = "dodge") +
  labs(title = "Toxicity by Game", x = "Game", y = "Count")

# Stacked bar chart (proportions)
ggplot(data, aes(x = game, fill = toxicity)) +
  geom_bar(position = "fill") +
  labs(title = "Toxicity Proportion by Game", y = "Proportion")

# Histogram
ggplot(data, aes(x = viewers)) +
  geom_histogram(bins = 30, fill = "coral", color = "black") +
```

```r
  labs(title = "Distribution of Viewers", x = "Viewers", y = "Frequency")

# Boxplot
ggplot(data, aes(x = game, y = donations)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = "Donations by Game", x = "Game", y = "Donations ($)")

# Scatterplot
ggplot(data, aes(x = viewers, y = donations)) +
  geom_point(color = "darkgreen") +
  labs(title = "Viewers vs Donations", x = "Viewers", y = "Donations")

# Add best-fit line
ggplot(data, aes(x = viewers, y = donations)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(title = "Viewers vs Donations")
```

Inferential Statistics

```r
# Chi-Square test
chisq.test(data$game, data$toxicity)

# Save Chi-Square results
chi_result <- chisq.test(data$game, data$toxicity)
chi_result$statistic  # Chi-square value
chi_result$p.value    # P-value
chi_result$parameter  # Degrees of freedom

# Independent samples t-test
t.test(viewers ~ uses_webcam, data = data)

# One-way ANOVA
aov_result <- aov(viewers ~ game, data = data)
summary(aov_result)

# Tukey post-hoc test
TukeyHSD(aov_result)

# Pearson correlation
```

```r
cor.test(data$viewers, data$donations)

# Effect size for Chi-Square (Cramér's V)
library(effectsize)
cramers_v(data$game, data$toxicity)

# Effect size for t-test (Cohen's d)
cohens_d(viewers ~ uses_webcam, data = data)

# Effect size for ANOVA (Eta-squared)
eta_squared(aov_result)
```

String Manipulation

```r
# Convert to lowercase
data %>% mutate(game = tolower(game))

# Replace values
data %>% mutate(game = str_replace_all(game, "valorant", "Valorant"))

# Detect pattern
data %>% filter(str_detect(message, "toxic"))

# Remove whitespace
data %>% mutate(game = str_trim(game))

# Combine strings
data %>% mutate(full_name = paste(first_name, last_name, sep = " "))
```

Handling Missing Data

```r
# Remove rows with ANY missing values
data_clean <- na.omit(data)

# Remove rows where specific column is missing
data_clean <- data %>% filter(!is.na(variable_name))
```

```r
# Replace missing values with specific value
data %>% mutate(variable = ifelse(is.na(variable), 0, variable))

# Replace specific values with NA
data %>% mutate(variable = na_if(variable, "Unknown"))
```

Frequency Tables

```r
# Simple frequency table
table(data$category)

# Two-way table
table(data$game, data$toxicity)

# With proportions
prop.table(table(data$category))

# Tidyverse approach
data %>% count(category)

data %>%
  group_by(game, toxicity) %>%
  summarize(count = n()) %>%
  mutate(proportion = count / sum(count))
```

Useful Functions

```r
# Descriptive statistics
mean(data$variable)      # Average
median(data$variable)    # Median
sd(data$variable)        # Standard deviation
min(data$variable)       # Minimum
max(data$variable)       # Maximum
quantile(data$variable)  # Quartiles

# Rounding
round(3.14159, 2)  # 3.14
```

```r
# Count observations
nrow(data)  # Number of rows
ncol(data)  # Number of columns
dim(data)   # Both

# Check for duplicates
data %>% distinct()  # Remove duplicates

# Conditional logic
ifelse(condition, value_if_true, value_if_false)

# Case when (multiple conditions)
case_when(
  variable < 10 ~ "Low",
  variable < 50 ~ "Medium",
  variable >= 50 ~ "High"
)
```

Getting Help

```r
# Help documentation
?function_name
help(function_name)

# Search for function
??topic

# See examples
example(function_name)

# View vignette
vignette("dplyr")
```

Common Operators

Operator	Meaning
`<-`	Assignment
`==`	Equal to
`!=`	Not equal to
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal
`<=`	Less than or equal
`&`	AND (both conditions true)
`|`	OR (either condition true)
`!`	NOT (negation)
`%>%`	Pipe (pass to next function)
`%in%`	Match within vector
`+`	Add layers (ggplot2)

Keyboard Shortcuts (RStudio)

Action	Windows/Linux	Mac
Insert `<-`	Alt + -	Option + -
Insert `%>%`	Ctrl + Shift + M	Cmd + Shift + M
Run current line/selection	Ctrl + Enter	Cmd + Return
Comment/Uncomment	Ctrl + Shift + C	Cmd + Shift + C
Knit document	Ctrl + Shift + K	Cmd + Shift + K
Help on function	F1 (with cursor on function)	F1

Pro tip: Keep this appendix open in a second browser tab while coding. Copy-paste and modify code rather than typing from scratch.

Appendix B: Troubleshooting Common Errors

This appendix addresses the most common technical problems students encounter. When you get stuck, check here first before asking for help.

R and RStudio Errors

Error: "could not find function"

Problem: R says a function doesn't exist.

```
Error in filter(data, game == "Valorant") :
  could not find function "filter"
```

Cause: You haven't loaded the package containing the function.

Solution: Load the package first.

```
library(dplyr)  # filter() comes from dplyr
filter(data, game == "Valorant")
```

Pro tip: Always load tidyverse at the start of your script:

```
library(tidyverse)  # Loads dplyr, ggplot2, readr, tidyr, etc.
```

Error: "object not found"

Problem: R says a variable or dataset doesn't exist.

```
Error: object 'my_data' not found
```

Cause: Either: 1. You haven't created the object yet 2. You made a typo (R is case-sensitive: `my_data` `My_Data`) 3. You're in a different R session than where you created it

Solution: - Check spelling and capitalization - Re-run the line that creates the object - Check your Environment pane (top right) to see what objects exist

---

Error: "unexpected symbol" or "unexpected ',' "

Problem: Syntax error in your code.

```
Error: unexpected symbol in "data <- read.csv file.csv"
```

Cause: Missing parentheses, quotes, or commas.

Solution: Check: - Are strings in quotes? `"file.csv"` not `file.csv` - Are function calls complete? `read.csv("file.csv")` not `read.csv "file.csv"` - Are all parentheses balanced? Every `(` needs a `)`

Error: "cannot open file"

Problem: R can't find your data file.

```
Error in file(file, "rt") : cannot open the connection
In addition: Warning message:
In file(file, "rt") :
  cannot open file 'data.csv': No such file or directory
```

Cause: Wrong file path or working directory.

Solution:

```r
# Check your working directory
getwd()

# Set working directory to your project folder
setwd("C:/Users/YourName/Documents/research-project")

# OR use relative paths from project root
read.csv("data/raw/my_data.csv")

# BEST: Use here package
library(here)
data <- read_csv(here("data", "raw", "my_data.csv"))
```

Pro tip: Use RStudio Projects (.Rproj files) to automatically set working directory.

Error: "non-numeric argument to binary operator"

Problem: Trying to do math on non-numeric data.

```
Error in data$age + 5 : non-numeric argument to binary operator
```

Cause: Variable is stored as text ("25") instead of number (25).

Solution: Convert to numeric.

```r
data <- data %>%
  mutate(age = as.numeric(age))
```

Warning: "NAs introduced by coercion"

Problem: R couldn't convert some values to numbers.

```
Warning message:
NAs introduced by coercion
```

Cause: Your column has text mixed with numbers (e.g., "Unknown" in an age column).

Solution: Clean data first, then convert.

```
data <- data %>%
  mutate(age = ifelse(age == "Unknown", NA, age)) %>%
  mutate(age = as.numeric(age))
```

Error: "subscript out of bounds"

Problem: Trying to access a column/row that doesn't exist.

```
Error in data[, 15] : subscript out of bounds
```

Cause: Your data has fewer than 15 columns.

Solution: Check dimensions.

```
dim(data)  # Shows rows and columns
ncol(data)  # Number of columns
```

Quarto Rendering Errors

Error: "failed to compile"

Problem: Quarto can't render your document.

Cause: Usually an R code error in one of your chunks.

Solution: 1. Look for the chunk number in error message 2. Run that chunk manually in Console 3. Fix the error 4. Try rendering again

Pro tip: Add `error=TRUE` to chunks during development:

```
::: {.cell}

```{.r .cell-code}
# This allows document to render even if chunk has errors
broken_code_here
```

::: {.cell-output .cell-output-error}

```
Error: object 'broken_code_here' not found
```


:::
:::
```

Error: "object not found" (but code works in Console)

Problem: Code runs fine when you test it, but fails during rendering.

Cause: Each Quarto render starts with a fresh R session—it doesn't "remember" objects from your console.

Solution: Make sure ALL necessary code is in your .qmd file, in order:

```r
# WRONG: Assume data is already loaded
ggplot(data, aes(x = game)) + geom_bar()

# RIGHT: Load data in the document
library(tidyverse)
data <- read_csv("data/cleaned_data.csv")
ggplot(data, aes(x = game)) + geom_bar()
```

Error: "LaTeX Error" or "PDF rendering failed"

Problem: Can't render to PDF.

Cause: Missing LaTeX installation.

Solution: Install TinyTeX (lightweight LaTeX).

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Alternative: Render to HTML instead.

```
format:
  html:
    toc: true
```

---

ggplot2 Errors

Error: "Aesthetics must be either length 1 or the same as the data"

Problem: Variable mismatch in ggplot.

Cause: You're referencing a variable not in your data frame.

Solution: Check variable names.

```
# Check what variables exist
names(data)

# Make sure spelling matches
ggplot(data, aes(x = game)) +  # Not "Game" or "games"
  geom_bar()
```

Error: "Don't know how to automatically pick scale for object of type"

Problem: ggplot doesn't know how to plot your variable type.

Cause: Variable is wrong data type (e.g., trying to plot a list).

Solution: Convert to appropriate type.

```
# If variable should be categorical
data <- data %>% mutate(game = as.factor(game))

# If variable should be numeric
data <- data %>% mutate(viewers = as.numeric(viewers))
```

Error: "Discrete value supplied to continuous scale"

Problem: Using the wrong scale for your variable.

```
Error: Discrete value supplied to continuous scale
```

Cause: Treating categorical variable as numeric (or vice versa).

Solution: Use correct geom and scale.

```
# For categorical x-axis
ggplot(data, aes(x = game, y = viewers)) +
  geom_col()  # Not geom_line()

# For continuous x-axis
ggplot(data, aes(x = stream_hours, y = viewers)) +
  geom_point() +
  geom_smooth()
```

Statistical Test Errors

Warning: "Chi-squared approximation may be incorrect"

Problem: Chi-Square test may be unreliable.

Cause: Expected cell counts are too small (< 5 in any cell).

Solution: 1. Collect more data, OR 2. Combine categories, OR 3. Use Fisher's Exact Test instead

```
fisher.test(data$game, data$toxicity)
```

Error: "data are essentially constant"

Problem: No variation in your variable.

Cause: All values are the same (e.g., all viewers = 1000).

Solution: This is a data problem, not a code problem. Check your data source.

```
# Check for variation
summary(data$variable)
```

Git/GitHub Errors

Error: "failed to push some refs"

Problem: GitHub rejects your push.

Cause: Remote repository has commits you don't have locally.

Solution: Pull first, then push.

```
git pull origin main
git push origin main
```

Error: "merge conflict"

Problem: Same file edited in two places.

Cause: You edited locally and on GitHub (or collaborator edited).

Solution: Open conflicted file, look for:

```
<<<<<<< HEAD
Your version
=======
Other version
>>>>>>> branch_name
```

Choose which to keep, delete conflict markers, save, then:

```
git add .
git commit -m "Resolved merge conflict"
git push
```

General Troubleshooting Strategy

1. **Read the error message carefully** - It often tells you exactly what's wrong
2. **Check for typos** - R is case-sensitive: `Data` ≠ `data`
3. **Run code line-by-line** - Find exactly which line fails
4. **Google the error** - Copy the exact error message into Google
5. **Check Stack Overflow** - Usually someone has had your exact problem
6. **Restart R** - Sometimes helps: Session → Restart R
7. **Ask for help** - But show:
   - The code that failed
   - The complete error message
   - What you've already tried

Getting Help Resources

- **RStudio Cheat Sheets**: Help → Cheat Sheets
- **R Documentation**: `?function_name` or `help(function_name)`
- **Stack Overflow**: https://stackoverflow.com/questions/tagged/r
- **RStudio Community**: https://community.rstudio.com/
- **Tidyverse Website**: https://www.tidyverse.org/
- **ChatGPT/AI**: Good for explaining errors (but verify solutions)

Prevention is Better Than Cure

Best practices to avoid errors:

1. **Use R Projects** - Keeps file paths consistent
2. **Load packages at top of script** - `library(tidyverse)` goes first
3. **Test code incrementally** - Don't write 100 lines then run
4. **Comment your code** - Future you will thank you
5. **Use consistent naming** - `snake_case` for everything
6. **Keep raw data untouched** - Work on copies
7. **Commit often** - Git saves you when things break
8. **Read documentation** - `?function_name` before using new functions

Remember: Every expert was once a beginner who refused to give up. Errors are learning opportunities, not failures.

Appendix C: Sample Codebook Template

Use this template to build your own codebook for content analysis. A complete codebook ensures reliable coding and transparent methods.

Project Title

Your Study Title Here

Researcher(s): Your Name
Date Created: MM/DD/YYYY
Last Updated: MM/DD/YYYY

Study Overview

Research Question: [State your main research question]

Population: [Describe the population you're studying - e.g., "Twitch chat messages from competitive gaming streams"]

Sample: [Describe your sample - e.g., "200 messages randomly sampled from 10 Valorant streams during ranked matches"]

Unit of Analysis: [What are you coding? - e.g., "Individual chat message"]

Coding Procedure

Step 1: Data Preparation

[Explain how data is organized before coding]

Example: "Messages are provided in a CSV file with columns: message_id, streamer_name, timestamp, message_text. Coders will add columns for each variable below."

Step 2: Coder Training

[Describe training process]

Example: "Coders will independently code a pilot sample of 50 messages, then meet to discuss disagreements and refine definitions. Inter-coder reliability (Krippendorff's Alpha) must reach .80 before proceeding to full sample."

Step 3: Coding Rules

[General guidelines]

Example: - Code based only on information in the message itself, not external knowledge - If unsure, code as "Unclear" and flag for discussion - Never leave a cell blank—every message must receive a code for every variable

---

Variable 1: [Variable Name]

Conceptual Definition: [What does this concept mean in theory?]

Operational Definition: [How will you identify and measure it in practice?]

Level of Measurement: [Nominal / Ordinal / Interval / Ratio]

Categories:

Category 1: [Name]

Definition: [Precise definition]

Examples: - "Example text 1" → CODE AS: Category 1 - "Example text 2" → CODE AS: Category 1

Non-examples (what this is NOT): - "Ambiguous text" → CODE AS: Category 2 (explain why)

Category 2: [Name]

Definition: [Precise definition]

Examples: - [Provide 2-3 examples]

Non-examples: - [Provide counterexamples]

Category 3: Unclear/Other

Definition: Use only when message does not clearly fit other categories

Decision Rule: If you cannot confidently assign to Category 1 or 2 after 30 seconds of consideration, code as "Unclear."

Variable 2: [Variable Name]

[Repeat the structure above for each variable]

Decision Rules for Edge Cases

Edge Case 1: [Describe situation]

Example: "What if a message contains both positive and negative content?"

Rule: "Code based on dominant sentiment. If equal, code as 'Mixed.' "

Example: - "GG but you got lucky lol" → CODE AS: Mixed (balanced positive and negative) - "GG you played great even though you lost" → CODE AS: Positive (dominant sentiment)

Edge Case 2: [Describe situation]

Example: "What if a message uses sarcasm?"

Rule: "Code the literal meaning, not inferred intent, unless context makes sarcasm unambiguous."

Example: - "Oh wow, great play " → CODE AS: Negative (eye-roll emoji signals sarcasm) - "Wow great play!" → CODE AS: Positive (no sarcasm markers)

Edge Case 3: Emojis, Emotes, and Non-Text

Rule: [How to handle visual elements]

Example: "Twitch emotes (e.g., KEKW, PogChamp) are coded based on their conventional meaning in Twitch culture. See Appendix D for emote reference guide."

Sample Coding Sheet

message_id	message_text	Variable_1	Variable_2	Notes
001	"GG nice try"	Neutral	Supportive	Standard sportsmanship
002	"u suck noob"	Toxic	Negative	Personal insult

message_id	message_text	Variable_1	Variable_2	Notes
003	"lol that lag was awful"	Neutral	Negative	Directed at game, not person
004	"KEKW"	Unclear	Neutral	Emote only, no clear target

Inter-Coder Reliability

Method: [Krippendorff's Alpha / Cohen's Kappa / Percent Agreement]

Threshold: .80 (acceptable)

Pilot Results: - Variable 1: = [value] - Variable 2: = [value]

Discrepancies: [How were disagreements resolved?]

Example: "Coders disagreed on 12% of messages in pilot. After discussion, 'trash talk among friends' was redefined to require explicit reciprocity markers (e.g., usernames, prior interaction). Revised = .85."

Codebook Revision History

Date	Change	Rationale
10/15/2024	Added "Sarcasm" edge case rule	Pilot testing revealed 15% of messages used sarcasm
10/22/2024	Revised "Toxic" to exclude profanity without personal attacks	Inter-coder reliability was .65; coders disagreed on swearing

---

References

[Cite theoretical sources that informed your variables and any validated scales you used]

Example: - Schramm, H., & Hartmann, T. (2008). The PSI-Process Scales. *Communication Research Reports*, 25(1), 30-43. - Massanari, A. (2017). #Gamergate and The Fappening. *Feminist Media Studies*, 17(4), 512-533.

Appendices

Appendix D: Twitch Emote Reference

Emote	Meaning	Valence
KEKW	Laughter	Neutral/Positive
PogChamp	Excitement/Hype	Positive
BibleThump	Sadness	Negative
4Head	Mockery	Negative

Notes for Future Researchers

[Any insights, warnings, or recommendations for others using this codebook]

Example: "Be cautious when coding messages with gamer slang. Terms like 'rekt' and 'pwned' are not inherently toxic—they're standard competitive language. Context is critical: 'You got rekt GG' (neutral) vs. 'Get rekt trash' (toxic)."

Codebook Approval:

- **Primary Researcher**: [Name, Date, Signature]
- **Coder 1**: [Name, Date, Signature]
- **Coder 2**: [Name, Date, Signature]
- **Faculty Advisor** (if applicable): [Name, Date, Signature]

Quick Coding Checklist

Before submitting coded data, verify:

- ☐ Every row has a code for every variable (no blanks)
- ☐ All codes match defined categories (no typos or improvised codes)
- ☐ "Unclear" category used <10% of the time (if more, codebook needs refinement)
- ☐ Notes column used for genuinely ambiguous cases
- ☐ Coder ID recorded (if multiple coders)
- ☐ Data saved with date: `coded_data_2024-10-25.csv`

Remember: The codebook is a living document. Update it as you discover new edge cases during coding. Clear documentation = reliable data = trustworthy findings.

Appendix D: Data Management Best Practices

Proper data management prevents disasters. This appendix provides essential practices for organizing, backing up, and documenting your research data.

The Three Rules of Data Management

1. **Never modify raw data** - Always work on copies
2. **Document everything** - Future you needs to understand past you's decisions
3. **Back up obsessively** - Assume your computer will crash the day before the deadline

---

Project Organization Structure

Use this folder structure for every research project:

```
my-research-project/
   README.md                 # Project overview
   data/
       raw/                  # Original, untouched data
           chat_logs.csv
           metadata.txt
       processed/            # Cleaned, analysis-ready data
           cleaned_data.csv
           coding_complete.csv
       backup/               # Dated backups
            backup_2024-10-15.csv
            backup_2024-10-22.csv
   code/
       01_data_cleaning.R    # Numbered scripts (run in order)
       02_descriptives.R
       03_inferential.R
```

```
    archive/              # Old versions
output/
    figures/              # Plots and visualizations
        fig1_toxicity_by_game.png
        fig2_viewers_distribution.png
    tables/               # Summary tables
        table1_descriptives.csv
    reports/              # Rendered documents
         analysis_report.html
         final_report.pdf
documents/
    codebook.qmd          # Your codebook
    literature_notes.qmd  # Literature review notes
    methods_documentation.qmd
project.Rproj              # RStudio project file
```

File Naming Conventions

Good file names are: - **Descriptive**: `fig1_toxicity_by_game.png` not `plot.png` - **Dated** (when appropriate): `coded_data_2024-10-25.csv` - **Machine-readable**: Use `snake_case` or `kebab-case`, not spaces - **Versioned** (if iterating): `analysis_v1.R`, `analysis_v2.R`

Bad:

```
final.csv
final_FINAL.csv
final_final_USE_THIS.csv
```

Good:

```
cleaned_data_2024-10-15.csv
cleaned_data_2024-10-22.csv  # Date shows which is newest
```

Raw Data Protection

Rule: NEVER Edit Raw Data Directly

Why? If you make a mistake, you've lost the original. Always work on copies.

Workflow:

1. **Store original in `data/raw/`**
2. **Make a copy for cleaning**:

```r
library(readr)
raw_data <- read_csv("data/raw/original_data.csv")

# All cleaning happens in R code, not Excel
clean_data <- raw_data %>%
  filter(!is.na(message)) %>%
  mutate(game = tolower(game))

# Save cleaned version
write_csv(clean_data, "data/processed/cleaned_data.csv")
```

3. **Raw data stays pristine** - If cleaning goes wrong, you can start over

Never: - Open raw CSV in Excel and save changes - Manually delete rows from raw data - "Fix" raw data without documenting what you changed

---

Backup Strategy

3-2-1 Rule

- **3 copies** of your data
- **2 different media** (local drive + cloud)
- **1 offsite** (not in your house/office)

Implementation:

1. **Local copy**: On your computer (`data/` folder)
2. **Cloud backup**: OneDrive, Google Drive, or Dropbox
3. **Version control**: GitHub (for code and small data files)

Automated Backups

For critical data (e.g., 6 weeks of coding work):

```
# Add to your script - saves dated backup automatically
backup_filename <- paste0("data/backup/coded_data_", Sys.Date(), ".csv")
write_csv(coded_data, backup_filename)
```

Or use scheduled cloud sync: - OneDrive: Auto-syncs entire project folder - Google Drive: Use Google Backup and Sync - Dropbox: Keeps 30 days of version history

---

Documentation Requirements

README.md (Project Root)

Every project needs a README explaining what it is and how to use it.

Template:

```
# Project Title

## Author
Your Name
Course: COM 400
Semester: Fall 2024

## Research Question
What is the relationship between streamer webcam use and viewer donations on Twitch?

## Data Sources
- Twitch API (via instructor-provided CSV)
- Manual coding of 200 streams
- Date range: September 1-30, 2024

## File Organization
- `data/raw/` - Original API output
- `data/processed/` - Cleaned and coded data
- `code/` - Analysis scripts (run in numbered order)
- `output/` - Figures and tables for final report
```

```
## How to Reproduce This Analysis
1. Open `project.Rproj` in RStudio
2. Install required packages: `install.packages(c("tidyverse", "effectsize"))`
3. Run scripts in order:
   - `01_data_cleaning.R`
   - `02_descriptives.R`
   - `03_inferential.R`
4. Final report: Render `analysis_report.qmd`

## Notes
- Inter-coder reliability (Krippendorff's Alpha) = .87
- One stream removed due to moderation issues (ID: stream_045)
- See `documents/codebook.qmd` for variable definitions

## Contact
your.email@example.com
```

Data Documentation (data/README.md)

```
# Data Documentation

## raw/chat_logs.csv
- **Source**: Twitch API via instructor (Dr. Smith)
- **Date collected**: September 1-30, 2024
- **Rows**: 10,000 messages from 50 streams
- **Columns**:
  - `message_id`: Unique identifier (1-10000)
  - `stream_id`: Stream identifier (stream_001 - stream_050)
  - `timestamp`: UTC timestamp
  - `username`: Anonymized (user_001 - user_XXXX)
  - `message_text`: Raw chat message
  - `emotes`: Twitch emotes used (comma-separated)

## processed/cleaned_data.csv
- **Created**: October 15, 2024
- **Script**: `code/01_data_cleaning.R`
- **Changes from raw**:
  - Removed 237 bot messages (username pattern: "^nightbot|^streamelements")
  - Standardized game names (see codebook)
  - Added `toxicity` column (hand-coded)
```

```
  - Added `message_length` (character count)

## processed/coding_complete.csv
- **Created**: October 25, 2024
- **Coders**: Yourself, Partner Name
- **ICR**:   = .87
- **New variables**:
  - `toxicity`: (Neutral / Toxic / Unclear)
  - `target`: (Streamer / Other_user / Game / None)
  - `coder_id`: (coder_1 / coder_2)
```

Version Control with Git

Basic Git Workflow

```
# Initial setup (do once)
git init
git add .
git commit -m "Initial commit"

# After each work session
git add .
git commit -m "Descriptive message about what changed"
git push origin main
```

What to Commit

DO commit: - All code files (`.R`, `.qmd`, `.Rmd`) - Codebooks and documentation (`.md`, `.qmd`) - Small data files (< 50 MB) - Figures and tables (`.png`, `.pdf`)

DON'T commit: - Large raw data files (> 50 MB) - use cloud storage instead - Sensitive data (personally identifiable information) - Temporary files (`.Rhistory`, `.RData`) - Output that can be regenerated from code

.gitignore File

Create `.gitignore` in project root:

```
# R files
.Rhistory
.RData
.Rproj.user/

# Large data files
data/raw/*.csv
data/processed/*.csv

# Temporary files
*.tmp
~$*.xlsx

# Output (can be regenerated)
output/figures/
output/tables/
_site/
```

Data Security and Ethics

Sensitive Data

If your data contains: - Real usernames - Email addresses - IP addresses - Other personally identifiable information (PII)

Then you must:

1. **Anonymize** before analysis:

   ```r
   # Replace real usernames with IDs
   data <- data %>%
     mutate(user_id = paste0("user_", row_number())) %>%
     select(-real_username)  # Remove original
   ```

2. **Store securely**:

   - Encrypted cloud storage (OneDrive with 2FA)

- Password-protected local folder
- NEVER upload to public GitHub

3. **Delete after retention period**:

   - Academic research: typically 3-7 years
   - Check IRB requirements
   - Document destruction date

Data Sharing

When submitting final projects, include: - Cleaned, anonymized data - All code (fully documented) - Codebook - README with reproduction instructions

Don't include: - Raw data with PII - Passwords or API keys - Large files (link to cloud storage instead)

Common Data Disasters (and How to Avoid Them)

Disaster 1: "I accidentally deleted my data"

Prevention: - Keep raw data read-only (don't open in Excel) - Use version control (Git) - Regular backups to cloud

Recovery: - Check `data/backup/` folder - Check cloud storage version history - Check Git history: `git checkout <old_commit> data/file.csv`

Disaster 2: "My computer crashed and I lost 3 weeks of work"

Prevention: - Commit to Git daily - Auto-sync project folder to OneDrive/Google Drive - Save dated backups weekly

Recovery: - Pull from GitHub: `git pull origin main` - Restore from cloud backup

Disaster 3: "I can't remember what this variable means"

Prevention: - Document variables in codebook AS YOU CREATE THEM - Use descriptive variable names (`toxicity_binary` not `x1`) - Comment your code

Recovery: - Check Git commit messages - Review original research plan - Ask collaborators - Re-read source literature

Disaster 4: "My analysis results changed and I don't know why"

Prevention: - Never modify cleaned data directly—make new versions - Use version control to track changes - Comment code explaining logic

Recovery: - Compare current vs. old data files - Check Git diff: `git diff <old_commit> <new_commit> code/analysis.R` - Re-run cleaning script from raw data

---

Quality Checks Before Finalizing Data

Run these checks before analysis:

```
# 1. Check for duplicates
sum(duplicated(data$message_id))  # Should be 0

# 2. Check for missing values
summary(data)  # Look for NA counts

# 3. Check variable ranges
summary(data$viewers)  # Min/Max reasonable?

# 4. Check category consistency
table(data$toxicity)  # Only expected values?

# 5. Check sample size
nrow(data)  # Matches research plan?

# 6. Check data types
str(data)  # Characters vs. factors vs. numeric correct?
```

Checklist: Before Submitting Final Project

- ☐ Raw data backed up (cloud + local)
- ☐ README.md explains project structure
- ☐ Codebook documents all variables
- ☐ Data cleaning script runs without errors
- ☐ All code commented and organized
- ☐ No personally identifiable information in shared files
- ☐ Git repository up to date
- ☐ Figures saved at publication quality (300 DPI)
- ☐ Final report renders without errors
- ☐ Collaborators credited appropriately

Remember: An hour spent organizing now saves ten hours of panic later. Treat data like your house keys—you need to know where they are at all times.